

ACM / ICPC ALGORITHM NOTEBOOK

算法竞赛模板

BY **CirnoNine**

01 数论

02 计算几何

03 数据结构

04 图论与网络流

05 多项式

06 杂项

目录

数论	4	多项式	86
1. 组合数与 $O(1)$ 逆元	5	1. 连续点拉格朗日插值	87
2. Barrett 约简	7	2. FFT	88
3. 基于值域预处理的 GCD	8	3. NTT: 基础版	89
4. EXGCD 与 EXCRT	10	4. NTT: 卡常版	90
5. Pohlig–Hellman	11	5. MTT 任意模卷积	93
6. 二次剩余: Cipolla	12	6. 二元 NTT	94
7. 常用狄利克雷卷积等式	13	7. 二元多项式卷积	95
8. 杜教筛	14	8. 任意因子长度 DFT	96
9. Meissel–Lehmer 质数计数	16	9. FWT: OR、AND、XOR 卷积	98
10. Pollard–Rho 质因数分解	18	10. 多项式乘法逆	100
11. Min25 筛	20	11. 稀疏多项式除法	101
12. 类欧几里得算法	22	12. 多项式加法与减法	102
13. 万能欧几里得算法	23	13. 多项式整除与取模	103
计算几何	24	14. 多项式对数与指数	104
1. 点与凸包	25	15. 多项式平方根	106
2. 直线	27	16. 多项式快速幂	107
- 直线与基本判定	27	17. 拉格朗日反演	108
- 投影与距离	27	18. 单位根反演	109
- 交点与线段相交	28	19. 分式第 N 项	110
3. 任意多边形	29	20. 多点求值	111
- 多边形面积	29	21. 点值平移	113
- 点与多边形关系	29	22. 快速阶乘	114
4. 凸多边形	30	23. Power Projection: 基础版	115
- 环形极值	30	24. Power Projection: 卡常版	116
- 点与凸多边形关系	31	25. 最短递推式	119
- 直线、线段与凸多边形求交	31	26. 常系数齐次线性递推	121
- 凸多边形直径	33	杂项	122
- 闵可夫斯基和	34	1. i128 输入输出与常用函数重载	123
5. 圆	35	2. Python datetime 库	125
- 外接圆	35	3. Python Fraction 库	127
- 圆与交点	35	4. Python math 库常用函数	129
- 切点与公切线	36		
6. 半平面交	37		
7. Delaunay 三角剖分	38		
数据结构	43		
1. 树状数组	44		
2. 线段树	45		
3. Treap	48		
4. pbds 平衡树	52		
5. pbds 配对堆	53		
6. 树链剖分	54		
7. Link–Cut Tree	56		
8. 后缀自动机	59		
图论与网络流	61		
1. 强连通分量	62		
2. 边双连通分量	64		
3. 点双连通分量	66		
4. 2-SAT	68		
5. Dinic 最大流	70		
6. HLPP 最高标号预流推进	72		
7. 有上下界流	75		
8. 最小费用流	77		
9. 有上下界最小费用流	80		
10. 二分图最大权匹配	82		
11. 图匹配与二分图常用性质	84		

比赛基础骨架

org.cpp 中的 `int` 实际为 `long long`；其他常用别名与辅助函数按比赛现场代码库补充。

```
#include <bits/stdc++.h>
using namespace std;

#define int long long
#define pii array<int,2>
#define endl "\n"

void solve() {

}

signed main() {
    ios::sync_with_stdio(0);
    cin.tie(0);
    int t = 1;
    cin >> t;
    for (int i = 1; i <= t; i++) {
        solve();
    }
    return 0;
}
```

顺序	模板	用途	主要复杂度
01	组合数与 $O(1)$ 逆元	阶乘、逆阶乘、逆元、组合数	预处理约 $O(n + B^2)$, 查询 $O(1)$
02	Barrett 约简	运行时模数快速取模	单次期望 $O(1)$
03	值域预处理 GCD	固定值域的大量 GCD 询问	预处理 $O(V + B^2)$, 查询 $O(1)$
04	EXGCD 与 EXCRT	Bézout 系数、非互质 CRT	每次合并对数级
05	Pohlig–Hellman	光滑群阶上的离散对数	$O(\log(mod) \sum p_i e_i)$
06	Cipolla 二次剩余	奇质数模平方根	期望 $O(\log mod)$
07	常用狄利克雷卷积等式	莫比乌斯反演与筛法换式	—
08	杜教筛	积性函数前缀和	常用配置 $O(n^{2/3})$
09	Meissel–Lehmer	质数计数	粗略 $O(n^{2/3})$
10	Pollard–Rho	64 位素性判定与分解	分解期望 $O(n^{1/4})$
11	Min_25 筛	特定积性函数前缀和	常见 $O(n^{3/4} / \log n)$
12	类欧几里得	线性式下取整求和	$O(\log \max(a, c))$
13	万能欧几里得	下取整序列的矩阵统计	$O(M^3 \log L)$

组合数与 $O(1)$ 逆元

用途： 在固定质数模数下预处理阶乘、逆阶乘，并在常数时间内计算逆元与组合数。

复杂度： 初始化约为 $O(n + B^2)$ 量级，空间复杂度 $O(n + B^2)$ ；单次 `inv`、`fac`、`ifac` 和组合数查询均为 $O(1)$ 。

使用条件： 当前参数针对 `mod = 998244353`、`B = 1024`；组合数参数需要落在预处理范围内。

```
const int mod = 998244353, N = 1e6;

int qpow(int a, int n) {
    a %= mod;
    int ans = 1;
    while (n) {
        if (n%2) ans = ans*a%mod;
        a = a*a%mod;
        n/=2;
    }
    return ans;
}

namespace Combination {
    struct Comb {
        const int B = 1024; // mod^(1/3)
        vector<int> fac, ifac;
        vector<int> INV, U;

        int inv(int x) {
            x %= mod;
            int u = U[x/B];
            int tmp = u*x%mod <= B*B*2 ? INV[u*x%mod] : mod-INV[-(u*x%mod-mod)];
            return u*tmp%mod;
        }
    };

    Comb(int n = 0) {
        if (n == 0) return;
        INV.resize(B*B*2+1);
        INV[1] = 1;
        for (int i = 2; i <= B*B*2; i++) {
            INV[i] = (mod-mod/i*INV[mod%i]%mod)%mod;
        }
        U.resize(B*B+1, 1);
        if (mod-B*B > 0) {
            int R = mod-B*B;
            for (int u = 1; u <= B; u++) {
                int tmp = u*B;
                for (int a = 0; a <= B*B; a++) {
                    int cur = tmp*a%mod;
                    if (cur <= B*B) U[a] = u;
                    else if (cur >= R) U[a] = mod-u;
                    else {

```

```

        int A = (R - cur) / tmp;
        a += A;
    }
    a++;
}
}

fac.resize(n+1);
ifac.resize(n+1);
fac[0] = 1;
for (int i = 1; i <= n; i++) {
    fac[i] = (int)fac[i-1]*i%mod;
}
ifac[n] = inv(fac[n]);
for (int i = n-1; i >= 0; i--) {
    ifac[i] = (int)ifac[i+1]*(i+1)%mod;
}
}

int operator()(int n, int k) {
    if (k < 0 || k > n) return 0;
    return (int)fac[n]*ifac[k]%mod*ifac[n-k]%mod;
}
}comb(N);
int fac(int n) {return comb.fac[n];}
int ifac(int n) {return comb.ifac[n];}
int inv(int n) {return comb.inv(n);}
}

using namespace Combination;

```

Barrett 约简

用途： 对运行时确定的正模数 m ，快速计算 $i64$ 整数 a 的有符号余数 $a \% m$ ；用一次高位乘法近似商，从而替代频繁的整数除法。

复杂度： 构造与单次取模均为 $O(1)$ ，额外空间 $O(1)$ 。

使用条件： `int`、`ull` 分别为 $i64$ 、 $u64$ ， $1 \leq m \leq 2^{63} - 1$ ，并已定义 `i128`、`u128`。负数结果与 C++ 原生 `%` 一致；`-(i128)a` 用于避免 a 为 $i64$ 最小值时取反溢出，`B*x` 使用 `u128`。

```
struct Barrett {
    int m;
    u128 B;
    Barrett(int m = 2) : m(m), B((u128(1) << 64) / m) {}
    friend int operator%(int a, const Barrett &mod) {
        ull x = a < 0 ? -(i128)a : a;
        ull q = (ull)(mod.B*x >> 64);
        ull r = x-q*mod.m;
        if (r >= (ull)mod.m) r -= mod.m;
        int ans = (int)r;
        return a < 0 ? -ans : ans;
    }
    friend int operator%=(int &a, const Barrett &mod) { return a = a%mod; }
    friend int operator+(const int a, const Barrett &mod) { return a + mod.m; }
    friend int operator-(const int a, const Barrett &mod) { return a - mod.m; }
    friend int operator-(const Barrett &mod, const int a) { return mod.m - a; }
    friend int operator/(const Barrett &mod, const int a) { return mod.m / a; }
    friend int operator%(const Barrett &mod, const int a) { return mod.m % a; }
} mod;
```

基于值域预处理的 GCD

用途： 当大量询问中的一个参数位于固定值域 $[0, V]$ 时，预处理其三因子分解与小数 GCD 表，把单次欧几里得算法降为常数查表。

复杂度： 预处理时间 $O(V + B^2)$ 、空间 $O(V + B^2)$ ，其中 $B \approx \sqrt{V}$ ；单次查询 $O(1)$ 。

使用条件： x 必须在预处理值域 $[0, N]$ 内； y 为不等于 -2^{63} 的 $i64$ 。

```
const int N = 1e6, B = 1e3; // B = sqrt(N)
vector<signed> prime;
vector<bool> isP(N+9, 1);
array<signed, 3> FAC[N+9]; // 分解成3个<=B或质数的因子
signed GCD[B+9][B+9];
void init() {
    for (int i = 0; i <= B; i++) {
        GCD[i][0] = GCD[0][i] = i;
    }
    for (int i = 1; i <= B; i++) {
        for (int j = i; j <= B; j++) {
            GCD[i][j] = GCD[j][i] = GCD[i][j%i];
        }
    }

    FAC[1] = {1, 1, 1};
    for (int i = 2; i <= N; i++) {
        if (isP[i]) {
            prime.push_back(i);
            FAC[i] = {1, 1, (signed)i};
        }
        for (auto p : prime) {
            if (i*p > N) break;
            isP[i*p] = 0;
            FAC[i*p] = FAC[i];
            *min_element(FAC[i*p].begin(), FAC[i*p].end()) *= p;
            if (i%p == 0) {
                break;
            }
        }
    }
}

template <class T>
int gcd2(signed x, T y) {
    int tmp = y%x;
    if (isP[x]) return tmp ? 1 : x;
    return GCD[tmp][x];
}

template <class T>
int gcd(T x, T y) {
    if (y < 0) y = -y;
```

```
if (!x) return y;
int a0 = gcd2(FAC[x][0], y);
int a1 = gcd2(FAC[x][1], y/=a0);
int a2 = gcd2(FAC[x][2], y/=a1);
return a0*a1*a2;
}
```

EXGCD 与 EXCRT

用途： `exgcd(a,b,x,y)` 求方程 $ax + by = \gcd(a, b)$ 的一组特解。若得到特解 (x_0, y_0) ，记 $g = \gcd(a, b)$ ，则通解为

$$x = x_0 + k \frac{b}{g}, \quad y = y_0 - k \frac{a}{g}, \quad k \in \mathbb{Z}.$$

`exCRT(a)` 求同余方程组

$$x \equiv a_{i,1} \pmod{a_{i,0}}, \quad 0 \leq i < n$$

的最小非负解，无解返回 `-1`，空方程组返回 `0`。

复杂度： `exgcd` 为 $O(\log \min(a, b))$ ；合并 k 个同余式约为 $O\left(\sum_{i=2}^k \log \min(M_{i-1}, m_i)\right)$ ，额外空间 $O(1)$ 。

接口： 每个 `pii` 按 {模数, 余数} 存储，要求模数为正且余数在 $[0, \text{模数})$ 内。

使用条件： 合并过程中的最小公倍数需要在 `i64` 范围内； $x \cdot (d/g)$ 可能超过 `i64`，这一步需要用 `i128` 计算。

```
int exgcd(int a, int b, int &x, int &y) {
    int x1 = 1, x2 = 0, x3 = 0, x4 = 1;
    while (b) {
        int c = a / b;
        tie(x1, x2, x3, x4, a, b) = make_tuple(x3, x4, x1 - x3 * c, x2 - x4 * c, b, a - b * c);
    }
    x = x1, y = x2;
    return a;
}

// x % a[i][0] = a[i][1]
int excrt(vector<pii> &a) {
    int n = a.size();
    if (!n) return 0;
    int preA = a[0][0], preB = a[0][1];
    for (int i = 1; i < n; i++) {
        int x, y;
        int g = exgcd(preA, a[i][0], x, y);
        int d = a[i][1] - preB;
        if (d % g) return -1;
        int m = a[i][0] / g;
        x = (i128)x * (d / g) % m;
        x = (x % m + m) % m;
        preB = preA * x + preB;
        preA = m * preA;
    }
    return (preB % preA + preA) % preA;
}
```

Pohlig–Hellman

用途： 在有限域乘法群的阶可以分解为小质因子幂时求离散对数；当前版本固定求解 $3^x \equiv a \pmod{998244353}$ 。

复杂度： 当前每一位直接枚举 $0 \leq x < p$ ，约为 $O(\log(mod) \sum p_i e_i)$ 次字运算；空间 $O(\sum e_i)$ 。若最大质因子很大，通常需把枚举替换为 BSGS。

依赖： `qpow`、`exCRT`，以及 `mod-1` 的完整质因数分解。

```
const int mod = 998244353, phi = mod-1;
const int g = 3;
const pii factor[] = {{2, 23}, {7, 1}, {17, 1}};

int ln(int a) {
    vector<pii> vt;
    for (auto [p, e] : factor) {
        int pe = qpow(p, e);
        int s = 0;
        int B = qpow(g, phi / p);

        for (int i = 0, pw = 1; i < e; i++, pw *= p) {
            int A = qpow(a, phi / (p * pw));
            int C = qpow(g, s * phi / (p * pw));
            for (int x = 0; x < p; x++) {
                if (qpow(B, x) * C % mod == A) {
                    s += pw * x;
                    break;
                }
            }
        }
        vt.push_back({pe, s});
    }
    return exCRT(vt);
}
```

二次剩余：Cipolla

用途： 对质数模数 mod ，求解 $x^2 \equiv n \pmod{\text{mod}}$ 。

复杂度： 期望 $O(\log \text{mod})$ 次模乘，额外空间 $O(1)$ 。

使用条件： int 为 i64 ， $0 \leq n < \text{mod}$ ，且 mod 必须是质数； $\text{mod}=2$ 由代码特判，合数模数不适用。

值域： 当 $\text{mod} \leq 2^{31} - 1$ 时，当前 i64 乘加中间值安全。若允许更大的 mod ， qpow 中的模乘、 $a*a$ 、扩域乘法中的每个乘积及其加法都要先用 i128 计算再取模；扩域乘法的中间和可能接近 $2(\text{mod} - 1)^2$ 。

返回： 无解返回 -1 ；有解时返回两个互为相反数的根中较小的一个； $n=0$ 返回 0 。

```
mt19937_64 rng(time(0));
int mod;
bool check(int n) {
    return qpow(n, (mod-1)/2) == 1;
}

int cipolla(int n) {
    if (mod == 2) return n;
    if (!n) return 0;
    if (!check(n)) return -1;
    int a = rng()%mod;
    while (check((a*a-n+mod)%mod)) a = rng()%mod;
    int t = (mod+1)/2;
    int i2 = (a*a-n+mod)%mod;
    pii ans = {1, 0}, b = {a, 1};
    while (t) {
        if (t%2) ans = {(ans[0]*b[0]+ans[1]*b[1]%mod*i2)%mod, (ans[0]*b[1]+ans[1]*b[0])%mod};
        b = {(b[0]*b[0]+b[1]*b[1]%mod*i2)%mod, (b[0]*b[1]+b[1]*b[0])%mod};
        t/=2;
    }
    if (mod-ans[0] < ans[0]) ans[0] = mod-ans[0];
    return ans[0];
}
```

常用狄利克雷卷积等式

用途： 莫比乌斯反演、整除分块、杜教筛和积性函数求和时快速代换。

复杂度： 本页是恒等式速查表，不对应独立运行复杂度。

- 1. $\mu * 1 = \varepsilon$
- 2. $\varphi * 1 = id$
- 3. $\mu * id = \varphi$
- 4. $\sum_{d|n} \mu(d) = [n = 1]$
- 5. $\sum_{d|n} \varphi(d) = n$

杜教筛

用途： 在不能线性筛到 n 时，借助一个容易求前缀和的卷积函数，递归计算积性函数前缀和。代码给出了 Mertens 函数 $\sum_{i \leq n} \mu(i)$ 与欧拉函数前缀和 $\sum_{i \leq n} \varphi(i)$ 。

复杂度： 取预处理阈值约为 $n^{2/3}$ 时，单次求值通常记作 $O(n^{2/3})$ ；空间复杂度约为 $O(N + \sqrt{n})$ 。

值域： `sumPhi(n)`、其缓存、三角和初值与递推乘积均使用 `i128`。因为 $\sum_{i=1}^n \varphi(i) \leq n(n+1)/2$ ，当 $0 \leq n \leq 4\,294\,967\,295$ 时，可以把这些 `i128` 改回 `i64`，但须先将 n 或 $n+1$ 除以 2 再相乘；若仍直接计算 $n*(n+1)$ ，则须进一步限制 $n \leq 3\,037\,000\,499$ 。

求 $s(n) = \sum_{i=1}^n f(i)$ ， f 为积性函数。

构造积性函数 g ，使得 $f * g$ 的前缀和容易计算。

根据递推式

$$g(1)s(n) = \sum_{i=1}^n (f * g)(i) - \sum_{i=2}^n g(i)s\left(\left\lfloor \frac{n}{i} \right\rfloor\right)$$

递归计算 $s(n)$ 。

```
const int N = 2e6;

vector<int> prime;
vector<bool> isP(N+9, 1);
int mu[N+9], phi[N+9];
unordered_map<int, int> mpMu;
unordered_map<int, i128> mpPhi;

void init() {
    isP[0] = isP[1] = 0;
    mu[1] = phi[1] = 1;
    for (int i = 2; i <= N; i++) {
        if (isP[i]) {
            prime.push_back(i);
            phi[i] = i-1, mu[i] = -1;
        }
        for (auto p : prime) {
            if (i*p > N) break;
            isP[i*p] = 0;
            if (i%p == 0) {
                phi[i*p] = p*phi[i];
                break;
            }
            phi[i*p] = phi[i]*(p-1), mu[i*p] = -mu[i];
        }
    }
    for (int i = 2; i <= N; i++) {
        mu[i] += mu[i-1], phi[i] += phi[i-1];
    }
}
```

```

}

i128 sumPhi(int n) {
    if (n <= N) return phi[n];
    if (mpPhi.count(n)) return mpPhi[n];
    i128 ans = (i128)n*(n+1)/2;
    int l = 2, r;
    while (l <= n) {
        r = n/(n/l);
        ans -= sumPhi(n/l)*(r-l+1);
        l = r+1;
    }
    return mpPhi[n] = ans;
}

int sumMu(int n) {
    if (n <= N) return mu[n];
    if (mpMu.count(n)) return mpMu[n];
    int ans = 1;
    int l = 2, r;
    while (l <= n) {
        r = n/(n/l);
        ans -= sumMu(n/l)*(r-l+1);
        l = r+1;
    }
    return mpMu[n] = ans;
}

```

Meissel–Lehmer 质数计数

用途： 计算 $\pi(n)$ ，即不超过 n 的质数个数，适合 n 远大于普通筛法上限的场景。

复杂度： 当前实现粗略按 $O(n^{2/3})$ 估计，预处理空间主要是 `phi[10^6][60]`；参数配置可处理约 10^{12} 的输入。

使用条件： 需要先调用 `init()`，且质数表必须覆盖查询过程中需要的平方根范围。

$$\pi(x) = \phi(x, a) + a - 1 - \sum_{i=a+1}^b \left(\pi\left(\frac{x}{p_i}\right) - \pi(p_i) + 1 \right)$$

$$\phi(x, a) = \begin{cases} x, & a = 0, \\ \phi(x, a-1) - \phi(\lfloor x/p_a \rfloor, a-1), & a \neq 0. \end{cases}$$

```
const int N = 4e6; //根号值域上限
const double eps = 1e-6;
vector<bool> isP(N+9, 1);
vector<int> prime, pi(N+9);
signed phi[1000000][60]; //预处理
void init() {
    isP[0] = isP[1] = 0;
    for (int i = 2; i <= N; i++) {
        if (isP[i]) {
            prime.push_back(i);
        }
        for (auto p : prime) {
            if (i*p > N) break;
            isP[i*p] = 0;
            if (i%p == 0) {
                break;
            }
        }
        pi[i] = pi[i-1] + isP[i];
    }
    for (int i = 1; i < (int)1e6; i++) {
        phi[i][0] = i;
        for (int j = 1; j < 60; j++) {
            phi[i][j] = phi[i][j-1] - phi[i/prime[j-1]][j-1];
        }
    }
}
int calPhi(int n, int a) {
    if (a == 0) return n;
    if (n < (int)1e6 && a < 60) return phi[n][a];
    if (n <= N && prime[a-1] * prime[a-1] >= n) return max(0ll, pi[n]-a+1);
    return calPhi(n, a-1) - calPhi(n/prime[a-1], a-1);
}
int calPi(int n) {
```

```
if (n <= N) return pi[n];
int a = calPi(powl(n, 1./3)+eps);
int res = a-1+calPhi(n, a);
for (int i = a+1; ; i++) {
    int p = prime[i-1];
    if (p*p > n) break;
    res -= calPi(n/p)-calPi(p)+1;
}
return res;
}
```

Pollard–Rho 质因数分解

用途： 对 64 位整数进行 Miller–Rabin 素性判定，并用 Pollard–Rho 随机算法分解质因数。

复杂度： 固定底数的 Miller–Rabin 约为 $O(\log n)$ 次模乘；Pollard–Rho 分解的常用期望估计为 $O(n^{1/4})$ ，空间复杂度由递归深度与输出因子决定。

依赖： `rng`、支持三参数的 `qpow(a,b,mod)`、`i128`。模乘必须使用足够宽的中间类型。

```
const int TEST[] = {2, 325, 9375, 28178, 450775, 9780504, 1795265022};
int lowbit(int x) {return x&-x;}
bool isPrime(int n) {
    if (n < 3 || n % 2 == 0) return n == 2;
    if (n % 3 == 0) return n == 3;
    int t = __lg(lowbit(n-1));
    int u = (n-1)/(1ll<<t);
    for (auto a : TEST) {
        if (a%n == 0) continue;
        a = qpow(a, u, n); // i128
        if (a == 1) continue;
        bool flag = 0;
        for (int i = 1; i <= t; i++, a = (i128)a*a%n) {
            if (a == n-1) {
                flag = 1;
                break;
            }
        }
        if (!flag) return 0;
    }
    return 1;
}

int pollardRho(int n) {
    if (n == 4) return 2;
    int c;
    auto f = [&](int x) {
        return ((i128)x*x+c)%n;
    };
    while (1) {
        c = rng()%n;
        int i = f(0), j = f(f(0));
        for (int lim = 1; i != j; lim *= 2) {
            if (lim > 128) lim = 128;
            int g = 1;
            for (int k = 0; k < lim && i != j; k++) {
                g = (i128)g*abs(i-j)%n;
                i = f(i), j = f(f(j));
            }
            g = gcd(g, n);
            if (g == n) break;
            if (g != 1) return g;
        }
    }
}
```

```

    }
    return -1;
}

vector<int> A;
int pp(int n, int cnt = 1) {
    if (n == 1) return 1;
    if (isPrime(n)) {
        while (cnt--) A.push_back(n);
        return n;
    }
    int res = 0;
    int p = pollardRho(n);

    int c = 0;
    while (n%p == 0) n/=p, c++;

    pp(p, c*cnt), pp(n, cnt);

    return res;
}

```

Min_25 筛

用途： 求一类积性函数的前缀和，尤其适合函数在质数与质数幂处容易计算、但查询上界很大的场景。

复杂度： 常见多项式型质数取值下，时间复杂度约为 $O(n^{3/4}/\log n)$ ，空间复杂度 $O(\sqrt{n})$ ；实际复杂度取决于 $f(p^e)$ 的计算方式。

依赖： 本页使用覆盖 $\sqrt{n}$ 的一基质数表，约定 `prime[1]=2`；`call(x)` 返回 $\sum_{i=1}^x i \bmod \text{mod}$ ，其中 $x(x+1)$ 需要用 `i128` 计算。递归部分还需要按具体积性函数提供 `fpe(p,e)=f(p^e)`、质数值和 `resG` 及其前缀和 `sfp`。

先考虑一个完全积性基函数 F ，并要求它在整数上的前缀和可以快速计算。

定义

$$G_j(x) = \sum_{i=2}^x F(i) [i \text{ 是质数, 或者 } i \text{ 的最小质因子} > p_j].$$

初始 $G_0(x) = \sum_{i=2}^x F(i)$ 。

转移

$$G_j(x) = G_{j-1}(x) - F(p_j) \left(G_{j-1} \left(\left\lfloor \frac{x}{p_j} \right\rfloor \right) - \sum_{i=1}^{j-1} F(p_i) \right).$$

下面的实现只演示 $F(i) = i$ ，即筛出所有离散商 $x = \lfloor n/l \rfloor$ 对应的质数和。

```
int N = sqrtl(n);
vector<int> idx1(N+1), idx2(N+1), val(1);
int cnt = 0;
for (int l = 1, r; l <= n; l = r+1) {
    r = n/(n/l);
    int x = n/l;
    val.push_back(x);
    if (x <= N) idx1[x] = ++cnt;
    else idx2[n/x] = ++cnt;
}
auto index = [&](int x) {return x <= N ? idx1[x] : idx2[n/x];};
vector<int> g(cnt+1);
for (int k = 1; k <= cnt; k++) {
    g[k] = (call(val[k])-1+mod)%mod; // 改为 preF(val[k])-F(1)
}
int sum = 0;
for (int i = 1; i < prime.size() && prime[i] <= n/prime[i]; i++) {
    int p = prime[i];
    int fp = p%mod; // 改为 F(p)
    for (int k = 1; k <= cnt && p <= val[k]/p; k++) {
        int pre = (g[index(val[k]/p)]-sum+mod)%mod;
        g[k] = (g[k]-(i128)fp*pre%mod+mod)%mod;
    }
}
```

```
(sum += fp) %= mod;
}
```

筛完后，对每个离散商 $x = \lfloor n/l \rfloor$ ， $g[\text{index}(x)]$ 表示 $\sum_{p \leq x, p \text{ 为质数}} p \bmod \text{mod}$ 。因此当前示例可令 $\text{resG} = g$ ；在下面的通用递归中， $\text{resG}[\text{index}(x)]$ 应表示 $\sum_{p \leq x, p \text{ 为质数}} f(p) \bmod \text{mod}$ 。

定义

$$S(x, j) = \sum_{i=2}^x f(i) [i \text{ 的最小质因子} > p_j],$$

其中 $j = 0$ 表示尚未排除任何质数，且 $\text{sfp}[j]$ 表示 $\sum_{i=1}^j f(p_i)$ 。转移为

$$S(x, j) = \text{resG}(x) - \sum_{i=1}^j f(p_i) + \sum_{k=j+1}^{p_k^2 \leq x} \sum_{e=1}^{p_k^e \leq x} f(p_k^e) \left(S\left(\left\lfloor \frac{x}{p_k^e} \right\rfloor, k\right) + [e > 1] \right).$$

```
int S(int x, int j) {
    if (x <= 1 || (j && prime[j] >= x)) return 0;
    int res = (resG[index(x)] - sfp[j] + mod) % mod;
    for (int i = j+1; i < prime.size() && prime[i] <= x/prime[i]; i++) {
        int p = prime[i];
        for (int e = 1, pe = p; pe <= x; e++) {
            res = (res + (i128)fpe(p, e)
                * (S(x/pe, i) + (e != 1)) % mod) % mod;
            if (pe > x/p) break;
            pe *= p;
        }
    }
    return res;
}
```

最终答案为 $S(n, 0) + f(1)$ 。 idx1 、 idx2 的下标不超过 N ，但不同离散商有 $N + \lfloor n/(N+1) \rfloor \leq 2N$ 个，因此存放状态的 g 与 resG 必须按 $\text{cnt}+1$ 分配。

类欧几里得算法

用途： 计算 $\sum_{i=0}^n \lfloor \frac{ai+b}{c} \rfloor$ 一类含线性式下取整的和。

复杂度： 递归层数与欧几里得算法同阶，为 $O(\log \max(a, c))$ ；额外空间为递归栈 $O(\log \max(a, c))$ 。

使用条件： 当前写法按 $a, b \geq 0, c > 0$ 设计，并把答案对 `mod` 取模。

```
int call(int n) {
    return n*(n+1)/2%mod;
}

int f(int a, int b, int c, int n) {
    if (a == 0) return b/c*(n+1)%mod;
    if (a >= c || b >= c) return (f(a%c, b%c, c, n) + a/c*call(n) + b/c*(n+1)) % mod;
    int m = (a*n+b)/c;
    return (n*m%mod-f(c, c-b-1, a, m-1)+mod)%mod;
}
```

万能欧几里得算法

用途：把 $\left\lfloor \frac{ai+b}{c} \right\rfloor$ 形成的 U 、 R 操作序列压缩成矩阵乘积，从而一次维护多种统计量，而不只计算下取整之和。

复杂度：对 $M \times M$ 矩阵，约为 $O(M^3 \log L)$ ，其中 L 表示参数与区间长度的数量级；递归栈 $O(\log L)$ 。

接口：`euclid(a,b,c,n,U,R)` 表示处理 $1 \dots n$ 上的序列，矩阵乘法顺序会影响所维护状态的含义。

```
template<int M>
struct Matrix {
    array<array<int, M>, M> a;

    Matrix const operator*(const Matrix &t) const {
        Matrix res;
        int r;
        for (int i = 0; i < M; ++i)
            for (int k = 0; k < M; ++k) {
                r = a[i][k];
                for (int j = 0; j < M; ++j)
                    (res.a[i][j] += t.a[k][j] * r) %= mod;
            }
        return res;
    }

    Matrix() {
        for (int i = 0; i < M; i++) {
            for (int j = 0; j < M; j++) a[i][j] = 0;
        }
    }

    friend Matrix qpow(Matrix a, int n) {
        Matrix res;
        for (int i = 0; i < M; i++) res.a[i][i] = 1;
        while (n) {
            if (n%2) res = res*a;
            a = a*a;
            n /= 2;
        }
        return res;
    }

    // 1~n: (ax+b)/c
    friend Matrix euclid(int a, int b, int c, int n, Matrix U, Matrix R) {
        if (b >= c) return qpow(U, b/c) * euclid(a, b%c, c, n, U, R);
        if (a >= c) return euclid(a%c, b, c, n, U, qpow(U, a/c)*R);
        int m = (a*n+b)/c;
        if (!m) return qpow(R, n);
        return qpow(R, (c-b-1)/a)*U*euclid(c, (c-b-1)%a, a, m-1, R, U)*qpow(R, n-(c*m-b-1)/a);
    }
};
```

计算几何

顺序	模板	用途	复杂度
01	点与凸包	二维点、向量运算与 Andrew 凸包	$O(n \log n)$
02	直线	直线与线段的位置、距离和相交关系	$O(1)$
03	任意多边形	面积与点在多边形中的位置关系	$O(n)$
04	凸多边形	点、直线和线段判定，直径与闵可夫斯基和	$O(n \log n)$
05	圆	外接圆、交点、切点与公切线	$O(1)$
06	半平面交	求有向直线左侧半平面的交多边形	$O(n \log n)$
07	Delaunay 三角剖分	求点集的 Delaunay 三角剖分	$O(n \log n)$

点与凸包

用途： 用 `Point<T>` 表示二维点或向量，支持距离、单位化、旋转与 Andrew 凸包。`convexHull(a)` 会先删除重复点，返回逆时针顶点序列，不重复首点；边上的共线点只保留两个端点。

接口	含义	复杂度	备注
<code>p*q</code> 、 <code>p&q</code>	叉积、点积	$O(1)$	乘法及加减的中间值超出 <code>i64</code> 时改用 <code>i128</code>
<code>p+q</code> 、 <code>p-q</code>	向量加法、减法	$O(1)$	坐标加减超出 <code>i64</code> 时改用 <code>i128</code>
<code>p*k</code> 、 <code>p/k</code>	数乘、数除	$O(1)$	除数不能为 <code>0</code> ；数乘超出 <code>i64</code> 时改用 <code>i128</code>
<code>p.dis2()</code> 、 <code>p.dis()</code>	点到原点距离的平方、点到原点的距离；两点 <code>a</code> 、 <code>b</code> 的距离写作 <code>(a-b).dis()</code>	$O(1)$	坐标差、平方及平方和超出 <code>i64</code> 时改用 <code>i128</code>
<code>p.unit()</code>	单位向量	$O(1)$	只用于非零 <code>Point<double></code>
<code>p.rotate90()</code>	逆时针旋转 90°	$O(1)$	坐标取负超出 <code>i64</code> 时改用 <code>i128</code>
<code>p.rotate(a)</code>	逆时针旋转 <code>a</code> 弧度	$O(1)$	只用于 <code>Point<double></code>
<code>convexHull(a)</code>	求 Andrew 凸包	时间 $O(n \log n)$ ，空间 $O(n)$	按精确比较书写；浮点比较需自行处理，叉积中间值超出 <code>i64</code> 时改用 <code>i128</code>

```
template <class T>
struct Point {
    T x, y;

    Point(T xx = 0, T yy = 0) { x = xx, y = yy; }

    T operator*(const Point &t) const { return x * t.y - t.x * y; }
    Point operator*(const T a) const { return Point<T>(x*a, y*a); }
    Point operator/(const T a) const { return Point<T>(x/a, y/a); }
    T operator&(const Point &t) const { return x * t.x + y * t.y; }
    Point operator-(const Point &t) const { return Point(x - t.x, y - t.y); }
    Point operator+(const Point &t) const { return Point(x + t.x, y + t.y); }
    bool operator==(const Point &t) const { return x == t.x && y == t.y; }
    bool operator<(const Point &t) const { return x != t.x ? x < t.x : y < t.y; }

    T dis2() const { return x*x + y*y; }
    double dis() const { return sqrt(dis2()); }
    Point unit() const { return *this/dis(); }
    Point rotate90() const { return Point(-y, x); }
    Point rotate(const double a) const { return Point(x*cos(a)-y*sin(a), x*sin(a)+y*cos(a)); }

    friend istream &operator>>(istream &in, Point &t) { return in >> t.x >> t.y; }

    friend ostream &operator<<(ostream &out, const Point &t) { return out << "(" << t.x << ", " << t.y << ")"; }

    friend vector<Point<T>> convexHull(vector<Point> a) {
        int n = a.size();
        if (n <= 1) return a;
        sort(a.begin(), a.end());
        a.erase(unique(a.begin(), a.end()), a.end());
        n = a.size();
        vector<Point<T>> s(n+1);
        int top = -1;
```

```

    for (int i = 0; i < n; i++) {
        while (top > 0 && (s[top]-s[top-1])*(a[i]-s[top-1]) <= 0) top--;
        s[++top] = a[i];
    }
    int tmp = top;
    for (int i = n-1; i >= 0; i--) {
        while (top > tmp && (s[top]-s[top-1])*(a[i]-s[top-1]) <= 0) top--;
        s[++top] = a[i];
    }
    s.resize(top);
    return s;
}
};

```

直线

用途： 用两个端点表示直线或线段，完成方向、位置、距离与相交关系计算。

直线与基本判定

接口	含义	复杂度	备注
<code>l.angle()</code>	有向直线的方向角	$O(1)$	两端点必须不同
<code>parallel(a, b)</code> 、 <code>perpendicular(a, b)</code>	两直线平行、垂直	$O(1)$	端点不同；浮点判定需改用 <code>eps</code> ，坐标差及点积、叉积中间值超出 <code>i64</code> 时改用 <code>i128</code>
<code>onLine(p, l)</code> 、 <code>onSegment(p, l)</code>	点在直线、线段上，线段包含端点	$O(1)$	端点不同；浮点判定需改用 <code>eps</code> ，坐标差及点积、叉积中间值超出 <code>i64</code> 时改用 <code>i128</code>

```
template <class T>
struct Line {
    Point<T> s, e;
    Line(Point<T> s_ = {0, 0}, Point<T> e_ = {0, 0}) {
        s = s_, e = e_;
    }

    double angle() const {
        return atan2l((e-s).y, (e-s).x);
    }
};

template <class T>
bool parallel(const Line<T> &a, const Line<T> &b) {
    return (a.e-a.s)*(b.e-b.s) == 0;
}

template <class T>
bool perpendicular(const Line<T> &a, const Line<T> &b) {
    return ((a.e-a.s)&(b.e-b.s)) == 0;
}

template <class T>
bool onLine(const Point<T> &p, const Line<T> &l) {
    return (l.e-l.s)*(p-l.s) == 0;
}

template <class T>
bool onSegment(const Point<T> &p, const Line<T> &l) {
    return onLine(p, l) && ((p-l.s)&(p-l.e)) <= 0;
}
```

投影与距离

接口	含义	复杂度	备注
<code>project(p, l)</code>	点在直线上的投影点	$O(1)$	只用于 <code>Point<double></code> ，直线两端点必须不同
<code>disLine(p, l)</code> 、 <code>disSegment(p, l)</code>	点到直线、线段的距离	$O(1)$	只用于 <code>Point<double></code> ，直线两端点必须不同

```
Point<double> project(const Point<double> &p, const Line<double> &l) {
    auto v = l.e-l.s;
```

```

    return l.s+v*((v&(p-l.s))/v.dis2());
}

double disLine(const Point<double> &p, const Line<double> &l) {
    auto v = l.e-l.s;
    return abs(v*(p-l.s))/v.dis();
}

double disSegment(const Point<double> &p, const Line<double> &l) {
    auto v = l.e-l.s;
    if ((v&(p-l.s)) <= 0) return (p-l.s).dis();
    if ((v&(p-l.e)) >= 0) return (p-l.e).dis();
    return disLine(p, l);
}

```

交点与线段相交

接口	含义	复杂度	备注
<code>crs(a, b)</code>	两直线交点	$O(1)$	只用于 <code>Point<double></code> ，端点不同且两直线不平行
<code>segmentIntersection(a, b)</code>	两线段交集类型： 0 不交， 1 严格相交， 2 单点接触， 3 共线重叠	$O(1)$	端点不同；浮点判定需改用 <code>eps</code> ，坐标差及叉积中间值超出 <code>i64</code> 时改用 <code>i128</code>

```

Point<double> crs(const Line<double> &a, const Line<double> &b) {
    double x = (a.e-b.e)*(b.e-b.s);
    double y = (a.e-a.s)*(b.e-b.s);
    return a.e-(a.e-a.s)*(x/y);
}

template <class T>
int segmentIntersection(Line<T> a, Line<T> b) {
    T x1 = (a.e-a.s)*(b.s-a.s), x2 = (a.e-a.s)*(b.e-a.s);
    T y1 = (b.e-b.s)*(a.s-b.s), y2 = (b.e-b.s)*(a.e-b.s);
    bool x = (x1 < 0 && x2 > 0) || (x1 > 0 && x2 < 0);
    bool y = (y1 < 0 && y2 > 0) || (y1 > 0 && y2 < 0);
    if (x && y) return 1;
    if (x1 == 0 && x2 == 0 && y1 == 0 && y2 == 0) {
        Point<T> l = max(min(a.s, a.e), min(b.s, b.e));
        Point<T> r = min(max(a.s, a.e), max(b.s, b.e));
        if (r < l) return 0;
        return l == r ? 2 : 3;
    }
    if (onSegment(a.s, b) || onSegment(a.e, b) || onSegment(b.s, a) || onSegment(b.e, a)) return 2;
    return 0;
}

```

任意多边形

用途： 计算任意简单多边形的有向二倍面积、面积，并判断点在多边形外部、边界或内部。

多边形面积

接口	含义	复杂度	备注
<code>polygonArea2(a)</code>	有向二倍面积，逆时针为正、顺时针为负	时间 $O(n)$ ，空间 $O(1)$	至少 3 点，按边界顺序且不重复首点，多边形不自交；叉积及总和超出 <code>i64</code> 时改用 <code>i128</code>
<code>polygonArea(a)</code>	多边形面积	时间 $O(n)$ ，空间 $O(1)$	条件同 <code>polygonArea2(a)</code>

```
template <class T>
T polygonArea2(const vector<Point<T>> &a) {
    int n = a.size();
    T ans = 0;
    for (int i = 0; i < n; i++) {
        ans += a[i]*a[(i+1)%n];
    }
    return ans;
}

template <class T>
double polygonArea(const vector<Point<T>> &a) {
    T ans = polygonArea2(a);
    if (ans < 0) ans = -ans;
    return ans/2.0;
}
```

点与多边形关系

接口	含义	复杂度	备注
<code>pointPolygonRelation(p, a)</code>	点在外、边界、内部分别返回 0,1,2	时间 $O(n)$ ，空间 $O(1)$	至少 3 点，按边界顺序且不重复首点，多边形不自交；浮点判定需改用 <code>eps</code> ，叉积中间值超出 <code>i64</code> 时改用 <code>i128</code>

```
template <class T>
int pointPolygonRelation(const Point<T> &p, const vector<Point<T>> &a) {
    int n = a.size(), in = 0;
    for (int i = 0; i < n; i++) {
        int j = (i+1)%n;
        if (onSegment(p, Line<T>(a[i], a[j]))) return 1;
        T cross = (a[j]-a[i])*(p-a[i]);
        if (a[i].y <= p.y && p.y < a[j].y && cross > 0) in ^= 1;
        if (a[j].y <= p.y && p.y < a[i].y && cross < 0) in ^= 1;
    }
    return in ? 2 : 0;
}
```

凸多边形

用途： 在环形双调范围内二分极值，并对凸多边形快速判断点、直线与线段的位置，求交点、直径及两个凸多边形的闵可夫斯基和。

环形极值

接口	含义	复杂度	备注
<code>cyclicMin(first, last, cmp)</code>	返回环形双调范围在 <code>cmp</code> 下的任一最小元素迭代器	时间 $O(\log n)$, 空间 $O(1)$	随机访问迭代器；绕环只有一个最小值段和一个最大值段，每段至多含两个等价元素，段间严格单调；空区间返回 <code>last</code>
<code>extremeVertex(a, dir)</code>	返回使 <code>a[i]&dir</code> 最大的支撑点下标	时间 $O(\log n)$, 空间 $O(1)$	至少 3 点，逆时针且不重复首点，相邻三点不共线；最大投影落在一条边上时返回其中一个端点；浮点比较需自行处理，点积中间值超出 <code>i64</code> 时改用 <code>i128</code>

```
template <class It, class Comp>
It cyclicMin(It first, It last, Comp cmp) {
    int n = last-first;
    if (n <= 1) return first;
    int dir;
    if (cmp(first[1], first[0])) dir = 1;
    else if (cmp(first[n-1], first[0])) dir = -1;
    else return first;
    auto at = [&](int i) -> decltype(auto) { return first[(dir*i+n)%n]; };
    auto le0 = [&](int i) { return !cmp(first[0], at(i)); };
    int l = 0, r = n-1;
    if (!le0(r)) {
        l = 1;
        while (l+1 < r) {
            int m = (l+r)/2;
            if (le0(m)) l = m;
            else r = m;
        }
        r = l, l = 0;
    }
    while (l < r) {
        int m = (l+r)/2;
        if (cmp(at(m+1), at(m))) l = m+1;
        else r = m;
    }
    return first+(dir*l+n)%n;
}

template <class T>
int extremeVertex(const vector<Point<T>> &a, const Point<T> &dir) {
    auto it = cyclicMin(a.begin(), a.end(), [&](const auto &x, const auto &y) { return (x&dir) > (y&dir); });
    return it-a.begin();
}
```

点与凸多边形关系

接口	含义	复杂度	备注
<code>pointConvexRelation(p, a)</code>	点在凸多边形外部、边界、内部分别返回 0, 1, 2	时间 $O(\log n)$ ，空间 $O(1)$	至少 3 点，逆时针且不重复首点，相邻三点不共线；浮点判定需改用 <code>eps</code> ，叉积中间值超出 <code>i64</code> 时改用 <code>i128</code>

```
template <class T>
int pointConvexRelation(const Point<T> &p, const vector<Point<T>> &a) {
    int n = a.size();
    T x = (a[1]-a[0])*(p-a[0]), y = (a[n-1]-a[0])*(p-a[0]);
    if (x < 0 || y > 0) return 0;
    if (onSegment(p, Line<T>(a[0], a[1])) || onSegment(p, Line<T>(a[0], a[n-1]))) return 1;
    int l = 1, r = n-1;
    while (r-l > 1) {
        int m = (l+r)/2;
        if ((a[m]-a[0])*(p-a[0]) >= 0) l = m;
        else r = m;
    }
    T z = (a[r]-a[l])*(p-a[l]);
    if (z < 0) return 0;
    return z == 0 ? 1 : 2;
}
```

直线、线段与凸多边形求交

接口	含义	复杂度	备注
<code>convexLineEdges(l, a)</code>	直线不交、切顶点、重合边、穿过两边分别返回 $\{-1, -1\}, \{i, -1\}, \{i, i\}, \{i, j\}$	时间 $O(\log n)$ ，空间 $O(1)$	至少 3 点，逆时针且不重复首点，相邻三点不共线；直线端点不同；边 i 表示 $a_i a_{i+1}$ ，穿过两边时按直线方向返回；浮点判定需自行改用 <code>eps</code> ，点积、叉积中间值超出 <code>i64</code> 时改用 <code>i128</code>
<code>convexLineIntersection(l, a)</code>	直线与凸多边形交集的 0 ~ 2 个端点	时间 $O(\log n)$ ，空间 $O(1)$	条件同 <code>convexLineEdges()</code> ；返回 <code>Point<double></code> ，按直线方向排列
<code>convexSegmentIntersection(l, a)</code>	线段与凸多边形交集的 0 ~ 2 个端点	时间 $O(\log n)$ ，空间 $O(1)$	条件同 <code>convexLineEdges()</code> ，距离平方超出 <code>i64</code> 时改用 <code>i128</code> ；相切返回一个点，完全在内部则返回原线段两端

```
template <class T>
array<int, 2> convexLineEdges(const Line<T> &l, const vector<Point<T>> &a) {
    int n = a.size();
    auto side = [&](int i) {
        T x = (l.e-l.s)*(a[i]-l.s);
        return (x > 0)-(x < 0);
    };
    int hi = extremeVertex(a, (l.e-l.s).rotate90());
    int lo = extremeVertex(a, (l.s-l.e).rotate90());
    if (side(hi) < 0 || side(lo) > 0) return {-1, -1};
    array<int, 2> ans;
    for (int k = 0; k < 2; k++) {
        int x = hi, y = lo;
        while ((x+1)%n != y) {
            int m = ((x+y+(x < y ? 0 : n))/2)%n;
            if (side(m) == side(hi)) x = m;
            else y = m;
        }
    }
}
```

```

        ans[k] = (x+(side(y) == 0))%n;
        swap(hi, lo);
    }
    if (ans[0] == ans[1]) return {ans[0], -1};
    if (side(ans[0]) == 0 && side(ans[1]) == 0) {
        if ((ans[0]-ans[1]+n+1)%n == 0) return {ans[0], ans[0]};
        if ((ans[0]-ans[1]+n+1)%n == 2) return {ans[1], ans[1]};
    }
    return ans;
}

template <class T>
vector<Point<double>> convexLineIntersection(const Line<T> &l, const vector<Point<T>> &a) {
    int n = a.size();
    auto e = convexLineEdges(l, a);
    if (e[0] < 0) return {};
    auto point = [&](int i) { return Point<double>(a[i].x, a[i].y); };
    if (e[1] < 0) return {point(e[0])};
    vector<Point<double>> res;
    if (e[0] == e[1]) res = {point(e[0]), point((e[0]+1)%n)};
    else for (int i : e) {
        auto d = l.e-l.s, p = a[i], v = a[(i+1)%n]-p;
        double t = 1.0*((l.s-p)*d)/(v*d);
        res.push_back(Point<double>(p.x+v.x*t, p.y+v.y*t));
    }
    auto pos = [&](const auto &p) {
        return (p.x-l.s.x)*(l.e.x-l.s.x)+(p.y-l.s.y)*(l.e.y-l.s.y);
    };
    if (pos(res[0]) > pos(res[1])) swap(res[0], res[1]);
    return res;
}

template <class T>
vector<Point<double>> convexSegmentIntersection(const Line<T> &l, const vector<Point<T>> &a) {
    auto p = convexLineIntersection(l, a);
    if (p.empty()) return {};
    Point<double> s(l.s.x, l.s.y), e(l.e.x, l.e.y), d = e-s;
    double x = max(0.0, ((p.front()-s)&d)/d.dis2());
    double y = min(1.0, ((p.back()-s)&d)/d.dis2());
    if (x > y+eps) return {};
    auto u = s+d*x, v = s+d*y;
    if ((u-v).dis2() <= eps*eps) return {u};
    return {u, v};
}

```

凸多边形直径

接口	含义	复杂度	备注
<code>convexDiameter2(a)</code>	凸多边形直径的平方	时间 $O(n)$, 空间 $O(1)$	允许 1 或 2 点; 更多点需逆时针且不重复首点, 相邻三点不共线; 浮点判定需自行改用 <code>eps</code> , 叉积、距离平方超出 <code>i64</code> 时改用 <code>i128</code>

```
template <class T>
T convexDiameter2(const vector<Point<T>> &a) {
    int n = a.size();
    if (n <= 1) return 0;
    if (n == 2) return (a[0]-a[1]).dis2();
    T ans = 0;
    for (int i = 0, j = 1; i < n; i++) {
        int ni = (i+1)%n;
        auto area = [&](int k) { return (a[ni]-a[i])*(a[k]-a[i]); };
        while (area((j+1)%n) > area(j)) j = (j+1)%n;
        ans = max(ans, max((a[i]-a[j]).dis2(), (a[ni]-a[j]).dis2()));
        int nj = (j+1)%n;
        if (area(nj) == area(j))
            ans = max(ans, max((a[i]-a[nj]).dis2(), (a[ni]-a[nj]).dis2()));
    }
    return ans;
}
```

闵可夫斯基和

接口	含义	复杂度	备注
<code>minkowskiSum(a, b)</code>	两个凸多边形的闵可夫斯基和，返回逆时针顶点序列	时间 $O(n + m)$ ，空间 $O(n + m)$	两者均至少 3 点，逆时针且不重复首点，相邻三点不共线；浮点判定需自行改用 <code>eps</code> ，坐标加法、叉积中间值超出 <code>i64</code> 时改用 <code>i128</code>

```
template <class T>
vector<Point<T>> minkowskiSum(vector<Point<T>> a, vector<Point<T>> b) {
    auto normalize = [](vector<Point<T>> &p) {
        auto it = min_element(p.begin(), p.end(), [](const auto &a, const auto &b) {
            return a.y != b.y ? a.y < b.y : a.x < b.x;
        });
        rotate(p.begin(), it, p.end());
    };
    normalize(a), normalize(b);
    int n = a.size(), m = b.size(), i = 0, j = 0;
    vector<Point<T>> c = {a[0]+b[0]};
    while (i < n || j < m) {
        Point<T> v;
        if (i == n) v = b[(j+1)%m]-b[j], j++;
        else if (j == m) v = a[(i+1)%n]-a[i], i++;
        else {
            auto x = a[(i+1)%n]-a[i], y = b[(j+1)%m]-b[j];
            bool sx = x.y < 0 || (x.y == 0 && x.x < 0);
            bool sy = y.y < 0 || (y.y == 0 && y.x < 0);
            if (sx != sy) {
                if (sx) v = y, j++;
                else v = x, i++;
            } else if (x*y > 0) v = x, i++;
            else if (x*y < 0) v = y, j++;
            else v = x+y, i++, j++;
        }
        c.push_back(c.back()+v);
    }
    c.pop_back();
    return c;
}
```

圆

用途： 用 `Circle<T>` 表示圆，判断点与三角形外接圆的位置关系，求三角形外心、圆的交点、切点与公切线。

外接圆

接口	含义	复杂度	备注
<code>inCircle(a,b,c,p)</code>	判断点 <code>p</code> 在三角形 <code>abc</code> 的外接圆内、圆上、圆外，分别返回 <code>1,0,-1</code>	$O(1)$	<code>a,b,c</code> 不共线，顺逆时针均可； <code>int</code> 为 <code>i64</code> ，坐标绝对值不超过 10^9 ，依赖 <code>i128</code>
<code>circumcenter(a,b,c)</code>	返回三角形 <code>abc</code> 的外心	$O(1)$	三点不共线；返回 <code>Point<double></code>

```
int inCircle(Point<int> a, Point<int> b, Point<int> c, const Point<int> &p) {
    if ((b-a)*(c-a) < 0) swap(b, c);
    a = a-p, b = b-p, c = c-p;
    i128 d = (i128)a.dis2()*(b*c)+(i128)b.dis2()*(c*a)+(i128)c.dis2()*(a*b);
    return (d > 0)-(d < 0);
}

template <class T>
Point<double> circumcenter(const Point<T> &a, const Point<T> &b, const Point<T> &c) {
    Point<double> o(a.x, a.y), u(b.x-a.x, b.y-a.y), v(c.x-a.x, c.y-a.y);
    return o+(u.rotate90()*v.dis2()-v.rotate90()*u.dis2()/(2*(u*v)));
}
```

圆与交点

接口	含义	复杂度	备注
<code>circleLineIntersection(c,l)</code>	直线与圆的交点，返回 <code>0~2</code> 个点	$O(1)$	仅用于 <code>Circle<double></code> ，半径为正；依赖 <code>Point</code> 、 <code>Line</code> 、 <code>project()</code> 与合适的 <code>eps</code> ；直线两端点不同
<code>circleIntersection(a,b)</code>	两圆交点，返回 <code>0~2</code> 个点	$O(1)$	仅用于 <code>Circle<double></code> ，半径为正；依赖 <code>Point</code> 与合适的 <code>eps</code> ；两圆不能重合

```
template <class T>
struct Circle {
    Point<T> o;
    T r;

    Circle(Point<T> o_ = {0, 0}, T r_ = 0) { o = o_, r = r_; }
};

vector<Point<double>> circleLineIntersection(const Circle<double> &c, const Line<double> &l) {
    auto p = project(c.o, l); double h2 = c.r*c.r-(p-c.o).dis2();
    if (h2 < -eps) return {};
    if (h2 <= eps) return {p};
    auto v = (l.e-l.s).unit()*sqrt(h2);
    return {p-v, p+v};
}
```

```
vector<Point<double>> circleIntersection(const Circle<double> &a, const Circle<double> &b) {
    auto v = b.o-a.o; double d = v.dis();
    if (d <= eps) return {};
    if (d > a.r+b.r+eps || d < abs(a.r-b.r)-eps) return {};
    double x = (a.r*a.r-b.r*b.r+d*d)/(2*d);
    auto p = a.o+v*(x/d);
    double h2 = a.r*a.r-x*x;
    if (h2 <= eps) return {p};
    auto q = v.rotate90()*(sqrt(h2)/d);
    return {p-q, p+q};
}
```

切点与公切线

接口	含义	复杂度	备注
<code>tangentPoints(p, c)</code>	过点作圆的切线，返回 0 ~ 2 个切点	$O(1)$	仅用于 <code>Circle<double></code> ，半径为正；依赖 <code>Point</code> 与合适的 <code>eps</code> ；点在圆内时返回空数组
<code>commonTangents(a, b)</code>	两圆公切线，返回 0 ~ 4 条直线	$O(1)$	仅用于 <code>Circle<double></code> ，半径为正；依赖 <code>Point</code> 、 <code>Line</code> 与合适的 <code>eps</code> ；两圆不能重合

```
vector<Point<double>> tangentPoints(const Point<double> &p, const Circle<double> &c) {
    auto v = p-c.o; double d2 = v.dis2(), r2 = c.r*c.r;
    if (d2 < r2-eps) return {};
    if (d2 <= r2+eps) return {c.o+v.unit()*c.r};
    double x = r2/d2;
    double y = c.r*sqrt(d2-r2)/d2;
    auto q = v.rotate90();
    return {c.o+v*x-q*y, c.o+v*x+q*y};
}

vector<Line<double>> commonTangents(const Circle<double> &a, const Circle<double> &b) {
    vector<Line<double>> res;
    auto v = b.o-a.o;
    double d2 = v.dis2();
    if (d2 <= eps) return res;
    for (int s : {1, -1}) {
        double r = a.r-s*b.r;
        double h2 = d2-r*r;
        if (h2 < -eps) continue;
        for (int t : {-1, 1}) {
            auto n = (v*r+v.rotate90()*(sqrt(max(0.0, h2))*t))/d2;
            auto p = a.o+n*a.r;
            res.push_back(Line<double>(p, p+n.rotate90()));
            if (h2 <= eps) break;
        }
    }
    return res;
}
```

半平面交

用途： 求若干有向直线左侧半平面的交，结果以逆时针交多边形顶点序列给出；空交、无界交或退化交返回空数组。

接口	含义	复杂度	备注
<code>halfPlaneIntersection(line)</code>	求各有向直线左侧半平面的交多边形	时间 $O(n \log n)$, 空间 $O(n)$	依赖 <code>Point<double></code> 、 <code>Line<double></code> 、 <code>crs</code> 与合适的 <code>eps</code> ；每条直线的两端点必须不同

```
bool right(const Line<double> &a, const Point<double> &p) {
    return (a.e-a.s)*(p-a.s) < -eps;
}

vector<Point<double>> halfPlaneIntersection(vector<Line<double>> line) {
    auto parallel = [](const Line<double> &a, const Line<double> &b) {
        auto u = a.e-a.s, v = b.e-b.s;
        return abs(u*v) <= eps*sqrt((u&u)*(v&v));
    };

    sort(line.begin(), line.end(), [](const auto &a, const auto &b) {
        double A = a.angle(), B = b.angle();
        if (A < 0) A += 2*acos(-1.0);
        if (B < 0) B += 2*acos(-1.0);
        if (A != B) return A < B;
        return (a.e-a.s)*(b.s-a.s) < 0;
    });

    deque<Line<double>> q;
    deque<Point<double>> p;
    for (auto l : line) {
        while (p.size() && right(l, p.back())) p.pop_back(), q.pop_back();
        while (p.size() && right(l, p.front())) p.pop_front(), q.pop_front();
        while (q.size() && parallel(q.back(), l)) {
            if (((q.back().e-q.back().s)&(l.e-l.s)) < 0) return {};
            if (!right(l, q.back().s)) break;
            q.pop_back();
            if (p.size()) p.pop_back();
        }
        if (q.size() && parallel(q.back(), l)) continue;
        if (q.size()) p.push_back(crs(q.back(), l));
        q.push_back(l);
    }

    while (p.size() && right(q.front(), p.back())) p.pop_back(), q.pop_back();
    while (p.size() && right(q.back(), p.front())) p.pop_front(), q.pop_front();
    if (q.size() < 3 || parallel(q.back(), q.front())) return {};
    p.push_back(crs(q.back(), q.front()));
    vector<Point<double>> res(p.begin(), p.end());
    double area = 0;
    for (int i = 0; i < (int)res.size(); i++)
        area += res[i]*res[(i+1)%res.size()];
    return abs(area) <= eps ? vector<Point<double>>() : res;
}
```

Delaunay 三角剖分

用途： 用分治 Quad-Edge 算法求点集的 Delaunay 三角剖分，并维护其 Voronoi 对偶图。构造时按坐标排序并去重，`p` 保存点，`tri` 保存三角形，`edge` 保存 Delaunay 无向边；`g` 是以三角形为点、共边关系为边的对偶图，`vor` 保存三角形外心。

依赖： `Point<int>`、`Point<double>`、`Line<double>`，以及“圆”中的 `inCircle()`、`circumcenter()`；其中 `int` 按仓库约定表示 `i64`，输入点坐标绝对值不超过 10^9 。

退化情况： 重复点只保留一个；少于 3 个点或所有点共线时 `tri`、`vor`、`g`、`ray` 为空，`edge` 仍返回点集的一维剖分，`line` 保存相邻点的垂直平分线；四点共圆时返回其中一种合法剖分，此时 `vor` 可能有重合点，`g` 中相应的 Voronoi 边长度为 0。

操作/结构	含义	复杂度	备注
<code>Delaunay(a)</code>	传入点集 <code>a</code> ，构造 Delaunay 三角剖分、对偶图与 Voronoi 图	时间 $O(n \log n)$ ，空间 $O(n)$	会排序并去重；分治合并使用 Quad-Edge
<code>build(a)</code>	用新点集 <code>a</code> 重新构造，覆盖原结果	时间 $O(n \log n)$ ，空间 $O(n)$	条件同构造函数
<code>p[i]</code>	去重后按坐标升序排列的第 <code>i</code> 个点	-	<code>tri</code> 、 <code>edge</code> 中的编号均指向这里
<code>tri[i]</code>	第 <code>i</code> 个三角形的三个顶点编号，按逆时针排列	-	三角形非退化；四点共圆时选取的对角线不固定
<code>edge[i]</code>	第 <code>i</code> 条 Delaunay 无向边的两个端点编号	-	端点满足 $u < v$ ，每条边只出现一次
<code>g[i]</code>	对偶图中与第 <code>i</code> 个三角形共边的三角形编号	-	不把外部面作为点；每条有限 Voronoi 边在 <code>g</code> 中出现两次
<code>vor[i]</code>	<code>tri[i]</code> 的外心，即对应的 Voronoi 顶点	-	<code>g</code> 中的相邻面 <code>i,j</code> 对应线段 <code>vor[i]vor[j]</code>
<code>ray[i]</code>	<code>{f,u,v,d}</code> 表示从 <code>vor[f]</code> 沿非零向量 <code>d</code> 延伸的 Voronoi 射线	-	对应凸包边 <code>(u,v)</code> ；端点满足 $u < v$
<code>line[i]</code>	全共线时的一条 Voronoi 边	-	是 <code>edge[i]</code> 两端点的垂直平分线；非全共线时空

Voronoi 的有限部分由 `vor` 与 `g` 给出：`i,j` 在 `g` 中相邻时，存在边段 `vor[i] - vor[j]`；凸包边对应 `ray` 中的无界射线。全共线时没有有限 Voronoi 顶点，直接用 `line` 表示相邻点的垂直平分线。

这里的 `g` 是删除外部面的平面对偶图，一般含有环，不是树。

常用性质

- 空圆性质：每个 Delaunay 三角形的外接圆内部都不含其他输入点；反之，满足局部空圆条件的三角剖分就是 Delaunay 三角剖分。
- 对偶关系：Delaunay 三角形对应 Voronoi 顶点，两个三角形的公共边对应有有限 Voronoi 边，凸包边对应无界 Voronoi 射线；Delaunay 点对应一个 Voronoi 区域。
- Delaunay 图是平面图，所有凸包边都在图中。若去重后有 n 个点、凸包边界上有 h 个点（包括边上的共线点）且不全共线，则边数为 $3n - h - 3$ ，三角形数为 $2n - h - 2$ 。
- 不存在四点共圆时，Delaunay 三角剖分唯一；存在共圆点时可能有多种合法剖分，合并重合外心并删除零长度边后都得到同一个几何 Voronoi 图。
- 在所有合法三角剖分中，Delaunay 三角剖分最大化最小角，因此通常能避免细长三角形。
- 最近点对的边、每个点到某个最近邻的边以及欧几里得最小生成树的所有边都属于 Delaunay 图，可把相应候选边从 $O(n^2)$ 降到 $O(n)$ 。

```
struct Delaunay {
    struct Ray {
        int f, u, v;
        Point<double> d;
    };
    // ... other members ...
};
```

```

};

vector<Point<int>> p;
vector<array<int, 3>> tri;
vector<array<int, 2>> edge;
vector<Point<double>> vor;
vector<vector<int>> g;
vector<Ray> ray;
vector<Line<double>> line;

private:
    struct Quad {
        Quad *rot, *next;
        int p, vis;

        Quad *rev() const { return rot->rot; }
        Quad *prev() const { return rot->next->rot; }
        Quad *lNext() const { return rev()->prev(); }
    };

    deque<array<Quad, 4>> pool;
    Quad *free = nullptr;

    int side(int a, int b, int c) const {
        return (p[b]-p[a])*(p[c]-p[a]);
    }

    int org(Quad *e) const { return e->p; }
    int dst(Quad *e) const { return e->rev()->p; }

    Quad *make(int u, int v) {
        Quad *a;
        if (free) a = free, free = free->next;
        else {
            pool.push_back({}), a = pool.back().data();
            for (int i = 0; i < 4; i++) a[i].rot = &a[(i+1)%4];
        }
        Quad *q[4] = {a, a->rot, a->rot->rot, a->rot->rot->rot};
        for (int i = 0; i < 4; i++) q[i]->p = -1, q[i]->vis = 0;
        q[0]->next = q[0], q[1]->next = q[3];
        q[2]->next = q[2], q[3]->next = q[1];
        q[0]->p = u, q[2]->p = v;
        return q[0];
    }

    void splice(Quad *a, Quad *b) {
        swap(a->next->rot->next, b->next->rot->next);
        swap(a->next, b->next);
    }

    Quad *connect(Quad *a, Quad *b) {
        Quad *e = make(dst(a), org(b));
        splice(e, a->lNext()), splice(e->rev(), b);
    }

```

```

    return e;
}

void erase(Quad *e) {
    splice(e, e->prev(), splice(e->rev(), e->rev()->prev()));
    e->next = free, free = e;
}

int valid(Quad *e, Quad *bas) const {
    return side(dst(e), dst(bas), org(bas)) > 0;
}

pair<Quad *, Quad *> solve(int l, int r) {
    if (r-l <= 3) {
        Quad *a = make(l, l+1);
        if (r-l == 2) return {a, a->rev()};
        Quad *b = make(l+1, l+2);
        splice(a->rev(), b);
        int s = side(l, l+1, l+2);
        Quad *c = s ? connect(b, a) : nullptr;
        return {s < 0 ? c->rev() : a, s < 0 ? c : b->rev()};
    }

    int m = (l+r)/2;
    auto [ra, a] = solve(l, m);
    auto [b, rb] = solve(m, r);
    while (1) {
        if (side(org(b), dst(a), org(a)) < 0) a = a->lNext();
        else if (side(org(a), dst(b), org(b)) > 0) b = b->rev()->next;
        else break;
    }
    Quad *base = connect(b->rev(), a);
    if (org(a) == org(ra)) ra = base->rev();
    if (org(b) == org(rb)) rb = base;

    while (1) {
        Quad *lc = base->rev()->next;
        if (valid(lc, base)) {
            while (inCircle(p[dst(base)], p[org(base)], p[dst(lc)], p[dst(lc->next)]) > 0) {
                Quad *t = lc->next;
                erase(lc), lc = t;
            }
        }
        Quad *rc = base->prev();
        if (valid(rc, base)) {
            while (inCircle(p[dst(base)], p[org(base)], p[dst(rc)], p[dst(rc->prev())]) > 0) {
                Quad *t = rc->prev();
                erase(rc), rc = t;
            }
        }
        int lv = valid(lc, base), rv = valid(rc, base);
        if (!lv && !rv) break;
    }
}

```

```

        if (!lv || (rv && inCircle(p[org(rc)], p[dst(lc)], p[org(lc)], p[dst(rc)]) > 0))
            base = connect(rc, base->rev());
        else
            base = connect(base->rev(), lc->rev());
    }
    return {ra, rb};
}

```

```

void addFace(Quad *s, vector<Quad *> &que, int keep) {
    vector<int> f;
    Quad *e = s;
    do {
        e->vis = 1, f.push_back(org(e));
        que.push_back(e->rev());
        int u = org(e), v = dst(e);
        if (u > v) swap(u, v);
        edge.push_back({u, v});
        e = e->lNext();
    } while (e != s);
    if (!keep || f.size() != 3) return;
    while (f[1] < f[0] || f[2] < f[0])
        rotate(f.begin(), f.begin()+1, f.end());
    if (side(f[0], f[1], f[2]) < 0) swap(f[1], f[2]);
    if (side(f[0], f[1], f[2])) tri.push_back({f[0], f[1], f[2]});
}

```

```

void extract(Quad *e) {
    while (side(dst(e->next), dst(e), org(e)) < 0) e = e->next;
    vector<Quad *> que;
    addFace(e, que, 0);
    for (int i = 0; i < (int)que.size(); i++) {
        e = que[i];
        if (!e->vis) addFace(e, que, 1);
    }
    sort(edge.begin(), edge.end());
    edge.erase(unique(edge.begin(), edge.end(), edge.end()));
    sort(tri.begin(), tri.end());
}

```

```

void buildDual() {
    for (auto t : tri)
        vor.push_back(circumcenter(p[t[0]], p[t[1]], p[t[2]]));
    g.resize(tri.size());

    map<array<int, 2>, int> f;
    for (int i = 0; i < (int)tri.size(); i++) {
        for (int j = 0; j < 3; j++) {
            int u = tri[i][j], v = tri[i][(j+1)%3];
            if (u > v) swap(u, v);
            auto [it, ok] = f.emplace(array<int, 2>{u, v}, i);
            if (!ok) {
                int k = it->second;
                g[i].push_back(k), g[k].push_back(i), f.erase(it);
            }
        }
    }
}

```

```

        }
    }
}

if (tri.empty()) {
    for (auto [u, v] : edge) {
        Point<double> m((p[u].x+p[v].x)/2.0, (p[u].y+p[v].y)/2.0);
        Point<double> d(p[u].y-p[v].y, p[v].x-p[u].x);
        line.push_back({m-d, m+d});
    }
} else for (auto &[uv, a] : f) {
    auto [u, v] = uv;
    int w = tri[a][0]+tri[a][1]+tri[a][2]-u-v;
    Point<double> d(p[v].y-p[u].y, p[u].x-p[v].x);
    if (side(u, v, w) < 0) d = d*-1.0;
    ray.push_back({a, u, v, d});
}
for (auto &a : g) sort(a.begin(), a.end());
}

public:
    Delaunay(vector<Point<int>> a = {}) { build(move(a)); }

    void build(vector<Point<int>> a) {
        p = move(a);
        sort(p.begin(), p.end());
        p.erase(unique(p.begin(), p.end()), p.end());
        tri.clear(), edge.clear(), pool.clear(), free = nullptr;
        vor.clear(), g.clear(), ray.clear(), line.clear();
        int n = p.size();
        if (n <= 1) return;
        int collinear = 1;
        for (int i = 2; i < n; i++) collinear &= side(0, 1, i) == 0;
        if (collinear) {
            for (int i = 1; i < n; i++) edge.push_back({i-1, i});
            buildDual();
            return;
        }
        extract(solve(0, n).first);
        buildDual();
        pool.clear(), free = nullptr;
    }
};

```

数据结构

顺序	模板	用途	主要复杂度
01	树状数组	单点加、区间和、前缀第 k 小	$O(\log n)$
02	线段树	区间 chmin 、区间最小值与区间二分	$O(\log n)$
03	Treap	可重有序集合与顺序统计	期望 $O(\log n)$
04	<code>pb_ds</code> 平衡树	GNU 顺序统计树	$O(\log n)$
05	<code>pb_ds</code> 配对堆	可合并堆与任意元素修改	均摊 $O(\log n)$
06	树链剖分	树上路径拆分为 DFS 序连续段	$O(\log n)$ 每条路径
07	Link-Cut Tree	动态森林与路径异或	均摊 $O(\log n)$
08	后缀自动机	转移 DAG、parent 树与 endpos	$O(nV)$ 构建

树状数组

用途：维护单点加、前缀和与区间和；当所有频次非负时， $kth(k)$ 可查找第一个前缀和不小于 k 的位置。

复杂度：建树空间 $O(n)$ ； add 、 qry 、区间查询和 kth 均为 $O(\log n)$ 。

下标：使用 1 到 n 的一基下标。

```
struct FenwickTree {
    int n;
    vector<int> s;
    FenwickTree(int nn) {
        n = nn;
        s.resize(n+1);
    }

    void add(int x, int k) {
        for (; x <= n; x+=x&-x) s[x] += k;
    }

    int qry(int x) {
        int ans = 0;
        for (; x > 0; x-=x&-x) ans += s[x];
        return ans;
    }

    int qry(int l, int r) {
        return qry(r)-qry(l-1);
    }

    int kth(int k) {
        int pos = 0;
        for (int step = 1 << __lg(n); step; step >>= 1) {
            int nxt = pos + step;
            if (nxt <= n && s[nxt] < k) pos = nxt, k -= s[nxt];
        }
        return pos + 1;
    }
};
```

线段树

用途：维护区间 `chmin` 标记，即把覆盖区间内的每个值更新为 `min(原值,x)`，查询区间最小值，或查找 `[l,r]` 内第一个值不大于 `x` 的位置。

复杂度：建树 $O(n)$ 、空间 $O(n)$ ；一次更新、查询或 `lowerBound` 均为 $O(\log n)$ 。

下标：使用 1 到 n 的一基下标；带初值构造时，输入数组也按一基下标访问。

```
const int INF = 1e18;

struct SegTree {
    #define ls (2*u)
    #define rs (2*u+1)

    struct Node {
        signed l, r;
        int x;
        int lazy;

        inline int size() { return r-l+1; }

        Node() {
            x = lazy = INF;
        }
    };
    vector<Node> tr;

    void merge(Node &res, const Node &A, const Node &B, int u) {
        res.x = min(A.x, B.x);
    }

    inline void pushUp(int u) { merge(tr[u], tr[ls], tr[rs], u); }

    inline void setTag(int u, int x) {
        tr[u].lazy = min(tr[u].lazy, x);
        tr[u].x = min(tr[u].x, x);
    }

    inline void pushDown(int u) {
        setTag(ls, tr[u].lazy), setTag(rs, tr[u].lazy);
        tr[u].lazy = INF;
    }

    void update(int l, int r, int x, int u = 1) {
        if (l > r) return;
        int tL = tr[u].l, tR = tr[u].r;
        if (l <= tL && tR <= r) {
            setTag(u, x);
            return;
        }
        pushDown(u);
```

```

    int mid = (tL+tR)/2;
    if (l <= mid) update(l, r, x, ls);
    if (r > mid) update(l, r, x, rs);
    pushUp(u);
}

Node query(int l, int r, int u = 1) {
    if (l > r) return Node();
    int tL = tr[u].l, tR = tr[u].r;
    if (l <= tL && tR <= r) return tr[u];
    pushDown(u);
    int mid = (tL+tR)/2;
    Node ans;
    if (l <= mid && r > mid) {
        merge(ans, query(l, r, ls), query(l, r, rs), u);
        return ans;
    }
    if (l <= mid) return query(l, r, ls);
    if (r > mid) return query(l, r, rs);
    return Node();
}

int lowerBound(int l, int r, int x, int u = 1) {
    int tL = tr[u].l, tR = tr[u].r;
    if (r < tL || tR < l || tr[u].x > x) return -1;
    if (tL == tR) return tL;
    pushDown(u);
    int ans = lowerBound(l, r, x, ls);
    return ans != -1 ? ans : lowerBound(l, r, x, rs);
}

void buildTree(int l, int r, vector<int> &a, int u) {
    tr[u].l = l, tr[u].r = r;
    if (l == r) {
        tr[u].x = a[l];
    }
    if (l >= r) return;
    int mid = (l+r)/2;
    buildTree(l, mid, a, ls), buildTree(mid+1, r, a, rs);
    pushUp(u);
}

SegTree(int n, vector<int> &a) {
    tr.resize(4*n+1);
    buildTree(1, n, a, 1);
}

void buildTree(int l, int r, int u) {
    tr[u].l = l, tr[u].r = r;
    if (l >= r) return;
    int mid = (l+r)/2;
    buildTree(l, mid, ls);
    buildTree(mid+1, r, rs);
}

```

```
        pushUp(u);
    }

    SegTree(int n) {
        tr.resize(4*n+1);
        buildTree(1, n, 1);
    }
    #undef ls
    #undef rs
};
```

Treap

用途： 实现支持重复元素的有序多重集合，包括插入、删除一个值、排名、第 k 小、前驱和后继。

复杂度： 随机优先级均匀时，各操作期望 $O(\log n)$ ，空间 $O(n)$ ；最坏时间和递归深度均可达 $O(n)$ 。

以下 n 表示 Treap 的结构节点数，即不同值的个数；`info.cnt` 和 `size()` 统计包含重数的元素总数。`unite` 中令 $n \geq m$ 分别为两棵树的结构节点数。

操作	作用	期望复杂度	备注
<code>split(p, val, equal)</code>	按值将 <code>p</code> 拆成两棵 Treap	$O(\log n)$	<code>equal=1</code> 时左段 $\leq val$ ，否则左段 $< val$
<code>merge(x, y)</code>	拼接两个值域严格有序的 Treap	$O(\log n)$	要求 <code>x</code> 中所有值严格小于 <code>y</code> 中所有值，否则使用 <code>unite</code>
<code>insert(val)</code>	插入一个 <code>val</code> ，返回对应节点指针	$O(\log n)$	相同值只增加节点的 <code>cnt</code>
<code>extract(val)</code>	删除一个已存在的 <code>val</code> ，返回操作后的节点指针	$O(\log n)$	要求 <code>val</code> 已存在；重数减至 0 时返回 <code>nullptr</code>
<code>getRank(val)</code>	查询 <code>val</code> 的排名，即严格小于它的元素数加一	$O(\log n)$	排名从 1 开始
<code>getValue(rank)</code>	查询第 <code>rank</code> 小的元素，返回对应节点指针	$O(\log n)$	要求 $1 \leq rank \leq size()$
<code>getPre(val)</code> 、 <code>getNext(val)</code>	查询严格前驱、严格后继，返回对应节点指针	$O(\log n)$	答案不存在时返回 <code>nullptr</code>
<code>size()</code>	查询整棵树的元素总数	$O(1)$	包含各节点的重数 <code>cnt</code>
<code>rangeAdd(l, r, x)</code>	给值位于 <code>[l, r]</code> 的所有元素加 <code>x</code>	$O(\log n)$	修改后必须仍严格有序；后缀加可调用 <code>rangeAdd(val, INF, x)</code>
<code>rangeSum(l, r)</code>	查询值域 <code>[l, r]</code> 内所有元素之和	$O(\log n)$	若和可能超过 <code>i64</code> ，将 <code>Info::sum</code> 及相关乘法改为 <code>i128</code>
<code>unite(x, y)</code>	合并两棵值域可重叠的 Treap，并合并相同值的重数	$O\left(m \log\left(\frac{n}{m} + 1\right)\right)$ ， $m = 0$ 时为 $O(1)$	会复用两棵树的节点，合并后原根不得再作为独立 Treap 使用

```
mt19937_64 rng(time(0));

struct Treap {
    using u64 = unsigned long long;

    struct Info {
        int cnt = 0, sum = 0;

        Info() {}
        Info(int v, int c = 1) : cnt(c), sum(v*c) {}

        Info operator+(const Info &b) const {
            Info c;
            c.cnt = cnt+b.cnt, c.sum = sum+b.sum;
            return c;
        }
    };

    struct Node {
        array<Node*, 2> s{};
```

```

int val, cnt = 1, add = 0;
Info info;
u64 key;

Node(int v) : val(v), info(v), key(rng()) {}

static Info getInfo(Node *p) {
    return p ? p->info : Info();
}

void pushUp() {
    info = getInfo(s[0])+Info(val, cnt)+getInfo(s[1]);
}

void addCnt(int x) {
    cnt += x, pushUp();
}

void setAdd(int x) {
    val += x, info.sum += x*info.cnt, add += x;
}

void pushDown() {
    if (!add) return;
    for (auto p : s) if (p) p->setAdd(add);
    add = 0;
}
};

Node *root = nullptr;

int size(Node *p) { return Node::getInfo(p).cnt; }
int sum(Node *p) { return Node::getInfo(p).sum; }

array<Node*, 2> split(Node *p, int val, bool equal = 1) {
    if (!p) return {};
    p->pushDown();
    if (p->val < val || (equal && p->val == val)) {
        auto a = split(p->s[1], val, equal);
        p->s[1] = a[0], p->pushUp();
        return {p, a[1]};
    }
    auto a = split(p->s[0], val, equal);
    p->s[0] = a[1], p->pushUp();
    return {a[0], p};
}

array<Node*, 3> splitRange(Node *p, int l, int r) {
    auto a = split(p, r);
    auto b = split(a[0], l, 0);
    return {b[0], b[1], a[1]};
}

```

```

Node *merge(Node *x, Node *y) {
    if (!x || !y) return x ? x : y;
    if (x->key > y->key) {
        x->pushDown();
        x->s[1] = merge(x->s[1], y), x->pushUp();
        return x;
    }
    y->pushDown();
    y->s[0] = merge(x, y->s[0]), y->pushUp();
    return y;
}

```

```

Node *unite(Node *x, Node *y) {
    if (!x || !y) return x ? x : y;
    if (x->key < y->key) swap(x, y);
    x->pushDown();
    auto a = splitRange(y, x->val, x->val);
    if (a[1]) x->cnt += a[1]->cnt, delete a[1];
    x->s[0] = unite(x->s[0], a[0]);
    x->s[1] = unite(x->s[1], a[2]);
    x->pushUp();
    return x;
}

```

```

Node *join(array<Node*, 3> a) {
    return merge(merge(a[0], a[1]), a[2]);
}

```

```

Node *insert(int val) {
    auto a = splitRange(root, val, val);
    if (a[1]) a[1]->addCnt(1);
    else a[1] = new Node(val);
    root = join(a);
    return a[1];
}

```

```

Node *extract(int val) {
    auto a = splitRange(root, val, val);
    if (a[1]->cnt > 1) a[1]->addCnt(-1);
    else delete a[1], a[1] = nullptr;
    root = join(a);
    return a[1];
}

```

```

void rangeAdd(int l, int r, int x) {
    auto a = splitRange(root, l, r);
    if (a[1]) a[1]->setAdd(x);
    root = join(a);
}

```

```

int rangeSum(int l, int r) {
    auto a = splitRange(root, l, r);
    int ans = sum(a[1]);
}

```

```

    root = join(a);
    return ans;
}

int getRank(int val) {
    int ans = 1;
    for (Node *p = root; p; ) {
        p->pushDown();
        if (p->val >= val) p = p->s[0];
        else ans += size(p->s[0])+p->cnt, p = p->s[1];
    }
    return ans;
}

Node *getValue(int rank) {
    Node *p = root;
    while (1) {
        p->pushDown();
        int lsz = size(p->s[0]);
        if (rank <= lsz) p = p->s[0];
        else if (rank <= lsz+p->cnt) return p;
        else rank -= lsz+p->cnt, p = p->s[1];
    }
}

Node *getPre(int val) {
    Node *p = root, *ans = nullptr;
    while (p) {
        p->pushDown();
        if (p->val < val) ans = p, p = p->s[1];
        else p = p->s[0];
    }
    return ans;
}

Node *getNext(int val) {
    Node *p = root, *ans = nullptr;
    while (p) {
        p->pushDown();
        if (p->val > val) ans = p, p = p->s[0];
        else p = p->s[1];
    }
    return ans;
}

int size() { return size(root); }
};

```

pb_ds 平衡树

用途： GNU 扩展的有序集合，额外支持按排名找元素和查询严格小于某键的元素个数。以 `pair<值, 唯一编号>` 作为键时可模拟可重集合。

编译环境： 依赖 GCC 的 `__gnu_pbds`，头文件要放在 `#define int long long` 之前。

操作	作用	复杂度	备注
<code>insert(x)</code>	插入键 <code>x</code>	$O(\log n)$	返回 <code>pair<point_iterator, bool></code> ；键已存在时插入失败
<code>erase(x)</code> 、 <code>erase(it)</code>	按键或迭代器删除元素	$O(\log n)$	前者返回是否删除成功，后者返回被删元素的后继迭代器
<code>erase_if(pred)</code>	删除所有满足谓词的元素	$O(n \log n)$	返回删除个数
<code>find(x)</code>	查找键 <code>x</code>	$O(\log n)$	不存在时返回 <code>end()</code>
<code>order_of_key(x)</code>	查询严格小于 <code>x</code> 的元素个数	$O(\log n)$	即 <code>x</code> 从 0 开始的排名
<code>find_by_order(k)</code>	查询排名为 <code>k</code> 的元素	$O(\log n)$	排名从 0 开始；越界时返回 <code>end()</code>
<code>lower_bound(x)</code>	查询第一个不小于 <code>x</code> 的元素	$O(\log n)$	若返回值不为 <code>begin()</code> ，前移一次得到严格前驱
<code>upper_bound(x)</code>	查询第一个严格大于 <code>x</code> 的元素	$O(\log n)$	即严格后继
<code>begin()</code> 、 <code>end()</code>	获取正向首、尾迭代器	$O(1)$	迭代顺序按键递增
<code>rbegin()</code> 、 <code>rend()</code>	获取反向首、尾迭代器	$O(1)$	仅节点型树提供，当前 <code>rb_tree_tag</code> 可用
<code>join(x)</code>	将树 <code>x</code> 并入当前树	$O(\log(n + m))$	要求一棵树的所有键严格小于另一棵树；完成后 <code>x</code> 为空
<code>split(x, b)</code>	按键 <code>x</code> 拆分到树 <code>b</code>	实现相关，常用稳定版最坏 $O(n)$	当前树保留 <code><=x</code> ，其余移入 <code>b</code> ；调用前会先清空 <code>b</code>
<code>clear()</code>	删除全部元素	$O(n)$	删除后树为空
<code>empty()</code> 、 <code>size()</code>	查询是否为空、元素个数	$O(1)$	<code>size()</code> 统计键的数量
<code>swap(x)</code>	交换两棵树	$O(1)$	同时交换比较器状态

```
#include <ext/pb_ds/assoc_container.hpp>
#include <ext/pb_ds/tree_policy.hpp>
// 放在 #define int long long 上面

__gnu_pbds::tree<pair<int, int>, __gnu_pbds::null_type, less<pair<int, int>>,
    __gnu_pbds::rb_tree_tag,
    __gnu_pbds::tree_order_statistics_node_update>
trr;
```

pb_ds 配对堆

用途：GNU 扩展的可合并优先队列；除堆顶、插入和删除外，还能通过 `push` 返回的点迭代器修改或删除任意元素。配对堆实现简单，实践常数通常较好，但并非无条件快于所有堆；只需普通 `push/pop` 时，连续内存的 `std::priority_queue` 往往更合适。

复杂度：`push`、`top`、`join` 为 $O(1)$ ，`pop`、`modify`、`erase` 均摊 $O(\log n)$ ，`split` 为 $O(n)$ ，空间 $O(n)$ 。

编译环境：依赖 GCC 的 `__gnu_pbds`，头文件要放在 `#define int long long` 之前。

操作	作用	复杂度	备注
<code>push(x)</code>	插入元素 <code>x</code>	$O(1)$	返回指向该元素的点迭代器 <code>point_iterator</code> ；允许重复元素
<code>top()</code>	查询堆顶	$O(1)$	<code>less<T></code> 为大根堆， <code>greater<T></code> 为小根堆
<code>pop()</code>	删除堆顶	均摊 $O(\log n)$	调用前堆必须非空
<code>modify(it,x)</code>	将点迭代器 <code>it</code> 对应的元素改为 <code>x</code>	均摊 $O(\log n)$	新值可以变大或变小，迭代器仍指向该元素
<code>erase(it)</code>	删除点迭代器 <code>it</code> 对应的元素	均摊 $O(\log n)$	删除后该迭代器失效
<code>join(x)</code>	将堆 <code>x</code> 合并到当前堆	$O(1)$	完成后 <code>x</code> 为空，两堆类型必须相同
<code>split(pred,x)</code>	将满足 <code>pred</code> 的元素移入堆 <code>x</code>	$O(n)$	调用前会先清空 <code>x</code>
<code>begin()</code> 、 <code>end()</code>	遍历所有元素	$O(n)$	遍历顺序不保证有序，也不是依次弹出的顺序
<code>clear()</code>	删除全部元素	$O(n)$	所有指向原元素的点迭代器失效
<code>empty()</code> 、 <code>size()</code>	查询是否为空、元素个数	$O(1)$	

点迭代器 `point_iterator` 只在对应元素被 `pop`、`erase` 或 `clear` 删除时失效；其他元素的插入、修改、删除和堆合并不会使它失效。为避免与 `std::priority_queue` 冲突，不要直接 `using namespace __gnu_pbds`。

```
#include <ext/pb_ds/priority_queue.hpp>
// 放在 #define int long long 上面

template<class T, class Cmp = less<T>>
using Heap = __gnu_pbds::priority_queue<T, Cmp,
                                         __gnu_pbds::pairing_heap_tag>;

Heap<int> q, r;           // 大根堆
Heap<int, greater<int>> minq; // 小根堆

auto it = q.push(3);      // 保存点迭代器
q.push(8);
q.modify(it, 10);         // 现在堆顶为 10
r.push(7);
q.join(r);                // r 变为空堆
q.erase(it);              // 删除原来的 3（现为 10）
```

树链剖分

用途： 将树上点路径 $u \rightarrow v$ 按顺序拆成 $O(\log n)$ 个 DFS 序连续段，便于交给线段树等序列数据结构处理；路径包含两个端点。

复杂度： 预处理时间与空间均为 $O(n)$ ； $\text{path}(u, v)$ 的时间和返回段数均为 $O(\log n)$ 。

返回值： 返回段按 $u \rightarrow v$ 排列； l, r 是 DFS 序编号 dfn ，不是原点编号，每段始终满足 $l \leq r$ ，表示闭区间 $[l, r]$ 。 $\text{rev}=1$ 时按 $r \rightarrow l$ 处理， $\text{rev}=0$ 时按 $l \rightarrow r$ 处理；交换律操作可忽略 rev 。

```
struct HLD {
    struct Seg {
        int l, r;
        bool rev;
    };
    int n, tim = 0;
    vector<vector<int>> e;
    vector<int> sz, son, fa, dep, top, dfn;

    HLD(int n_) : n(n_), e(n+1), sz(n+1), son(n+1), fa(n+1),
        dep(n+1), top(n+1), dfn(n+1) {}

    void addEdge(int u, int v) {
        e[u].push_back(v), e[v].push_back(u);
    }

    void dfs1(int u) {
        sz[u] = 1;
        dep[u] = dep[fa[u]] + 1;
        for (int v : e[u]) {
            if (v == fa[u]) continue;
            fa[v] = u;
            dfs1(v);
            sz[u] += sz[v];
            if (sz[v] > sz[son[u]]) son[u] = v;
        }
    }

    void dfs2(int u, int tp) {
        top[u] = tp, dfn[u] = ++tim;
        if (son[u]) dfs2(son[u], tp);
        for (int v : e[u]) {
            if (v == fa[u] || v == son[u]) continue;
            dfs2(v, v);
        }
    }

    void build(int rt = 1) {
        dfs1(rt);
        dfs2(rt, rt);
    }
}
```

```

vector<Seg> path(int u, int v) const {
    vector<Seg> a, b;
    while (top[u] != top[v]) {
        if (dep[top[u]] > dep[top[v]]) {
            a.push_back({dfn[top[u]], dfn[u], 1});
            u = fa[top[u]];
        } else {
            b.push_back({dfn[top[v]], dfn[v], 0});
            v = fa[top[v]];
        }
    }
    if (dep[u] > dep[v]) a.push_back({dfn[v], dfn[u], 1});
    else b.push_back({dfn[u], dfn[v], 0});
    reverse(b.begin(), b.end());
    a.insert(a.end(), b.begin(), b.end());
    return a;
}
};

```

Link-Cut Tree

用途： 维护动态森林的连边、删边、换根、连通性和路径异或值； `split(x,y)` 后，节点 `y` 的聚合值表示路径 $x \rightarrow y$ 。

复杂度： 每次 `access`、`makeRoot`、`findRoot`、`link`、`cut`、路径暴露或点修改的均摊复杂度为 $O(\log n)$ ；空间 $O(n)$ 。

聚合： 当前 `pushUp` 使用异或；更换维护信息时需要同步修改节点字段、合并和懒标记。

```
struct Splay {
    #define ls(u) tr[u].s[0]
    #define rs(u) tr[u].s[1]
    #define fa(u) tr[u].p
    int n;
    struct Node {
        pii s;
        int p;
        int val, w;
        int lazy;
    };
    vector<Node> tr;

    void pushUp(int u) {
        if (!u) return;
        tr[u].val = tr[ls(u)].val ^ tr[rs(u)].val ^ tr[u].w;
    }

    void setTag(int u, int x) {
        if (u == 0) return;
        swap(ls(u), rs(u));
        tr[u].lazy ^= x;
    }

    void pushDown(int u) {
        if (tr[u].lazy == 0) return;
        setTag(ls(u), tr[u].lazy), setTag(rs(u), tr[u].lazy);
        tr[u].lazy = 0;
    }

    bool isRoot(int u) {
        return ls(fa(u)) != u && rs(fa(u)) != u;
    }

    void pushAll(int u) {
        if (!isRoot(u)) pushAll(fa(u));
        pushDown(u);
    }

    void rotate(int x) {
        int y = fa(x), z = fa(y), k = rs(y) == x, w = tr[x].s[k^1];
        if (!isRoot(y)) tr[z].s[rs(z)==y] = x;
    }
}
```

```

    tr[y].s[k] = w, tr[x].s[k^1] = y;
    fa(w) = y, fa(y) = x, fa(x) = z;
    pushUp(y), pushUp(x), pushUp(z);
}

void splay(int x) {
    pushAll(x);
    while (!isRoot(x)) {
        int y = fa(x), z = fa(y);
        if (!isRoot(y)) ((rs(z)==y)^(rs(y)==x)) ? rotate(x) : rotate(y);
        rotate(x);
    }
}

int access(int x) {
    int y = 0;
    while (x) {
        splay(x);
        rs(x) = y;
        pushUp(x);
        y = x, x = fa(x);
    }
    return y;
}

void makeRoot(int x) {
    access(x);
    splay(x);
    setTag(x, 1);
}

int findRoot(int x) {
    access(x);
    splay(x);
    while (ls(x)) {
        pushDown(x);
        x = ls(x);
    }
    splay(x);
    return x;
}

void split(int x, int y) {
    makeRoot(x);
    access(y);
    splay(y);
}

void link(int x, int y) {
    makeRoot(x);
    if (findRoot(y) == x) return;
    fa(x) = y;
}

```

```

void setValue(int x, int w) {
    splay(x);
    tr[x].w = w;
    pushUp(x);
}

void cut(int x, int y) {
    makeRoot(x);
    if (findRoot(y) == x && fa(y) == x && !ls(y)) {
        fa(y) = rs(x) = 0;
        pushUp(x);
    }
}

Splay(int n_) {
    n = n_;
    tr.resize(n+1);
}

};

```

后缀自动机

用途： 对一个小写字母串构建后缀自动机，得到转移 DAG、parent 树、每个状态的 `len`、suffix link 和 `endpos` 大小，供后续按题意处理。

复杂度： 长度为 n 的串构建时间为 $O(nV)$ ，空间为 $O(nV)$ ，状态数不超过 $2n - 1$ ；字符集大小 V 视为常数时均为 $O(n)$ 。

接口： `build(s)` 可重复重建；`extend(c)` 在线加入编号为 `c` 的字符并返回新状态；`walk(t)` 返回子串 `t` 对应的状态，不存在时返回 `-1`。默认字符映射为 `c-'a'`，需要其他字符集时同步修改映射。

在线扩展： `extend` 动态维护 `nxt`、`link`、`len` 和 `last`；`tree`、`ord` 与汇总后的 `endpos` 由 `build` 收尾生成，手动调用 `extend` 后不可继续使用旧值。

结构	含义
<code>0</code>	初始状态，也是 parent 树的根
<code>nxt[u][c]</code>	状态 <code>u</code> 加字符 <code>c</code> 的转移，使用 32 位 <code>signed</code> 存状态编号， <code>0</code> 表示不存在
<code>link[u]</code>	状态 <code>u</code> 的 suffix link，也是 parent 树中的父亲
<code>len[u]</code>	状态 <code>u</code> 表示的子串中最大长度
<code>endpos[u]</code>	<code>build</code> 完成后，非根状态 <code>u</code> 的 <code>endpos</code> 集合大小，即其表示的子串出现次数
<code>tree[u]</code>	<code>build</code> 完成后，状态 <code>u</code> 在 parent 树中的所有儿子
<code>ord</code>	<code>build</code> 完成后，所有状态按 <code>len</code> 非降序排列；正序用于从短到长 DP，逆序用于向 suffix link 汇总
<code>last</code>	整个字符串对应的状态；沿 suffix link 到根依次经过所有后缀的终止状态

```
template<int V = 26>
struct SuffixAutomaton {
    vector<array<signed, V>> nxt;
    vector<int> link, len, endpos;
    vector<vector<int>> tree;
    vector<int> ord;
    int last;

    SuffixAutomaton() { build(""); }
    SuffixAutomaton(const string &s) { build(s); }

    void build(const string &s) {
        nxt.clear(), link.clear(), len.clear(), endpos.clear();
        int cap = s.size() * 2 + 1;
        nxt.reserve(cap), link.reserve(cap), len.reserve(cap), endpos.reserve(cap);
        last = newNode();
        for (char c : s) extend(c - 'a');

        int n = s.size(), sz = nxt.size();
        vector<int> cnt(n + 1);
        ord.resize(sz);
        for (int u = 0; u < sz; u++) cnt[len[u]]++;
        for (int i = 1; i <= n; i++) cnt[i] += cnt[i - 1];
        for (int u = sz - 1; u >= 0; u--)
            ord[--cnt[len[u]]] = u;
    }
};
```

```

    tree.assign(sz, {});
    for (int u = 1; u < sz; u++) tree[link[u]].push_back(u);
    for (int i = sz - 1; i > 0; i--) {
        int u = ord[i];
        endpos[link[u]] += endpos[u];
    }
}

int walk(const string &s) const {
    int u = 0;
    for (char c : s) {
        u = nxt[u][c - 'a'];
        if (!u) return -1;
    }
    return u;
}

int newNode() {
    int u = nxt.size();
    nxt.push_back({}), link.push_back(-1), len.push_back(0), endpos.push_back(0);
    return u;
}

int extend(int c) {
    int cur = newNode(), p = last;
    len[cur] = len[last] + 1, endpos[cur] = 1;
    while (p != -1 && !nxt[p][c]) nxt[p][c] = cur, p = link[p];
    if (p == -1) link[cur] = 0;
    else {
        int q = nxt[p][c];
        if (len[p] + 1 == len[q]) link[cur] = q;
        else {
            int clone = newNode();
            nxt[clone] = nxt[q], link[clone] = link[q], len[clone] = len[p] + 1;
            while (p != -1 && nxt[p][c] == q)
                nxt[p][c] = clone, p = link[p];
            link[q] = link[cur] = clone;
        }
    }
    last = cur;
    return cur;
}
};

```

图论与网络流

顺序	模板	用途	复杂度
01	强连通分量	有向图强连通分量与缩点 DAG	$O(V + E)$
02	边双连通分量	桥、边双连通分量与桥树	$O(V + E)$
03	点双连通分量	割点、点双连通分量与圆方树	$O(V + E)$
04	2-SAT	2-SAT 可满足性与一组赋值	$O(V + E)$
05	Dinic 最大流	通用容量网络最大流	一般上界 $O(V^2E)$
06	HLPP 最大流	最高标号预流推进	$O(V^2\sqrt{E})$
07	有上下界流	可行流及 $s-t$ 最小、最大流	$O(V^2E)$
08	最小费用流	最小费用最大流及限流增广	$O(VE + AE\log V)$
09	有上下界最小费用流	最小费用 b -流及定流量 $s-t$ 流	$O(VE + AE\log V)$
10	二分图最大权匹配	KM 求二分图最大权匹配与完美匹配	$O(n^2m)$
11	图匹配与二分图常用性质	匹配、覆盖、独立集、Hall/Kőnig 定理与 DAG 路径覆盖	-

强连通分量

用途： 将有向图分解为强连通分量；同一分量内任意两点互相可达，缩点后得到 DAG。允许自环和平行边。

编号： 点编号为 $0..n-1$ 。

操作/结构	含义	复杂度	备注
<code>SCC(n)</code>	传入点数 n ，构造空图	$O(n)$	点数固定
<code>addEdge(u,v)</code>	传入端点 u,v ，添加有向边 $u \rightarrow v$	均摊 $O(1)$	允许自环和平行边
<code>build()</code>	无传入和返回值，重新计算强连通分量	$O(n + m)$	使用递归 DFS；继续加边后可重建
<code>bel[u]</code>	维护点 u 所属的强连通分量编号	-	<code>build()</code> 后有效；跨分量边 $u \rightarrow v$ 满足 $bel[u] < bel[v]$
<code>comp[i]</code>	维护第 i 个强连通分量的点集	-	分量编号为拓扑序
<code>dag()</code>	无传入，返回去重后的缩点 DAG 邻接表	$O(n + m)$	要求先调用 <code>build()</code>

```
struct SCC {
    int n, tim;
    vector<vector<int>> e, comp;
    vector<int> dfn, low, bel, stk;

    SCC(int n_ = 0) : n(n_), e(n_) {}

    void addEdge(int u, int v) {
        e[u].push_back(v);
    }

    void dfs(int u) {
        dfn[u] = low[u] = tim++;
        stk.push_back(u);
        for (int v : e[u]) {
            if (dfn[v] == -1) {
                dfs(v);
                low[u] = min(low[u], low[v]);
            } else if (bel[v] == -1) {
                low[u] = min(low[u], dfn[v]);
            }
        }
        if (dfn[u] != low[u]) return;
        comp.push_back({});
        while (1) {
            int v = stk.back();
            stk.pop_back();
            bel[v] = comp.size()-1;
            comp.back().push_back(v);
            if (v == u) break;
        }
    }

    void build() {
        tim = 0, comp.clear(), stk.clear();
    }
};
```

```

    dfn.assign(n, -1), low.resize(n), bel.assign(n, -1);
    for (int s = 0; s < n; s++) {
        if (dfn[s] != -1) continue;
        dfs(s);
    }
    reverse(comp.begin(), comp.end());
    for (int i = 0; i < (int)comp.size(); i++)
        for (int u : comp[i]) bel[u] = i;
}

vector<vector<int>> dag() const {
    vector<vector<int>> g(comp.size());
    vector<int> vis(comp.size(), -1);
    for (int a = 0; a < (int)comp.size(); a++) {
        for (int u : comp[a]) {
            for (int v : e[u]) {
                int b = bel[v];
                if (a == b || vis[b] == a) continue;
                vis[b] = a;
                g[a].push_back(b);
            }
        }
    }
    return g;
}
};

```

边双连通分量

用途： 将无向图按桥分解为边双连通分量；同一分量内任意两点之间不存在必经边。允许自环和平行边，自环不会成为桥。

编号： 点编号为 $0..n-1$ ，边按加入顺序从 0 开始编号。

依赖： `pii` 为 `array<int,2>`。

操作/结构	含义	复杂度	备注
<code>EBCC(n)</code>	传入点数 <code>n</code> ，构造空图	$O(n)$	点数固定
<code>addEdge(u,v)</code>	传入端点 <code>u,v</code> ，添加无向边并返回边号	均摊 $O(1)$	允许自环和平行边
<code>build()</code>	无传入和返回值，重新计算桥与边双连通分量	$O(n + m)$	使用递归 DFS；继续加边后可重建
<code>edge[i]</code>	维护第 <code>i</code> 条边的两个端点	-	端点类型为 <code>pii</code>
<code>bridge[i]</code>	维护第 <code>i</code> 条边是否为桥	-	<code>build()</code> 后有效；桥为 <code>1</code> ，否则为 <code>0</code>
<code>bel[u]</code>	维护点 <code>u</code> 所属的边双连通分量编号	-	<code>build()</code> 后有效，每个点恰属一个分量
<code>comp[i]</code>	维护第 <code>i</code> 个边双连通分量的点集	-	包含孤立点形成的单点分量
<code>bridgeTree()</code>	无传入，返回以边双为点、桥为边的森林邻接表	$O(n + m)$	要求先调用 <code>build()</code> ；节点 <code>i</code> 对应 <code>comp[i]</code>

```
struct EBCC {
    int n, tim;
    vector<pii> edge;
    vector<vector<int>> e, comp;
    vector<int> dfn, low, bridge, bel;

    EBCC(int n_ = 0) : n(n_), e(n_) {}

    int addEdge(int u, int v) {
        int id = edge.size();
        edge.push_back({u, v});
        e[u].push_back(id), e[v].push_back(id);
        return id;
    }

    int other(int id, int u) const {
        return edge[id][0]^edge[id][1]^u;
    }

    void dfs(int u, int pe) {
        dfn[u] = low[u] = tim++;
        for (int id : e[u]) {
            if (id == pe) continue;
            int v = other(id, u);
            if (dfn[v] == -1) {
                dfs(v, id);
                low[u] = min(low[u], low[v]);
                if (low[v] > dfn[u]) bridge[id] = 1;
            } else {
                low[u] = min(low[u], dfn[v]);
            }
        }
    }
};
```

```

    }
}

void paint(int u, int id) {
    bel[u] = id;
    comp[id].push_back(u);
    for (int i : e[u]) {
        int v = other(i, u);
        if (!bridge[i] && bel[v] == -1) paint(v, id);
    }
}

void build() {
    tim = 0, comp.clear();
    dfn.assign(n, -1), low.resize(n), bridge.assign(edge.size(), 0);
    for (int s = 0; s < n; s++) {
        if (dfn[s] != -1) continue;
        dfs(s, -1);
    }

    bel.assign(n, -1);
    for (int s = 0; s < n; s++) {
        if (bel[s] != -1) continue;
        int id = comp.size();
        comp.push_back({});
        paint(s, id);
    }
}

vector<vector<int>> bridgeTree() const {
    vector<vector<int>> g(comp.size());
    for (int i = 0; i < (int)edge.size(); i++) {
        if (!bridge[i]) continue;
        int u = bel[edge[i][0]], v = bel[edge[i][1]];
        g[u].push_back(v), g[v].push_back(u);
    }
    return g;
}

};

```

点双连通分量

用途： 将无自环无向图按割点分解为极大点双连通块；点双不是点集划分。允许平行边。

编号： 点编号为 $0..n-1$ ，边按加入顺序从 0 开始编号。

依赖： pii 为 `array<int,2>`。

操作/结构	含义	复杂度	备注
<code>VBCC(n)</code>	传入点数 n ，构造空图	$O(n)$	点数固定
<code>addEdge(u,v)</code>	传入端点 u,v ，添加无向边并返回边号	均摊 $O(1)$	要求 $u \neq v$ ；允许平行边
<code>build()</code>	无传入和返回值，重新计算割点与点双连通分量	$O(n+m)$	使用递归 DFS；继续加边后可重建
<code>edge[i]</code>	维护第 i 条边的两个端点	-	端点类型为 pii
<code>cut[u]</code>	维护点 u 是否为割点	-	<code>build()</code> 后有效；割点为 1 ，否则为 0
<code>comp[i]</code>	维护第 i 个点双连通分量的点集	-	割点可出现多次；桥形成二点分量，孤立点形成单点分量
<code>blockCutTree()</code>	无传入，返回点双节点与原图点相连的森林邻接表	$O(n+m)$	原图点为 $0..n-1$ ，点双 i 对应节点 $n+i$

```
struct VBCC {
    int n, tim;
    vector<pii> edge;
    vector<vector<int>> e, comp;
    vector<int> dfn, low, cut, stk, mark;

    VBCC(int n_ = 0) : n(n_), e(n_) {}

    int addEdge(int u, int v) {
        int id = edge.size();
        edge.push_back({u, v});
        e[u].push_back(id), e[v].push_back(id);
        return id;
    }

    int other(int id, int u) const {
        return edge[id][0]^edge[id][1]^u;
    }

    void dfs(int u, int pe, int root) {
        dfn[u] = low[u] = tim++;
        int child = 0;
        for (int id : e[u]) {
            if (id == pe) continue;
            int v = other(id, u);
            if (dfn[v] == -1) {
                child++;
                stk.push_back(id);
                dfs(v, id, root);
                low[u] = min(low[u], low[v]);
                if (low[v] < dfn[u]) continue;
                if (u != root || child > 1) cut[u] = 1;
                int c = comp.size();
            }
        }
    }
};
```

```

        comp.push_back({});
        while (1) {
            int i = stk.back();
            stk.pop_back();
            for (int x : edge[i]) {
                if (mark[x] == c) continue;
                mark[x] = c;
                comp[c].push_back(x);
            }
            if (i == id) break;
        }
    } else if (dfn[v] < dfn[u]) {
        low[u] = min(low[u], dfn[v]);
        stk.push_back(id);
    }
}
if (u == root && child == 0) comp.push_back({u});
}

void build() {
    tim = 0, comp.clear();
    dfn.assign(n, -1), low.resize(n), cut.assign(n, 0);
    stk.clear(), mark.assign(n, -1);
    for (int s = 0; s < n; s++) {
        if (dfn[s] != -1) continue;
        dfs(s, -1, s);
    }
}

vector<vector<int>> blockCutTree() const {
    vector<vector<int>> g(n+comp.size());
    for (int i = 0; i < (int)comp.size(); i++) {
        int v = n+i;
        for (int u : comp[i]) g[u].push_back(v), g[v].push_back(u);
    }
    return g;
}
};

```

2-SAT

用途： 给定若干形如 $(x_u = f) \vee (x_v = g)$ 的子句，判断是否存在一个布尔赋值使所有子句成立，并在可满足时给出一组赋值。通过蕴含图上的强连通分量求解；文字 $x_i=0$ 、 $x_i=1$ 分别编码为 $2*i$ 、 $2*i+1$ 。

编号： 变量编号为 $0..n-1$ ；`addClause(u,f,v,g)` 中 f 、 g 取 0 或 1 ，表示对应文字是 $x_u = f$ 、 $x_v = g$ 。

依赖： 无。

操作/结构	含义	复杂度	备注
<code>TwoSat(n)</code>	传入变量数 n ，构造空公式	$O(n)$	变量数固定
<code>addClause(u,f,v,g)</code>	添加子句 $(x_u = f) \vee (x_v = g)$	均摊 $O(1)$	f 、 g 只取 0 或 1 ；允许重复子句
<code>satisfiable()</code>	无传入，判断公式是否可满足并保存一组赋值	$O(n + m)$	可重复调用；继续加子句后可重算
<code>ans[i]</code>	维护变量 x_i 的一组可行赋值	-	仅在最近一次 <code>satisfiable()</code> 返回 1 时有效，值为 0 或 1

```
struct TwoSat {
    int n, tim, cnt;
    vector<vector<int>> e;
    vector<int> dfn, low, bel, stk, ans;

    TwoSat(int n_ = 0) : n(n_), e(2*n_), ans(n_) {}

    void addClause(int u, bool f, int v, bool g) {
        e[2*u+!f].push_back(2*v+g);
        e[2*v+!g].push_back(2*u+f);
    }

    void dfs(int u) {
        dfn[u] = low[u] = tim++;
        stk.push_back(u);
        for (int v : e[u]) {
            if (dfn[v] == -1) {
                dfs(v);
                low[u] = min(low[u], low[v]);
            } else if (bel[v] == -1) {
                low[u] = min(low[u], dfn[v]);
            }
        }
        if (dfn[u] != low[u]) return;
        while (1) {
            int v = stk.back();
            stk.pop_back();
            bel[v] = cnt;
            if (v == u) break;
        }
        cnt++;
    }

    bool satisfiable() {
        tim = cnt = 0;
```

```
stk.clear();
dfn.assign(2*n, -1), low.resize(2*n), bel.assign(2*n, -1);
for (int i = 0; i < 2*n; i++) if (dfn[i] == -1) dfs(i);
for (int i = 0; i < n; i++) {
    if (bel[2*i] == bel[2*i+1]) return 0;
    ans[i] = bel[2*i] > bel[2*i+1];
}
return 1;
}
};
```

Dinic 最大流

用途：在有向容量网络中计算从源点 S 到汇点 T 的最大流；平行边与反向同端点边分别保留。

编号：点可以从 0 开始并自动扩展；`addEdge` 返回正向残量边编号，正反边可用 i^1 互相定位。要求 $S \neq T$ 。

操作/结构	含义	复杂度	备注
<code>Flow(n)</code>	传入点数 n ，构造空网络	时间、空间 $O(n)$	可默认构造，之后由 <code>addEdge</code> 自动扩点
<code>addEdge(u,v,cap)</code>	传入端点与容量，添加有向边并返回正向残量边编号	均摊 $O(1)$	自动扩点的总开销为 $O(V)$ ；平行边分别保留
<code>maxFlow(S,T,limit)</code>	传入源点、汇点与流量上限，从当前残量网络继续增广并返回本次增加的流量	一般时间 $O(V^2E)$ ，额外空间 $O(V)$	要求 $S \neq T$ ；省略 <code>limit</code> 时求最大流
<code>getEdge(i)</code>	传入正向残量边编号 i ，返回原边的起点、终点、容量与当前流量	$O(1)$	i 为 <code>addEdge</code> 的返回值
<code>changeEdge(i,cap,flow)</code>	传入边号、新容量与新流量，修改对应原边	$O(1)$	要求 $0 \leq flow \leq cap$

```
const int INF = 1e18;

struct Flow {
    int S, T;
    vector<array<int, 3>> e;
    vector<int> h, d, cur;

    Flow(int n = 0) : h(n), d(n), cur(n) {
        e.resize(2);
    }

    void resize(int n) {
        h.resize(n), d.resize(n), cur.resize(n);
    }

    int addEdge(int u, int v, int cap) {
        int n = max(u, v)+1;
        if ((int)h.size() < n) resize(n);
        int id = e.size();
        e.push_back({v, cap, h[u]}), h[u] = id;
        e.push_back({u, 0, h[v]}), h[v] = id^1;
        return id;
    }

    struct Edge {
        int from, to, cap, flow;
    };

    Edge getEdge(int i) {
        return {e[i^1][0], e[i][0], e[i][1]+e[i^1][1], e[i^1][1]};
    }

    void changeEdge(int i, int cap, int flow) {
```

```

        e[i][1] = cap-flow, e[i^1][1] = flow;
    }

    bool bfs() {
        fill(d.begin(), d.end(), -1);
        queue<int> q;
        d[S] = 0, q.push(S);
        while (q.size()) {
            int u = q.front();
            q.pop();
            for (int i = h[u]; i; i = e[i][2]) {
                int v = e[i][0];
                if (e[i][1] && d[v] == -1) d[v] = d[u]+1, q.push(v);
            }
        }
        return d[T] != -1;
    }

    int dfs(int u, int mf) {
        if (u == T || mf == 0) return mf;
        int sum = 0;
        for (int &i = cur[u]; i; i = e[i][2]) {
            int v = e[i][0];
            if (!e[i][1] || d[v] != d[u]+1) continue;
            int f = dfs(v, min(mf, e[i][1]));
            e[i][1] -= f, e[i^1][1] += f;
            sum += f, mf -= f;
            if (mf == 0) break;
        }
        if (sum == 0) d[u] = -1;
        return sum;
    }

    int maxFlow(int S_, int T_, int limit = INF) {
        S = S_, T = T_;
        int n = max(S, T)+1;
        if ((int)h.size() < n) resize(n);
        int flow = 0;
        while (flow < limit && bfs()) {
            cur = h;
            flow += dfs(S, limit-flow);
        }
        return flow;
    }
};

```

HLPP 最高标号预流推进

用途： 在稠密图或 Dinic 容易退化的网络中计算最大流；使用最高标号选择、Gap 优化与全局重标号。

复杂度： 常用上界记作 $O(V^2\sqrt{E})$ ，空间 $O(V + E)$ 。

编号： 构造 `Flow(n)` 后，通常使用 `1..n` 的点编号，`0` 号位置作为备用；源点和汇点应不同。

```
struct Flow {
    struct Edge {
        int to, c, rev;
        Edge(int to, int c, int rev) : to(to), c(c), rev(rev) {}
    };

    int n, s, t;
    int maxh, maxgaph, workcnt;
    vector<vector<Edge>> vec;
    vector<int> ov, h, cur;
    vector<int> ovList, ovNxt;
    vector<int> gap, gapPrv, gapNxt;

    Flow(int n)
        : n(n), maxh(0), maxgaph(0), workcnt(0), vec(n + 1),
          ov(n + 1), h(n + 1), cur(n + 1), ovList(n + 1, -1),
          ovNxt(n + 1, -1), gap(n + 1, -1), gapPrv(n + 1, -1),
          gapNxt(n + 1, -1) {}

    void addEdge(int u, int v, int c) {
        vec[u].push_back(Edge(v, c, vec[v].size()));
        vec[v].push_back(Edge(u, 0, vec[u].size() - 1));
    }

    int maxFlow(int s_, int t_) {
        s = s_, t = t_;
        globalRelabel();
        for (auto &e : vec[s]) {
            if (e.c) pushFlow(s, e, e.c);
        }
        for (; maxh >= 0; --maxh) {
            while (~ovList[maxh]) {
                int x = ovList[maxh];
                ovList[maxh] = ovNxt[x];
                ovNxt[x] = -1;

                discharge(x);

                if (workcnt > (n << 2)) globalRelabel();
            }
        }
        return ov[t];
    }
};
```

```

private:
void discharge(int x) {
    int nh = n, sz = vec[x].size();
    for (int i = cur[x]; i < sz; ++i) {
        auto &e = vec[x][i];
        if (e.c > 0) {
            if (h[x] == h[e.to] + 1) {
                pushFlow(x, e, min(ov[x], e.c));
                if (ov[x] == 0) {
                    cur[x] = i;
                    return;
                }
            } else {
                nh = min(nh, h[e.to] + 1);
            }
        }
    }
    for (int i = 0; i < cur[x]; ++i) {
        auto &e = vec[x][i];
        if (e.c > 0) nh = min(nh, h[e.to] + 1);
    }
    cur[x] = 0;
    ++workcnt;
    int oldh = h[x], head = gap[oldh];
    if (~gapNxt[head]) {
        setHeight(x, nh);
    } else {
        for (int i = oldh; i <= maxgaph; ++i) {
            for (int j = gap[i]; ~j; ) {
                int nxt = gapNxt[j];
                h[j] = n;
                gapPrv[j] = gapNxt[j] = -1;
                j = nxt;
            }
            gap[i] = -1;
        }
        maxgaph = oldh - 1;
    }
}

void globalRelabel() {
    workcnt = maxh = maxgaph = 0;
    fill(h.begin(), h.end(), n);
    h[t] = 0;
    fill(gapPrv.begin(), gapPrv.end(), -1), fill(gapNxt.begin(), gapNxt.end(), -1);
    fill(gap.begin(), gap.end(), -1), fill(ovList.begin(), ovList.end(), -1);
    fill(ovNxt.begin(), ovNxt.end(), -1), fill(cur.begin(), cur.end(), 0);

    queue<int> q;
    q.push(t);
    while (!q.empty()) {
        int x = q.front();
        q.pop();
    }
}

```

```

        for (auto &e : vec[x]) {
            if (h[e.to] == n && e.to != s && vec[e.to][e.rev].c > 0) {
                setHeight(e.to, h[x] + 1);
                q.push(e.to);
            }
        }
    }
}

void setHeight(int x, int newh) {
    if (~gapPrv[x]) {
        if (gapPrv[x] == x) {
            gap[h[x]] = gapNxt[x];
            if (~gapNxt[x]) gapPrv[gapNxt[x]] = gapNxt[x];
        } else {
            gapNxt[gapPrv[x]] = gapNxt[x];
            if (~gapNxt[x]) gapPrv[gapNxt[x]] = gapPrv[x];
        }
        gapPrv[x] = gapNxt[x] = -1;
    }
    h[x] = newh;
    if (h[x] >= n) return;
    maxgaph = max(maxgaph, h[x]);
    if (ov[x] > 0) {
        maxh = max(maxh, h[x]);
        ovNxt[x] = ovList[h[x]];
        ovList[h[x]] = x;
    }
    gapNxt[x] = gap[h[x]];
    if (~gapNxt[x]) gapPrv[gapNxt[x]] = x;
    gap[h[x]] = gapPrv[x] = x;
}

void pushFlow(int from, Edge &e, int flow) {
    if (!ov[e.to] && e.to != t && h[e.to] < n) {
        maxh = max(maxh, h[e.to]);
        ovNxt[e.to] = ovList[h[e.to]];
        ovList[h[e.to]] = e.to;
    }
    e.c -= flow, vec[e.to][e.rev].c += flow;
    ov[from] -= flow, ov[e.to] += flow;
}

};

```

有上下界流

用途： 对每条有向边限制 $l \leq f \leq r$ ，求满足流量守恒的可行流，或求非负 s - t 流量的最小值与最大值。

依赖： 使用 Dinic 最大流中的 `Flow`；`INF` 应大于可能的总流量，所有容量、平衡量与答案应在 `i64` 范围内。

操作/结构	含义	复杂度	备注
<code>BoundedFlow(n)</code>	传入点数 <code>n</code> ，构造空网络	$O(1)$	点编号为 <code>0..n-1</code>
<code>addEdge(u,v,l,r)</code>	传入端点与流量上下界，添加有向边	均摊 $O(1)$	要求 $0 \leq l \leq r$ ，边按加入顺序编号
<code>feasible()</code>	无传入，判断并返回是否存在循环可行流	时间 $O(V^2E)$ ，空间 $O(V + E)$	每次调用均从原始边重新建图
<code>ans[i]</code>	维护第 <code>i</code> 条边在循环可行流中的流量	-	<code>feasible()</code> 返回 <code>1</code> 后有效
<code>flowRange(s,t)</code>	传入源点与汇点，返回 <code>Result{ok,minFlow,maxFlow}</code>	时间 $O(V^2E)$ ，空间 $O(V + E)$	只考虑非负流量；每次调用均从原始边重新建图
<code>minAns[i]</code> 、 <code>maxAns[i]</code>	分别维护最小流量、最大流量方案中第 <code>i</code> 条边的流量	-	<code>flowRange(s,t)</code> 返回的 <code>ok</code> 为 <code>1</code> 后有效

先令每条边流过下界 l ，再向残量网络加入容量 $r - l$ 。记 `bal[u]` 为下界流造成的流入减流出，则加边 $u \rightarrow v$ 时有 `bal[u] -= 1`，`bal[v] += 1`。对 `bal[i] > 0` 加 $SS \rightarrow i$ ，对 `bal[i] < 0` 加 $i \rightarrow TT$ ；从 SS 到 TT 的边全部满流时得到一组特解。

求 s - t 流量范围时先加辅助边 $t \rightarrow s$ 并求特解，其流量记为 `base`。删除辅助边与超级源汇后，在同一残量网络中继续求 $s \rightarrow t$ 最大流得到最大值；反向增广至多 `base` 单位的 $t \rightarrow s$ 流得到最小值。

```
struct BoundedFlow {
    struct Edge {
        int u, v, l, r;
    };
    struct Result {
        bool ok;
        int minFlow, maxFlow;
    };

    int n;
    vector<Edge> e;
    vector<int> ans, minAns, maxAns;

    BoundedFlow(int n_ = 0) : n(n_) {}

    void addEdge(int u, int v, int l, int r) {
        e.push_back({u, v, l, r});
    }

    bool feasible() {
        ans.clear();
        Flow f(n+2);
        vector<int> id, bal;
        init(f, id, bal);
        if (!balance(f, bal)) return 0;
        ans = getFlow(f, id);
        return 1;
    }
};
```

```

}

Result flowRange(int s, int t) {
    minAns.clear(), maxAns.clear();
    Flow f(n+2);
    vector<int> id, bal;
    init(f, id, bal);
    int back = f.addEdge(t, s, INF);
    if (!balance(f, bal)) return {0, 0, 0};

    int base = f.getEdge(back).flow;
    f.changeEdge(back, 0, 0);
    Flow mn = f, mx = f;
    int minFlow = base-mn.maxFlow(t, s, base);
    int maxFlow = base+mx.maxFlow(s, t);
    minAns = getFlow(mn, id), maxAns = getFlow(mx, id);
    return {1, minFlow, maxFlow};
}

private:
void init(Flow &f, vector<int> &id, vector<int> &bal) {
    id.resize(e.size()), bal.assign(n, 0);
    for (int i = 0; i < (int)e.size(); i++) {
        auto [u, v, l, r] = e[i];
        id[i] = f.addEdge(u, v, r-l);
        bal[u] -= l, bal[v] += l;
    }
}

bool balance(Flow &f, vector<int> &bal) {
    int S = n, T = n+1, need = 0;
    vector<int> id;
    for (int i = 0; i < n; i++) {
        if (bal[i] > 0) id.push_back(f.addEdge(S, i, bal[i])); need += bal[i];
        if (bal[i] < 0) id.push_back(f.addEdge(i, T, -bal[i]));
    }
    if (f.maxFlow(S, T, need) != need) return 0;
    for (int i : id) f.changeEdge(i, 0, 0);
    return 1;
}

vector<int> getFlow(Flow &f, vector<int> &id) {
    vector<int> a(e.size());
    for (int i = 0; i < (int)e.size(); i++) {
        a[i] = e[i].l+f.getEdge(id[i]).flow;
    }
    return a;
}
};

```

最小费用流

用途：在有向容量网络中，从 s 到 t 至多发送 $limit$ 单位流量；先最大化流量，再在该流量下最小化总费用。

类型：`MinCostFlow<T>` 中 T 只表示单边费用、势能、距离与总费用的类型，容量和流量固定为 `int`；费用为浮点数时可取 `double`。

条件：当前残量网络中不能存在从 s 可达的负费用环，且 $s \neq t$ 。若整数总费用可能超出 `i64`，应使用 `i128`；浮点费用按 T 的精度比较。

操作/结构	含义	复杂度	备注
<code>MinCostFlow<T>(n)</code>	传入点数 n ，构造空网络	时间、空间 $O(n)$	点编号从 0 开始，也可默认构造后自动扩点
<code>init(n)</code>	传入点数 n ，清空原图并重新建空网络	时间、空间 $O(n)$	已保存的边编号失效
<code>addEdge(u,v,cap,cost)</code>	传入端点、容量与单位费用，添加有向边并返回正向残量边编号	均摊 $O(1)$	<code>cap</code> 为 <code>int</code> ， <code>cost</code> 为 T ；平行边分别保留
<code>flow(s,t,limit)</code>	传入源点、汇点与流量上限，返回本次增加的流量与费用	$O(VE + AE \log V)$	A 为增广次数；省略 <code>limit</code> 时求最小费用最大流
<code>getEdge(i)</code>	传入正向残量边编号 i ，返回原边的端点、容量、流量与单位费用	$O(1)$	i 为 <code>addEdge</code> 的返回值
<code>changeEdge(i,cap,flow)</code>	传入边号、新容量与新流量，修改对应原边	$O(1)$	要求 $0 \leq flow \leq cap$ ，单位费用不变

先用 Bellman-Ford 求从 s 出发的初始最短距离作为势能 h ，使可达残量边的约化费用非负。之后每轮用 Dijkstra 求最短增广路，并令所有本轮可达点满足 $h[i] += dis[i]$ 。对浮点费用，将理论上非负但因误差略小于 0 的约化费用截为 0 。

```
template<class T>
struct MinCostFlow {
    struct _Edge {
        int to, cap;
        T cost;
    };
    struct Edge {
        int from, to, cap, flow;
        T cost;
    };

    int n;
    vector<_Edge> e;
    vector<vector<int>> g;
    vector<T> h, dis;
    vector<int> pre;

    MinCostFlow(int n_ = 0) {
        init(n_);
    }

    void init(int n_) {
        n = n_, e.clear(), g.assign(n, {});
        h.resize(n), dis.resize(n), pre.resize(n);
    }
};
```

```

int addEdge(int u, int v, int cap, T cost) {
    int n_ = max(u, v)+1;
    if (n < n_) resize(n_);
    int id = e.size();
    g[u].push_back(id), e.push_back({v, cap, cost});
    g[v].push_back(id^1), e.push_back({u, 0, -cost});
    return id;
}

Edge getEdge(int i) {
    return {e[i^1].to, e[i].to, e[i].cap+e[i^1].cap, e[i^1].cap, e[i].cost};
}

void changeEdge(int i, int cap, int flow) {
    e[i].cap = cap-flow, e[i^1].cap = flow;
}

pair<int, T> flow(int s, int t, int limit = INF) {
    int n_ = max(s, t)+1;
    if (n < n_) resize(n_);
    initPotential(s);
    int ans = 0;
    T cost = 0, inf = infCost();
    while (ans < limit && dijkstra(s, t)) {
        for (int i = 0; i < n; i++) {
            if (dis[i] != inf) h[i] += dis[i];
        }
        int aug = limit-ans;
        for (int v = t; v != s; v = e[pre[v]^1].to) {
            aug = min(aug, e[pre[v]].cap);
        }
        for (int v = t; v != s; v = e[pre[v]^1].to) {
            e[pre[v]].cap -= aug, e[pre[v]^1].cap += aug;
        }
        ans += aug, cost += T(aug)*h[t];
    }
    return {ans, cost};
}

private:
T infCost() {
    return numeric_limits<T>::max()/4;
}

void resize(int n_) {
    n = n_, g.resize(n), h.resize(n), dis.resize(n), pre.resize(n);
}

void initPotential(int s) {
    T inf = infCost();
    h.assign(n, inf), h[s] = 0;
    for (int k = 1; k < n; k++) {

```

```

    bool update = 0;
    for (int u = 0; u < n; u++) {
        if (h[u] == inf) continue;
        for (int i : g[u]) {
            int v = e[i].to;
            if (e[i].cap && h[v] > h[u]+e[i].cost) {
                h[v] = h[u]+e[i].cost, update = 1;
            }
        }
    }
    if (!update) break;
}
for (T &x : h) if (x == inf) x = 0;
}

bool dijkstra(int s, int t) {
    T inf = infCost();
    dis.assign(n, inf), pre.assign(n, -1);
    priority_queue<pair<T, int>, vector<pair<T, int>>, greater<pair<T, int>>> q;
    dis[s] = 0, q.push({0, s});
    while (q.size()) {
        auto [d, u] = q.top();
        q.pop();
        if (d > dis[u]) continue;
        for (int i : g[u]) {
            if (!e[i].cap) continue;
            int v = e[i].to;
            T w = e[i].cost+h[u]-h[v];
            if constexpr (is_floating_point_v<T>) w = max(w, T(0));
            if (dis[v] > d+w) {
                dis[v] = d+w, pre[v] = i;
                q.push({dis[v], v});
            }
        }
    }
    return dis[t] != inf;
}
};

```

有上下界最小费用流

用途： 对每条有向边限制 $l \leq f \leq r$ 并给定单位费用，求指定各点净流出量 $b[i]$ 的最小费用可行流；也可求指定流量的最小费用 $s-t$ 流。

依赖： 使用最小费用流中的 `MinCostFlow<T>`；容量、流量与 $b[i]$ 为 `int`，费用为 `T`。若整数总费用可能超出 `i64`，应使用 `i128`。

操作/结构	含义	复杂度	备注
<code>BoundedMinCostFlow<T>(n)</code>	传入点数 n ，构造空网络	$O(1)$	点编号为 $0..n-1$
<code>addEdge(u,v,l,r,cost)</code>	传入端点、流量上下界与单位费用，添加有向边	均摊 $O(1)$	要求 $l \leq r$ ，边按加入顺序编号
<code>minCost(b)</code>	传入各点目标净流出量 b ，返回 <code>Result{ok,cost}</code>	$O(VE + AE \log V)$	要求 <code>b.size()==n</code> ；省略 b 时求最小费用循环流
<code>flow(s,t,need)</code>	传入源点、汇点与指定流量，返回 <code>Result{ok,cost}</code>	$O(VE + AE \log V)$	求流量恰为 <code>need</code> 的最小费用流；要求 $s \neq t$ 且 <code>need >= 0</code>
<code>ans[i]</code>	维护最优方案中第 i 条边的流量	-	<code>minCost</code> 或 <code>flow</code> 返回的 <code>ok</code> 为 <code>1</code> 后有效

对费用非负的边先取下界 l ，加入容量 $r-l$ 、费用 `cost` 的正向调整边；对费用为负的边先取上界 r ，加入容量 $r-l$ 、费用 `-cost` 的反向调整边。这样所有调整费用非负，且每种合法原流都与一组调整流——对应。

记 $bal[i]$ 为基准流在点 i 的净流出量。对 $d=b[i]-bal[i]$ ，若 $d>0$ 则加 $SS \rightarrow i$ ，若 $d<0$ 则加 $i \rightarrow TT$ ； SS 发出的辅助边全部满流时得到最优解。最终原边流量分别为 $l+调整量$ 或 $r-调整量$ 。

```
template<class T>
struct BoundedMinCostFlow {
    struct Edge {
        int u, v, l, r;
        T cost;
    };
    struct Result {
        bool ok;
        T cost;
    };

    int n;
    vector<Edge> e;
    vector<int> ans;

    BoundedMinCostFlow(int n_ = 0) : n(n_) {}

    void addEdge(int u, int v, int l, int r, T cost) {
        e.push_back({u, v, l, r, cost});
    }

    Result minCost(vector<int> b = {}) {
        ans.clear();
        if (b.empty()) b.assign(n, 0);
        MinCostFlow<T> f(n+2);
        vector<int> id(e.size()), bal(n);
        T cost = 0;
    }
```

```

    for (int i = 0; i < (int)e.size(); i++) {
        auto [u, v, l, r, c] = e[i];
        if (c >= T(0)) {
            id[i] = f.addEdge(u, v, r-l, c);
            bal[u] += l, bal[v] -= l, cost += T(l)*c;
        } else {
            id[i] = f.addEdge(v, u, r-l, -c);
            bal[u] += r, bal[v] -= r, cost += T(r)*c;
        }
    }
}

int SS = n, TT = n+1, need = 0, demand = 0;
for (int i = 0; i < n; i++) {
    int d = b[i]-bal[i];
    if (d > 0) f.addEdge(SS, i, d, T(0)), need += d;
    if (d < 0) f.addEdge(i, TT, -d, T(0)), demand -= d;
}
if (need != demand) return {0, T(0)};
auto [got, extra] = f.flow(SS, TT, need);
if (got != need) return {0, T(0)};

ans.resize(e.size());
for (int i = 0; i < (int)e.size(); i++) {
    int x = f.getEdge(id[i]).flow;
    ans[i] = e[i].cost >= T(0) ? e[i].l+x : e[i].r-x;
}
return {1, cost+extra};
}

Result flow(int s, int t, int need) {
    vector<int> b(n);
    b[s] = need, b[t] = -need;
    return minCost(b);
}
};

```

二分图最大权匹配

用途： 给定完全二分图的权值矩阵 $a_{i,j}$ ，求覆盖左侧全部点的最大权匹配；当两侧点数相等时，即为二分图最大权完美匹配。

条件： 左侧点数 n 不超过右侧点数 m ，且 a 为 $n \times m$ 矩阵。若要求允许任意点不匹配，可在右侧补充 n 个权值全为 0 的虚点后调用本板子；若边权可能为负，补点不可省略。

类型： T 表示边权与答案的类型。整数边权通常使用 $i64$ ；若答案、 $-a_{i,j}$ 、顶标或差值可能超出 $i64$ ，应使用 $i128$ 。

操作/结构	含义	复杂度	备注
<code>solve(a)</code>	传入权值矩阵 a ，返回覆盖左侧全部点的最大权值	$O(n^2m)$	$n==m$ 时为最大权完美匹配；结果中的 <code>match[i]</code> 是左点 i 匹配的右点
<code>match</code>	维护上一次求解得到的匹配方案	-	下标从 0 开始；调用 <code>solve</code> 前内容无效

```
template<class T>
struct KM {
    vector<int> match;

    T solve(const vector<vector<T>> &a) {
        int n = a.size();
        if (!n) return match.clear(), T(0);
        int m = a[0].size();
        T inf = numeric_limits<T>::max()/4;
        vector<T> u(n+1), v(m+1);
        vector<int> p(m+1), way(m+1);

        for (int i = 1; i <= n; i++) {
            p[0] = i;
            int j0 = 0;
            vector<T> minv(m+1, inf);
            vector<char> used(m+1);
            do {
                used[j0] = 1;
                int i0 = p[j0], j1 = 0;
                T delta = inf;
                for (int j = 1; j <= m; j++) {
                    if (used[j]) continue;
                    T cur = -a[i0-1][j-1]-u[i0]-v[j];
                    if (cur < minv[j]) minv[j] = cur, way[j] = j0;
                    if (minv[j] < delta) delta = minv[j], j1 = j;
                }
                for (int j = 0; j <= m; j++) {
                    if (used[j]) u[p[j]] += delta, v[j] -= delta;
                    else minv[j] -= delta;
                }
                j0 = j1;
            } while (p[j0]);
            do {
                int j1 = way[j0];
                p[j0] = p[j1], j0 = j1;
            } while (j0);
        }
    }
};
```

```
    }

    match.assign(n, -1);
    for (int j = 1; j <= m; j++) {
        if (p[j]) match[p[j]-1] = j-1;
    }
    return v[0];
}

};
```

图匹配与二分图常用性质

用途： 记录一般图、二分图匹配，以及匹配和覆盖、独立集、路径覆盖之间的常用关系。

复杂度： 本页只列性质，不对应独立代码。

一般图

增广路

对当前匹配 M ，若存在一条首尾都是未匹配点、边在匹配边与非匹配边之间交替出现的路径，则称其为增广路。把路径上的边取反后，匹配边数增加 1。

Berge 引理： 匹配 M 是最大匹配，当且仅当不存在相对于 M 的增广路。这里“最大”指边数最多，不是“极大”（不能再加入一条边）。

对称差

对两个匹配 M, M' ，记它们的对称差为 $M \triangle M'$ ，即只保留恰好属于其中一个匹配的边。其每个连通分量都是交替环或交替路。若 $|M'| > |M|$ ，其中必有一条相对于 M 的增广路。

Tutte 定理

一般图 G 有完美匹配，当且仅当对任意点集 $S \subseteq V(G)$ ，删去 S 后的奇数连通分量数不超过 $|S|$ ：

$$\text{odd}(G - S) \leq |S|.$$

其中 $\text{odd}(G - S)$ 表示 $G - S$ 的奇数点数连通分量个数。

最小边覆盖

若图 G 没有孤立点，设 $n = |V(G)|$ ， $\nu(G)$ 为最大匹配的边数， $\rho(G)$ 为覆盖所有顶点的最少边数，则

$$\rho(G) = n - \nu(G).$$

构造方法：先取一个最大匹配，再为每个未匹配点补一条关联边。若图中有孤立点，则不存在覆盖全部顶点的边覆盖。

二分图

设 $G = (L \cup R, E)$ 是二分图， L, R 是两侧点集。

Hall 婚配定理

对 $X \subseteq L$ ，记 $N(X)$ 为 X 在右侧的邻集。存在一个匹配覆盖左侧全部点，当且仅当对每个 $X \subseteq L$ 都有

$$|N(X)| \geq |X|.$$

当 $|L| = |R|$ 时，这等价于存在完美匹配；若两侧大小不同，只能要求点数较少的一侧被完全覆盖。

Kőnig 定理

记 $\nu(G)$ 为二分图最大匹配的边数， $\tau(G)$ 为最小点覆盖的点数，则

$$\tau(G) = \nu(G).$$

给定一组最大匹配，从左侧未匹配点出发，沿“非匹配边、匹配边”交替搜索。若搜索得到的可达点集为 $Z_L \subseteq L$ 、 $Z_R \subseteq R$ ，则

$$(L \setminus Z_L) \cup Z_R$$

是一个最小点覆盖。

最大独立集

记 $\alpha(G)$ 为最大独立集的点数， $\tau(G)$ 为最小点覆盖的点数。任意图都有

$$\alpha(G) + \tau(G) = |V(G)|.$$

因此二分图满足

$$\alpha(G) = |V(G)| - \nu(G).$$

最大独立集就是最小点覆盖的补集。这个等式不能直接推广到一般图。

DAG 最小路径覆盖

对有向无环图 G ，把每个顶点 v 拆成左点 v_L 和右点 v_R ，把原图每条边 $u \rightarrow v$ 变成二分图边 $u_L v_R$ 。若拆出的二分图最大匹配大小为 k ，原图的最小点不相交路径覆盖数为

$$|V(G)| - k.$$

单个顶点可以作为一条路径。若题目允许直接连接可达点，应先对 DAG 求传递闭包，再建立二分图。

Dilworth 定理

偏序集的最小链覆盖数等于最大反链大小。把每个元素拆成左右两个点，对所有严格小于关系连边；若元素个数为 n ，二分图最大匹配大小为 k ，则最小链覆盖数为

$$n - k.$$

多项式

顺序	模板	用途	主要复杂度
01	连续点拉格朗日插值	从 $f(0..n)$ 求 $f(x)$	$O(n)$
02	FFT	复数域卷积	$O(n \log n)$
03	NTT 基础版	NTT 友好模数卷积	$O(n \log n)$
04	NTT 卡常版	998244353 高性能卷积	$O(n \log n)$
05	MTT	任意模卷积	$O(n \log n)$
06	二元 NTT	二元多项式点值变换	$O(nm(\log n + \log m))$
07	二元多项式卷积	二元卷积压成一元卷积	$O(L \log L)$
08	任意因子长度 DFT	长度整除 $\text{mod}-1$ 的循环卷积	$O(n \log n)$
09	FWT	OR、AND、XOR 卷积	$O(n \log n)$
10	多项式乘法逆	$f^{-1} \bmod x^n$	$O(n \log n)$
11	稀疏多项式除法	稀疏除数的形式除法	$O(nd)$
12	多项式加法与减法	模意义下逐项加减	$O(\max(n, m))$
13	多项式整除与取模	商与余式	$O(n \log n)$
14	多项式对数与指数	形式幂级数 $\ln$ 、 $\exp$	$O(n \log n)$
15	多项式平方根	$g^2 = f \bmod x^n$	$O(n \log n)$
16	多项式快速幂	大整数指数的 f^k	$O(n \log n)$
17	拉格朗日反演	隐式生成函数系数提取	公式速查
18	单位根反演	生成函数同余类筛选	公式速查
19	分式第 N 项	$[x^N]f/g$	$O(n \log n \log N)$
20	多点求值	同时计算多个 $f(x_i)$	$O(n \log^2 n)$ 量级
21	点值平移	$f(0..n) \rightarrow f(m..m+n)$	$O(n \log n)$
22	快速阶乘	大 n 的 $n! \bmod p$	$\tilde{O}(\sqrt{n})$
23	Power Projection 基础版	批量求 $[x^N]fg^i$	准线性量级
24	Power Projection 卡常版	高性能批量幂投影	准线性量级
25	最短递推式	从序列前缀求最短线性递推	$O(nL)$ 或 $O(n \log^2 n)$
26	常数系数齐次线性递推	将递推转成分式求第 N 项	$O(k \log k \log N)$

连续点拉格朗日插值

用途： 已知次数不超过 n 的多项式在 $0, 1, \dots, n$ 处的值，计算它在 x 处的值。

复杂度： 若 inv 、 ifac 为 $O(1)$ 查询，则单次插值 $O(n)$ ，额外空间 $O(1)$ 。

依赖： 阶乘逆元、模逆元；要求 mod 为质数且 $0 \leq n < \text{mod}$ ， x 为 i64 范围内的整数。

```
int Lagrange(vector<int> &f, int x) {
    int n = f.size()-1;
    x = (x%mod+mod)%mod;
    if (x <= n) return f[x];
    int res = 1;
    for (int j = x-n; j <= x; j++) res = res*j%mod;
    int ans = 0;
    for (int i = n, flag = 1; i >= 0; i--, flag = -flag) {
        ans += f[i]*res%mod*inv(x-i)%mod*ifac(i)%mod*ifac(n-i)%mod*flag;
        ans = (ans%mod+mod)%mod;
    }
    return ans;
}
```

FFT

用途： 在复数域做离散傅里叶变换，进而实现整数或实数卷积。

复杂度： 时间 $O(n \log n)$ ，额外空间 $O(n)$ 。

使用条件： n 应为二的幂，`A.size()` 至少为 n ；`op=1` 为正变换，`op=-1` 为逆变换。规模或系数较大时要评估浮点误差。

当 $n \leq 10^5$ 时，可以按系数范围考虑是否把 `double` 换为 `long double`。

```
const double PI = acos(-1);
using cd = complex<double>;
const cd I_(0, 1);
void fft(vector<cd> &A, int n, int op) {
    static vector<int> r;
    static vector<cd> wn;
    static int pre = 0;
    if ((int)r.size() != n) {
        r.resize(n);
        for (int i = 0; i < n; i++) r[i] = (r[i/2]/2) | ((i%2)?n/2:0);
        wn.resize(n+1);
        for (int i = 0; i <= n; i++) wn[i] = exp(I_ * (2 * PI * i / n * op));
        pre = op;
    } else if (pre != op) {
        reverse(wn.begin(), wn.end());
        pre = op;
    }
    for (int i = 0; i < n; i++) if (i < r[i]) swap(A[i], A[r[i]]);

    for (int i = 2; i <= n; i <= 1) {
        int tmp = n/i;
        for (int j = 0; j < n; j += i) {
            for (int k = j; k < j+i/2; k++) {
                cd w = wn[(k - j) * tmp];
                cd x = A[k], y = w * A[k+i/2];
                A[k] = x+y;
                A[k+i/2] = x-y;
            }
        }
    }
    if (op == -1) for (auto &x : A) x /= n;
}
```

NTT：基础版

用途：在 NTT 友好模数下精确计算多项式卷积，避免 FFT 的浮点误差。

复杂度：单次变换 $O(n \log n)$ ，卷积 $O((n+m) \log(n+m))$ ，额外空间 $O(n+m)$ 。

使用条件：变换长度必须为二的幂且整除 $\text{mod}-1$ ； G 是原根， G_i 是其逆元。`operator*=` 覆盖左操作数，`operator*` 返回乘积。

```
const int G = 3, Gi = qpow(G, mod-2);

void ntt(vector<int> &A, int n, int op) {
    static vector<int> r, g(1, 1);
    if ((int)r.size() != n) {
        r.resize(n);
        for (int i = 0; i < n; i++) r[i] = (r[i/2]/2) | ((i%2)?n/2:0);
    }
    for (int i = 0; i < n; i++) if (i < r[i]) swap(A[i], A[r[i]]);
    if ((int)g.size() < n/2+1) g.resize(n/2+1);
    for (int i = 2; i <= n; i<<=1) {
        g[1] = qpow(op == 1 ? G : Gi, (mod-1)/i);
        for (int j = 2; j < i/2; j++) g[j] = g[j-1]*g[1]%mod;
        for (int j = 0; j < n; j+=i) {
            for (int k = j; k < j+i/2; k++) {
                int x = A[k], y = g[k-j]*A[k+i/2]%mod;
                A[k] = (x+y);
                if (A[k] >= mod) A[k] -= mod;
                A[k+i/2] = (x-y+mod);
                if (A[k+i/2] >= mod) A[k+i/2] -= mod;
            }
        }
    }
    if (op == -1) for (auto &x : A) x = x*inv(n)%mod;
}

void operator*=(vector<int> &A, vector<int> B) {
    int n = A.size(), m = B.size(), cnt = 1;
    if (!n || !m) return A.clear();
    while (cnt < n+m-1) cnt<<=1;
    A.resize(cnt), B.resize(cnt);
    ntt(A, cnt, 1), ntt(B, cnt, 1);
    for (int i = 0; i < cnt; i++) {
        (A[i] *= B[i]) %= mod;
    }
    ntt(A, cnt, -1);
    A.resize(n+m-1);
}

vector<int> operator*(vector<int> A, vector<int> B) {A *= B; return A;}
```

NTT：卡常版

用途： 针对模数 `998244353` 的高性能多项式卷积，使用 radix-4 蝶形、预处理旋转因子、小规模朴素乘法和长度削减优化。

复杂度： 单次变换 $O(n \log n)$ ，卷积 $O((n + m) \log(n + m))$ ，额外空间 $O(n + m)$ 。

使用条件： 常量 `31`、`23` 与根表均绑定 `998244353`；输入系数应先规范到 $[0, \text{mod})$ 。`operator*=` 覆盖左操作数，`operator*` 返回乘积。

```
void ntt(vector<int> &A, int n, int op) {
    static array<int, 30> root, iroot, rate2, irate2, rate3, irate3;
    static bool ok = 0;
    if (!ok) {
        ok = 1;
        root[23] = 31, iroot[23] = inv(root[23]);
        for (int i = 22; i >= 0; i--) root[i] = root[i+1]*root[i+1]%mod, iroot[i] = iroot[i+1]*iroot[i+1]%mod;

        int prod = 1, iprod = 1;
        for (int i = 0; i <= 21; i++) {
            rate2[i] = root[i+2]*prod%mod, irate2[i] = iroot[i+2]*iprod%mod;
            prod = prod*iroot[i+2]%mod, iprod = iprod*root[i+2]%mod;
        }

        prod = 1, iprod = 1;
        for (int i = 0; i <= 20; i++) {
            rate3[i] = root[i+3]*prod%mod, irate3[i] = iroot[i+3]*iprod%mod;
            prod = prod*iroot[i+3]%mod, iprod = iprod*root[i+3]%mod;
        }
    }

    int h = __lg(n);
    if (op == 1) {
        int len = 0;
        while (len < h) {
            if (h-len == 1) {
                int p = 1<<(h-len-1), rot = 1;
                for (int s = 0; s < (1<<len); s++) {
                    int offset = s<<(h-len);
                    for (int i = 0; i < p; i++) {
                        int l = A[i+offset], r = A[i+offset+p]*rot%mod;
                        A[i+offset] = l+r < mod ? l+r : l+r-mod;
                        A[i+offset+p] = l-r >= 0 ? l-r : l-r+mod;
                    }
                    rot = rot*rate2[__lg((~s)&(~~s))]%mod;
                }
                len++;
            } else {
                int p = 1<<(h-len-2), rot = 1, imag = root[2];
                for (int s = 0; s < (1<<len); s++) {
                    int rot2 = rot*rot%mod, rot3 = rot2*rot%mod;
                    int offset = s<<(h-len);
```

```

        for (int i = 0; i < p; i++) {
            int mod2 = mod*mod;
            int a0 = A[i+offset];
            int a1 = A[i+offset+p]*rot%mod;
            int a2 = A[i+offset+2*p]*rot2%mod;
            int a3 = A[i+offset+3*p]*rot3%mod;
            int a1na3imag = (a1+mod2-a3)%mod*iimag%mod, na2 = mod2-a2;
            A[i+offset] = (a0+a2+a1+a3)%mod;
            A[i+offset+p] = (a0+a2+(2*mod2-(a1+a3)))%mod;
            A[i+offset+2*p] = (a0+na2+a1na3imag)%mod;
            A[i+offset+3*p] = (a0+na2+(mod2-a1na3imag))%mod;
        }
        rot = rot*rate3[__lg((~s)&(~~s))]%mod;
    }
    len += 2;
}
}
} else {
    int invn = inv(n);
    for (auto &x : A) x = x*invn%mod;

    int len = h;
    while (len) {
        if (len == 1) {
            int p = 1<<(h-len), irot = 1;
            for (int s = 0; s < (1<<(len-1)); s++) {
                int offset = s<<(h-len+1);
                for (int i = 0; i < p; i++) {
                    int l = A[i+offset], r = A[i+offset+p];
                    A[i+offset] = l+r < mod ? l+r : l+r-mod;
                    A[i+offset+p] = (l-r+mod)*irot%mod;
                }
                irot = irot*irate2[__lg((~s)&(~~s))]%mod;
            }
            len--;
        } else {
            int p = 1<<(h-len), irot = 1, iimag = iroot[2];
            for (int s = 0; s < (1<<(len-2)); s++) {
                int irot2 = irot*irot%mod, irot3 = irot2*irot%mod;
                int offset = s<<(h-len+2);
                for (int i = 0; i < p; i++) {
                    int a0 = A[i+offset], a1 = A[i+offset+p], a2 = A[i+offset+2*p], a3 = A[i+offset+3*p];
                    int x = (mod+a2-a3)%mod*iimag%mod;
                    A[i+offset] = (a0+a1+a2+a3)%mod;
                    A[i+offset+p] = (a0+mod-a1+x)%mod*irot%mod;
                    A[i+offset+2*p] = (a0+a1+2*mod-a2-a3)%mod*irot2%mod;
                    A[i+offset+3*p] = (a0+2*mod-a1-x)%mod*irot3%mod;
                }
                irot = irot*irate3[__lg((~s)&(~~s))]%mod;
            }
            len -= 2;
        }
    }
}
}

```

```

    }
}

void operator*=(vector<int> &A, vector<int> B) {
    int n = A.size(), m = B.size();
    if (!n || !m) {
        A.clear();
        return;
    }
    if (min(n, m) <= 64) {
        vector<int> C(n+m-1);
        for (int i = 0; i < n; i++) for (int j = 0; j < m; j++) C[i+j] = (C[i+j]+A[i]*B[j])%mod;
        A.swap(C);
        return;
    }

    int cnt = 1;
    while (cnt < n+m-1) cnt <= 1;

    if (n+m-3 <= cnt/2) {
        int al = A.back(), bl = B.back();
        A.pop_back(), B.pop_back();
        vector<int> a = A, b = B;
        A *= B;
        A.resize(n+m-1), A[n+m-2] = al*bl%mod;
        for (int i = 0; i < n-1; i++) A[i+m-1] = (A[i+m-1]+a[i]*bl)%mod;
        for (int i = 0; i < m-1; i++) A[i+n-1] = (A[i+n-1]+b[i]*al)%mod;
        return;
    }

    A.resize(cnt), B.resize(cnt);
    bool same = A == B;
    ntt(A, cnt, 1);
    if (same) B = A;
    else ntt(B, cnt, 1);

    for (int i = 0; i < cnt; i++) A[i] = A[i]*B[i]%mod;
    ntt(A, cnt, -1);
    A.resize(n+m-1);
}

vector<int> operator*(vector<int> A, vector<int> B) {A *= B; return A;}

```

MTT 任意模卷积

用途： 把系数拆成高低位后用复数 FFT 做卷积，从而支持任意运行时模数。

复杂度： 时间 $O((n+m)\log(n+m))$ ，额外空间 $O(n+m)$ 。

使用条件： 依赖 FFT；输入系数应先规范到 $[0, \text{mod})$ 。结果正确性还受系数规模、卷积长度与浮点精度影响。

当 $n \leq 10^5$ 时，可以按误差范围考虑是否把 `double` 换为 `long double`。

```
int mod;
const int M = 1ll << 15;
void operator*=(vector<int> &AA, vector<int> BB) {
    int n = AA.size(), m = BB.size(), cnt = 1;
    while (cnt < n+m-1) cnt *= 2;
    AA.resize(cnt), BB.resize(cnt);

    vector<cd> A(cnt), B(cnt), C(cnt), D(cnt);
    for (int i = 0; i < cnt; i++) {
        A[i] = cd(AA[i]/M, AA[i]%M), C[i] = cd(BB[i]/M, BB[i]%M);
    }
    fft(A, cnt, 1), fft(C, cnt, 1);

    for (int i = 1; i < cnt; i++) B[i] = conj(A[cnt-i]);
    B[0] = conj(A[0]);
    for (int i = 1; i < cnt; i++) D[i] = conj(C[cnt-i]);
    D[0] = conj(C[0]);
    for (int i = 0; i < cnt; i++) {
        cd aa = (A[i]+B[i])*cd(0.5, 0), bb = (A[i]-B[i])*cd(0, -0.5);
        cd cc = (C[i]+D[i])*cd(0.5, 0), dd = (C[i]-D[i])*cd(0, -0.5);
        A[i] = aa*cc+cd(0, 1)*(aa*dd+bb*cc), B[i] = bb*dd;
    }

    fft(A, cnt, -1), fft(B, cnt, -1);
    AA.resize(n+m-1);
    for (int i = 0; i < n+m-1; i++) {
        int aa = (int)(A[i].real()+0.5)%mod;
        int bb = (int)(A[i].imag()+0.5)%mod;
        int cc = (int)(B[i].real()+0.5)%mod;
        AA[i] = ((aa*M+bb*M+cc)%mod+mod)%mod;
    }
}
```

二元 NTT

用途： 对系数矩阵表示的二元多项式 $f(x, y)$ 先按行、再按列做 NTT。

复杂度： 时间 $O(nm(\log n + \log m))$ ，额外空间 $O(n)$ ；矩阵本身占用 $O(nm)$ 。

使用条件： 行列数 n 、 m 都必须满足所用 NTT 的长度条件，矩阵必须是完整的 $n \times m$ 矩形。

```
void ntt2D(vector<vector<int>> &A, int n, int m, int op) {
    for (int i = 0; i < n; i++) ntt(A[i], m, op);
    vector<int> t1(n);
    for (int j = 0; j < m; j++) {
        for (int i = 0; i < n; i++) t1[i] = A[i][j];
        ntt(t1, n, op);
        for (int i = 0; i < n; i++) A[i][j] = t1[i];
    }
}
```

二元多项式卷积

用途： 通过进制压缩把 $f(x, y)$ 映射成一元多项式，再复用一元卷积；步长 $W=m_1+m_2-1$ 可避免不同 x 次数之间发生进位混叠。

复杂度： 若一元卷积长度为 $L = (n_1 + n_2 - 1)(m_1 + m_2 - 1)$ ，时间为 $O(L \log L)$ ，空间为 $O(L)$ 。

使用条件： 两个输入都必须是非空矩形，且依赖一元 `operator*=`。

```
void operator*=(vector<vector<int>> &A, const vector<vector<int>> &B) {
    int n1 = A.size(), m1 = A[0].size();
    int n2 = B.size(), m2 = B[0].size();

    int W = m1 + m2 - 1;
    static vector<int> a, b;
    a.assign(n1 * W, 0), b.assign(n2 * W, 0);

    for (int i = 0; i < n1; i++) {
        for (int j = 0; j < m1; j++) a[i * W + j] = A[i][j];
    }
    for (int i = 0; i < n2; i++) {
        for (int j = 0; j < m2; j++) b[i * W + j] = B[i][j];
    }
    a *= b;

    int N = n1 + n2 - 1;
    int M = m1 + m2 - 1;
    A.resize(N);
    for (int i = 0; i < N; i++) {
        A[i].resize(M);
        for (int j = 0; j < M; j++) A[i][j] = a[i * W + j];
    }
}
```

任意因子长度 DFT

用途： 在 998244353 下处理长度不必为二次幂、但整除 $\text{mod}-1$ 的循环卷积；按因子 2、7、17 递归分解变换长度。

复杂度： 对当前平滑长度为 $O(n \log n)$ ；小规模分支使用 $O(n^2)$ 朴素 DFT。额外空间 $O(n)$ ，递归中会产生临时向量。

使用条件： 必须满足 $n \mid (\text{mod}-1)$ ；G、Gi 和基础 NTT 与当前模数匹配。

```
void dftNaive(vector<int> &A, int op) {
    int n = A.size();
    if (n <= 1) return;

    int wn = qpow(op == 1 ? G : Gi, (mod-1)/n);
    vector<int> B(n);

    for (int i = 0; i < n; i++) {
        int wi = qpow(wn, i), pw = 1;
        for (int j = 0; j < n; j++) {
            B[i] = (B[i] + A[j] * pw) % mod;
            pw = pw * wi % mod;
        }
    }

    if (op == -1) {
        int invn = inv(n);
        for (auto &x : B) x = x * invn % mod;
    }
    A.swap(B);
}

void dft(vector<int> &A, int op = 1) {
    int n = A.size();
    if (!n) return;
    assert((mod-1)%n == 0), assert(op == 1 || op == -1);

    auto dfs = [&](auto &&self, vector<int> &A, int op) -> void {
        int n = A.size();
        if (n <= 1) return;
        if ((n&-n) == n) {
            ntt(A, n, op);
            return;
        }
        if (n <= 64) {
            dftNaive(A, op);
            return;
        }

        int r;
        if (n%17 == 0) r = 17;
        else if (n%7 == 0) r = 7;
```

```

else {
    ntt(A, n, op);
    return;
}

int m = n/r;
vector<int> B(n), t(max(r, m));

for (int x = 0; x < m; x++) {
    t.resize(r);
    for (int y = 0; y < r; y++) t[y] = A[x*m+y];
    self(self, t, op);
    for (int s = 0; s < r; s++) B[s*m+x] = t[s];
}

int wn = qpow(op == 1 ? G : Gi, (mod-1)/n);
for (int s = 0; s < r; s++) {
    int step = qpow(wn, s), pw = 1;
    t.resize(m);

    for (int x = 0; x < m; x++) {
        t[x] = B[s*m+x]*pw%mod;
        pw = pw*step%mod;
    }

    self(self, t, op);

    for (int x = 0; x < m; x++) A[s+r*x] = t[x];
}
};

dfs(dfs, A, op);
}

```

FWT: OR、AND、XOR 卷积

用途： 计算下标按位 OR、AND 或 XOR 合并的卷积。

复杂度： 每种变换与卷积均为 $O(n \log n)$ ，空间 $O(n)$ ； n 是补齐后的二次幂长度。

使用条件： 两个卷积输入必须落在同一个下标全集中，重载会将二者补到相同长度；XOR 逆变换要求 2 在模数下可逆。

OR:

$$\text{FWT}(a)_i = \sum_{i|j=i} a_j.$$

AND:

$$\text{FWT}(a)_i = \sum_{i \& j=i} a_j.$$

XOR:

$$\text{FWT}(a)_i = \sum_{i \circ j=0} a_j - \sum_{i \circ j=1} a_j, \quad a \circ b = \text{popcount}(a \& b) \& 1.$$

```
void Or(vector<int> &a, int type) {
    int m = a.size(), n = 1;
    while (n < m) n *= 2;
    a.resize(n);
    for (int x = 2; x <= n; x <= 1) {
        int k = x >> 1;
        for (int i = 0; i < n; i += x) {
            for (int j = 0; j < k; j++) {
                (a[i+j+k] += a[i+j]*type+mod) %= mod;
            }
        }
    }
}

vector<int> operator|(vector<int> a, vector<int> b) {
    a.resize(max(a.size(), b.size()), b.resize(a.size()));
    Or(a, 1), Or(b, 1);
    int n = a.size();
    for (int i = 0; i < n; i++) (a[i] += b[i]) %= mod;
    Or(a, -1);
    return a;
}

void And(vector<int> &a, int type) {
    int m = a.size(), n = 1;
```

```

while (n < m) n *= 2;
a.resize(n);
for (int x = 2; x <= n; x <= 1) {
    int k = x >> 1;
    for (int i = 0; i < n; i += x) {
        for (int j = 0; j < k; j++) {
            (a[i+j] += a[i+j+k]*type+mod) %= mod;
        }
    }
}

vector<int> operator&(vector<int> a, vector<int> b) {
    a.resize(max(a.size(), b.size())), b.resize(a.size());
    And(a, 1), And(b, 1);
    int n = a.size();
    for (int i = 0; i < n; i++) (a[i] *= b[i]) %= mod;
    And(a, -1);
    return a;
}

void Xor(vector<int> &a, int type) {
    int m = a.size(), n = 1;
    while (n < m) n *= 2;
    a.resize(n);
    for (int x = 2; x <= n; x <= 1) {
        int k = x >> 1;
        for (int i = 0; i < n; i += x) {
            for (int j = 0; j < k; j++) {
                (a[i+j] += a[i+j+k]) %= mod, (a[i+j+k] = a[i+j]-a[i+j+k]*2+2*mod) %= mod;
                (a[i+j] *= type) %= mod, (a[i+j+k] *= type) %= mod;
            }
        }
    }
}

vector<int> operator^(vector<int> a, vector<int> b) {
    a.resize(max(a.size(), b.size())), b.resize(a.size());
    Xor(a, 1), Xor(b, 1);
    int n = a.size();
    for (int i = 0; i < n; i++) (a[i] *= b[i]) %= mod;
    Xor(a, inv(2));
    return a;
}

```

多项式乘法逆

用途： 求形式幂级数 $g(x)$, 使 $f(x)g(x) \equiv 1 \pmod{x^n}$ 。

复杂度： 使用 NTT 卷积时为 $O(n \log n)$, 递归与临时向量占用 $O(n)$ 空间。

使用条件： 必须有 $f(0) \neq 0$; 代码对 998244353 取模, 任意模数时需替换卷积实现。

可以先记录并除去前导的 x^k , 使常数项非零, 再处理带平移的逆。

```
vector<int> inv(vector<int> f) {
    if (f.size() == 1) return {qpow(f[0], mod-2)};
    int n = f.size();
    int m = (n+1)/2;
    auto h = f;
    h.resize(m);
    h = inv(h);

    int cnt = 1;
    while (cnt <= n+m+m) cnt<<=1;
    f.resize(cnt), h.resize(cnt);
    ntt(f, cnt, 1), ntt(h, cnt, 1);
    for (int i = 0; i < cnt; i++) {
        f[i] = h[i]*(2-f[i]*h[i]%mod+mod)%mod;
    }
    ntt(f, cnt, -1);
    f.resize(n);
    return f;
}
```

稀疏多项式除法

用途： 计算 $g(x)/f(x)$ 在指定次数内的系数，其中除数 f 只有少量非零项。

复杂度： 若结果最高次数为 n 、 f 的非零项数为 d ，时间 $O(nd)$ ，额外空间 $O(d)$ 。

使用条件： f 非空且 $f(0)$ 可逆；非零项按次数递增收集。

```
vector<int> div(vector<int> g, vector<int> f) { // g*inv(f) f:稀疏
    int n = g.size()-1;
    int inv0 = inv(f[0]);
    g[0] = g[0]*inv0%mod;
    vector<pii> F;
    for (int j = 1; j < (int)f.size(); j++) if (f[j]) F.push_back({j, f[j]});
    for (int i = 1; i <= n; i++) {
        for (auto [j, x] : F) {
            if (j > i) break;
            g[i] = (g[i]+mod-x*g[i-j]%mod) % mod;
        }
        g[i] = g[i]*inv0%mod;
    }
    return g;
}
```

多项式加法与减法

用途： 对两个多项式逐项做模意义下的加法或减法。

复杂度： 时间与空间均为 $O(\max(n, m))$ 。

使用条件： 输入系数均在 $[0, \text{mod})$ 内；返回结果保留较长输入的长度，不主动删除末尾零系数。

```
vector<int> operator+(vector<int> A, vector<int> B) {
    if (A.size() < B.size()) swap(A, B);
    for (int i = 0; i < (int)B.size(); i++) {
        A[i] += B[i];
        if (A[i] >= mod) A[i] -= mod;
    }
    return A;
}

vector<int> operator-(vector<int> A, vector<int> B) {
    A.resize(max(A.size(), B.size()));
    for (int i = 0; i < (int)B.size(); i++) {
        A[i] -= B[i];
        if (A[i] < 0) A[i] += mod;
    }
    return A;
}
```

多项式整除与取模

用途： 计算 $f(x) = q(x)g(x) + r(x)$ 中的商 q 和余式 r 。

复杂度： 使用快速卷积和多项式求逆时为 $O(n \log n)$ ，额外空间 $O(n)$ 。

使用条件： 除数最高次项必须非零且可逆；向量末尾应去除无意义的高次零系数。

```
vector<int> operator/(vector<int> f, vector<int> g) {
    int n = f.size(), m = g.size();
    if (n < m) return {0};
    reverse(f.begin(), f.end()), reverse(g.begin(), g.end());
    f.resize(n-m+1), g.resize(n-m+1);
    f *= inv(g);
    f.resize(n-m+1);
    reverse(f.begin(), f.end());
    return f;
}

vector<int> operator%(vector<int> f, vector<int> g) {
    f = f-f/g*g;
    f.resize(g.size()-1);
    return f;
}
```

多项式对数与指数

用途：形式幂级数运算： $\ln f = \int f'/f$ ，以及求满足 $\ln g = f$ 的 $g = \exp f$ 。

复杂度：使用快速卷积时，求导和积分为 $O(n)$ ，对数与指数为 $O(n \log n)$ ；空间 $O(n)$ 。

使用条件： $\ln$ 要求 $f(0) = 1$ ； $\exp$ 要求 $f(0) = 0$ ；积分所需的 $1, \dots, n$ 均应在模数下可逆。

```
vector<int> D(vector<int> f) { //求导
    int n = f.size();
    for (int i = 0; i < n-1; i++) {
        f[i] = (i+1)*f[i+1]%mod;
    }
    f.resize(n-1);
    return f;
}

vector<int> I(vector<int> f) {
    int n = f.size();
    f.resize(n+1);
    for (int i = n; i >= 1; i--) {
        f[i] = f[i-1]*inv(i)%mod;
    }
    f[0] = 0;
    return f;
}

vector<int> ln(vector<int> f) {
    auto invF = inv(f);
    auto res = D(f);
    res *= invF;
    res.resize(f.size());
    res = I(res);
    res.resize(f.size());
    return res;
}

vector<int> exp(vector<int> f) {
    if (f.size() == 1) return {1};
    int n = f.size();
    int m = (n+1)/2;
    auto h = f;
    h.resize(m);
    h = exp(h);
    h.resize(n);
    auto g = ln(h);
    for (int i = 0; i < n; i++) {
        f[i] = f[i]-g[i];
        if (f[i] < 0) f[i] += mod;
    }
    f[0] += 1;
    f *= h;
}
```

```
f.resize(n);  
return f;  
}
```

多项式平方根

用途： 求形式幂级数 $g(x)$, 使 $g(x)^2 \equiv f(x) \pmod{x^n}$ 。

复杂度： 使用快速卷积时为 $O(n \log n)$, 空间 $O(n)$ 。

使用条件： 当前 Newton 迭代要求选出的常数项平方根非零, 从而可以求逆; `cipolla(f[0])` 必须有解。

```
vector<int> sqrt(vector<int> f) {
    if (f.size() == 1) return {cipolla(f[0])};
    int n = f.size();
    int m = (n+1)/2;
    auto h = f;
    h.resize(m);
    h = sqrt(h);
    h.resize(n);
    auto g = inv(h);
    f *= g;
    f.resize(n);
    for (int i = 0; i < n; i++) {
        f[i] = (f[i]+h[i])*inv(2)%mod;
    }
    return f;
}
```

多项式快速幂

用途： 对十进制字符串表示的非负整数 k ，计算 $f(x)^k \bmod x^n$ ；通过提取最低非零项、对数和指数完成。

复杂度： 稠密版本使用快速卷积时为 $O(n \log n)$ ；稀疏替代为 $O(ns)$ ，其中 s 是原多项式非零项数。空间 $O(n)$ 。

使用条件： 归一化后的常数项必须可逆，并满足多项式 `ln/exp` 的前置条件。

```
vector<int> qpow(vector<int> f, const string &k) {
    int km = 0, km2 = 0;
    for (auto c : k) km = (km*10+c-'0')%mod, km2 = (km2*10+c-'0')%(mod-1);
    int n = f.size();
    int t = -1;
    for (int i = 0; i < n; i++) {
        if (!f[i]) continue;
        t = i;
        for (int j = t; j < n; j++) f[j-t] = f[j];
        for (int j = n-t; j < n; j++) f[j] = 0;
        break;
    }
    if (t == -1 || (t != 0 && (int)k.size() > 8)) {
        fill(f.begin(), f.end(), 0);
        if (k == "0") f[0] = 1;
        return f;
    }
    int inv0 = qpow(f[0], mod-2), pow0 = qpow(f[0], km2);
    for (auto &x : f) x = x*inv0%mod;

    f = ln(f);
    for (auto &x : f) x = x*km%mod;
    f = exp(f);

    for (auto &x : f) x = x*pow0%mod;
    t*=km;
    for (int i = n-1; i-t >= 0; i--) f[i] = f[i-t];
    for (int i = 0; i < n && i < t; i++) f[i] = 0;
    return f;
}
```

对于稀疏多项式，可以用下列 $O(ns)$ 递推替换对数、指数部分：

```
vector<pii> nz;
for (int i = 0; i < n; i++) if (f[i]) nz.push_back({i, f[i]});
fill(f.begin()+1, f.end(), 0);
for (int i = 1; i < n; i++) {
    for (auto [j, fj] : nz) {
        if (j > i) break;
        (f[i] += fj*f[i-j]%mod * (((km+1)*j-i+mod)%mod)) %= mod;
    }
    (f[i] *= inv(i)) %= mod;
}
```

拉格朗日反演

用途： 从隐式方程 $T(x) = x\Phi(T(x))$ 直接提取 $H(T(x))$ 的系数，无需先展开 $T(x)$ 。

复杂度： 本页是恒等式速查；实际复杂度取决于变形后 $\Phi(t)^n$ 的系数计算。

使用条件： $\Phi(0) \neq 0$ ，取唯一满足 $T(0) = 0$ 的形式幂级数解。式中出现的正整数均须可除；模质数 p 时，相应分母须在模 p 下可逆。

对 $n \geq 1$ ，拉格朗日-Bürmann 公式为

$$[x^n]H(T(x)) = \frac{1}{n}[t^{n-1}]H'(t)\Phi(t)^n, \quad [x^0]H(T(x)) = H(0).$$

常用变形

1. 令 $H(t) = t^k$ 。对 $n \geq k \geq 1$,

$$[x^n]T(x)^k = \frac{k}{n}[t^{n-k}]\Phi(t)^n.$$

2. 由 $\Phi(T(x)) = \frac{T(x)}{x}$ ，对 $n \geq 0, k \geq 1$,

$$[x^n]\Phi(T(x))^k = \frac{k}{n+k}[t^n]\Phi(t)^{n+k}.$$

3. 消去导数。对 $n \geq 0$,

$$[x^n]\frac{H(T(x))}{1-x\Phi'(T(x))} = [t^n]H(t)\Phi(t)^n, \quad x\Phi'(T) = \frac{T\Phi'(T)}{\Phi(T)}.$$

4. 若 $F(t) = \frac{t}{\Phi(t)}$ ，则 $T = F^{(-1)}$ ，因此对 $n \geq 1$,

$$[x^n]H(F^{(-1)}(x)) = \frac{1}{n}[t^{n-1}]H'(t)\left(\frac{t}{F(t)}\right)^n.$$

5. 若方程写成 $T(x) = a + x\Phi(T(x))$ ，令 $U(x) = T(x) - a$ 后可得

$$[x^n]H(T(x)) = \frac{1}{n}[t^{n-1}]H'(a+t)\Phi(a+t)^n \quad (n \geq 1).$$

例如 $T = x(1+T)^2$ 时， $[x^n]T$ 即为卡特兰数：

$$[x^n]T = \frac{1}{n}[t^{n-1}](1+t)^{2n} = \frac{1}{n+1}\binom{2n}{n}.$$

单位根反演

用途：从生成函数 $A(x) = \sum_{n \geq 0} a_n x^n$ 中筛出 $n \equiv r \pmod{m}$ 的项。

使用条件：系数域中存在本原 m 次单位根 ω ，且 m 可逆；模质数 p 时通常要求 $m \mid p-1$ ，取 $\omega = g^{(p-1)/m}$ 。

单位根正交关系为

$$\frac{1}{m} \sum_{j=0}^{m-1} \omega^{jt} = [m \mid t].$$

因此对 $0 \leq r < m$,

$$\sum_{q \geq 0} a_{qm+r} x^{qm+r} = \frac{1}{m} \sum_{j=0}^{m-1} \omega^{-rj} A(\omega^j x)$$

若 $\deg A < m$ ，令 $x = 1$ 即得 $a_r = \frac{1}{m} \sum_{j=0}^{m-1} \omega^{-rj} A(\omega^j)$ ；否则右侧会把 $r, m+r, 2m+r, \dots$ 次项相加。

常用变形

1. **压缩下标：**若 $B_r(z) = \sum_{q \geq 0} a_{qm+r} z^q$ 且 $y^m = z$,

$$B_r(z) = \frac{y^{-r}}{m} \sum_{j=0}^{m-1} \omega^{-rj} A(\omega^j y).$$

2. **二项式和：**对 $N \geq 0$,

$$\sum_{\substack{0 \leq k \leq N \\ k \equiv r \pmod{m}}} \binom{N}{k} z^k = \frac{1}{m} \sum_{j=0}^{m-1} \omega^{-rj} (1 + \omega^j z)^N.$$

3. **等比级数：**

$$\frac{x^r}{1-x^m} = \frac{1}{m} \sum_{j=0}^{m-1} \frac{\omega^{-rj}}{1-\omega^j x}, \quad 1-x^m = \prod_{j=0}^{m-1} (1-\omega^j x).$$

4. **奇偶拆分：**取 $m = 2$, $\omega = -1$,

$$A_{\text{even}}(x) = \frac{A(x) + A(-x)}{2}, \quad A_{\text{odd}}(x) = \frac{A(x) - A(-x)}{2}.$$

分式第 N 项

用途： 用多项式带余除法与 Bostan–Mori 算法计算 $[x^N] \frac{f(x)}{g(x)}$ ，对 f, g 的次数关系没有要求，也无需展开前 N 项。

复杂度： 若多项式长度为 n ，使用快速卷积时为 $O(n \log n \log N)$ ，空间 $O(n)$ 。

使用条件： $N \geq 0$ ， $g[0]$ 与最高次项必须可逆， f, g 末尾没有无意义的零系数；依赖外部多项式乘法、整除与取模。

```
int divAt(vector<int> f, vector<int> g, int N) {
    vector<int> p;
    if (f.size() >= g.size()) p = f/g, f = f%g;
    else f.resize(g.size()-1);
    int res = 0;
    if (N < (int)p.size()) res = p[N];
    if (f.empty()) return res;
    for (int i; N; N /= 2) {
        auto h = g;
        for (i = 1; i < (int)h.size(); i+=2) h[i] = (mod-h[i])%mod;
        f *= h, g *= h;
        for (i = 0; i*2 < (int)f.size(); i++) f[i] = f[i*2+N%2];
        f.resize(i);
        for (i = 0; i*2 < (int)g.size(); i++) g[i] = g[i*2];
        g.resize(i);
    }
    (res += f.size() ? f[0]*inv(g[0]) : 0) %= mod;

    return res;
}
```

多点求值

用途： 给定多项式 f 与 m 个点 x_i ，利用乘积树与递归取模同时计算全部 $f(x_i)$ 。

复杂度： 使用快速卷积时为 $O((n+m)\log^2(n+m))$ ，乘积树及递归临时对象占用约 $O((n+m)\log m)$ 的累计存储。

依赖： 多项式乘法、整除与取模；代码是带输入输出的完整 `solve` 片段。

分治

$$X_0 = \{x_0, x_1, \dots, x_{mid}\}, \quad X_1 = \{x_{mid+1}, \dots, x_{m-1}\}.$$

构造

$$g_0(x) = \prod_{i \in X_0} (x - x_i).$$

对任意 $x_i \in X_0$ ，若 $f = g_0 Q_0 + f_0$ ，则 $f(x_i) = f_0(x_i)$ 。递归计算对应余式即可。

```
void solve() {
    int n, m;
    cin >> n >> m;
    vector<int> a(n+1);
    for (int i = 0; i <= n; i++) cin >> a[i];
    vector<int> x(m);
    for (int i = 0; i < m; i++) cin >> x[i];
    struct Node {
        vector<int> f;
    };
    vector<Node> tr(4*m+1);
    function<void(int, int, int)> build = [&](int l, int r, int u) {
        if (l == r) {
            tr[u].f = {(mod-x[l])%mod, 1};
            return;
        }
        int mid = (l+r)/2;
        build(l, mid, 2*u), build(mid+1, r, 2*u+1);
        tr[u].f = tr[2*u].f * tr[2*u+1].f;
    };
    build(0, m-1, 1);
    vector<int> res(m);
    function<void(vector<int>, int, int, int)> dfs = [&](vector<int> f, int l, int r, int u) {
        if (l == r) {
            res[l] = f[0];
            return;
        }
        int mid = (l+r)/2;
        dfs(f%tr[2*u].f, l, mid, 2*u), dfs(f%tr[2*u+1].f, mid+1, r, 2*u+1);
    };
    dfs(a%tr[1].f, 0, m-1, 1);
}
```

```
for (auto x : res) cout << x << endl;  
}
```

点值平移

用途： 已知次数不超过 n 的多项式在 $0, 1, \dots, n$ 的点值，求连续区间 $m, m + 1, \dots, m + n$ 的点值。

复杂度： 使用快速卷积时为 $O(n \log n)$ ，空间 $O(n)$ 。

使用条件： 当前实现要求 $m \geq 0$ 或 $m > n$ ，并要求构造出的所有分母在模数下可逆。

由拉格朗日插值

$$f(m+x) = \sum_{i=0}^n f(i) \prod_{j \neq i} \frac{m+x-j}{i-j}.$$

令

$$u_i = \frac{f(i)}{(-1)^{n-i} i! (n-i)!}, \quad v_i = \frac{1}{m-n+i},$$

则

$$f(m+x) = (u * v)_{n+x} \prod_{i=m+x-n}^{m+x} i.$$

```
vector<int> move(vector<int> f, int m) {
    if (m == 0) return f;
    int n = f.size()-1;
    vector<int> a(n+1), b(2*n+1);
    for (int i = 0; i <= n; i++) {
        a[i] = f[i]*ifac(i)%mod*ifac(n-i)%mod;
        if ((n-i)%2) a[i] = -a[i];
        if (a[i] < 0) a[i] += mod;
    }
    for (int i = 0; i <= 2*n; i++) b[i] = inv(m-n+i);
    a *= b;
    int tmp = 1;
    for (int i = m-n; i <= m; i++) tmp = tmp*i%mod;
    for (int i = 0; i <= n; i++) {
        f[i] = a[i+n]*tmp%mod;
        tmp = tmp*(m+i+1)%mod*b[i]%mod; // b[i] = inv(m-n+i)
    }
    return f;
}
```

快速阶乘

用途： 用多项式点值平移与分块乘积计算 $n! \bmod mod$ ，适合单次 n 很大而 $O(n)$ 连乘过慢的场景。

复杂度： 设 $B \approx \sqrt{n}$ ，使用快速卷积时约为 $O(M(B) \log B + \sqrt{n})$ ，可简记为 $\tilde{O}(\sqrt{n})$ ；空间 $O(\sqrt{n})$ 。

使用条件： 依赖点值平移、快速卷积和模逆元；通常要求 mod 为质数且 $0 \leq n < mod$ 。

```
int cal(int n) {
    int B = sqrtl(n)+1;
    vector<int> f(1, 1);
    int now = 0;
    auto add = [&]() {
        for (int i = 0; i <= now; i++) {
            f[i] = f[i]*(now+1+i*B)%mod;
        }
        now++;
        f.push_back(1);
        for (int i = 1; i <= now; i++) {
            f[now] = f[now]*(i+now*B)%mod;
        }
    };
    auto mul = [&]() {
        if (!now) return;
        auto g = f;
        auto h = move(g, now+1);
        for (auto x : h) g.push_back(x);
        h = move(g, now*inv(B)%mod);
        f.resize(now*2+1);

        for (int i = 0; i <= now*2; i++) {
            f[i] = h[i]*g[i]%mod;
        }
        now *= 2;
    };

    for (int i = __lg(B); i >= 0; i--) {
        mul();
        if ((B)>>i&1) add();
    }

    int ans = 1;
    for (int i = 0; i < n/B; i++) ans = ans*f[i]%mod;
    for (int i = 0; i < n%B; i++) ans = ans*( n/B*B+i+1 )%mod;
    return ans;
}
```

Power Projection: 基础版

用途: 对 $h(i) = [x^N]f(x)g(x)^i$, 批量求出 $i = 0, 1, \dots, n$ 的结果。

复杂度: 主要由若干轮二元卷积、一次多项式求逆和一次一元卷积构成; 在快速卷积模型下可按 $\tilde{O}(N + n)$ 理解, 具体常数取决于二元卷积的压缩长度。

依赖: 二元多项式卷积、多项式求逆、阶乘与逆阶乘。

```
vector<int> powerProjection(vector<int> f, vector<int> g, int n) {
    int C = g[0];
    g[0] = 0;

    if (f.size() < g.size()) f.resize(g.size());
    int N = g.size()-1;
    vector P(f.size(), vector<int>(2)), Q(g.size(), vector<int>(2));
    for (int i = 0; i < (int)f.size(); i++) P[i][0] = f[i];
    Q[0][0] = 1;
    for (int i = 0; i < (int)g.size(); i++) Q[i][1] = (mod-g[i])%mod;
    int i;
    for (; N; N /= 2) {
        auto H = Q;
        for (i = 1; i < (int)H.size(); i+=2) for (auto &x : H[i]) if (x) x = mod-x;
        P *= H, Q *= H;
        for (i = 0; i*4 < (int)P.size(); i++) P[i].swap(P[i*2+N%2]);
        P.resize(i);
        for (i = 0; i*4 < (int)Q.size(); i++) Q[i].swap(Q[i*2]);
        Q.resize(i);
    }

    Q[0].resize(n+1);
    P[0] *= inv(Q[0]);
    P[0].resize(n+1);

    vector<int> A(n+1);
    for (int i = 0, j = 1; i <= n; i++, j = j*C%mod) A[i] = j*ifac(i)%mod, P[0][i] = P[0][i]*ifac(i)%mod;
    A *= P[0];
    A.resize(n+1);
    for (int i = 0; i <= n; i++) (A[i] *= fac(i)) %= mod;

    return A;
}
```

Power Projection: 卡常版

用途：与基础版相同，计算 $h(i) = [x^N]f(x)g(x)^i$ ，但将核心消元与 NTT 排布融合，减少通用二元卷积的常数。

复杂度：快速卷积模型下为准线性量级，通常写作 $\tilde{O}(N + n)$ ；空间 $O(N + n)$ 。

使用条件：强依赖卡常 NTT 中的根 31、fac/ifac 和模数 998244353；核心还会使用 $2 \cdot \text{len}$ 次单位根，因此这里的 len 至多为 2^{22} 。

```
vector<int> powerProjection(vector<int> f, vector<int> g, int n) {
    auto pp0 = [&](vector<int> wt, vector<int> h, int m) -> vector<int> {
        int len = 1;
        while (len < (int)h.size()) len <= len * 2;
        h.resize(len), wt.resize(len);
        for (auto &x : h) if (x) x = mod - x;
        reverse(wt.begin(), wt.end());

        vector<int> P = wt, Q = h;
        P.resize(4 * len), Q.resize(4 * len);

        vector<int> W(len);
        if (len > 1) {
            vector<int> rev(len);
            int lg = __lg(len);
            for (int i = 0; i < len; i++) rev[i] = (rev[i >> 1] >> 1) | ((i & 1) << (lg - 1));
            int dw = qpow(inv(31), (1 << 23) / (2 * len)), w = 1;
            for (auto &i : rev) W[i] = w, w = w * dw % mod;
        } else W[0] = 1;

        for (int k = 1, cur = len; cur > 1; cur >>= 1, k <= 1) {
            vector<int> fy(k), hx(2 * cur), F(2 * cur), G(2 * cur), f(cur), g(cur);

            auto dy = [&](vector<int> &A, int l, int r) {
                int z = inv(W[k >> 1]);
                for (int i = l; i < r; i++) {
                    for (int j = 0; j < k; j++) fy[j] = A[2 * cur * j + i];
                    ntt(fy, k, -1);
                    for (int j = 1, w = z; j < k; j++, w = w * z % mod) fy[j] = fy[j] * w % mod;
                    ntt(fy, k, 1);
                    for (int j = 0; j < k; j++) A[2 * cur * (k + j) + i] = fy[j];
                }
            };

            auto fx = [&](vector<int> &A, int l, int r) {
                for (int j = l; j < r; j++) {
                    for (int i = 0; i < 2 * cur; i++) hx[i] = A[2 * cur * j + i];
                    ntt(hx, 2 * cur, 1);
                    for (int i = 0; i < 2 * cur; i++) A[2 * cur * j + i] = hx[i];
                }
            };
        }
    };
}
```

```

        if (cur <= k) {
            dy(P, 0, cur), dy(Q, 1, cur);
            fx(P, 0, 2*k), fx(Q, 0, 2*k);
        } else {
            fx(P, 0, k), fx(Q, 0, k);
            dy(P, 0, 2*cur), dy(Q, 0, 2*cur);
        }

        for (int i = 0; i < 2*cur*k; i++) if (++Q[i] == mod) Q[i] = 0;
        for (int i = 2*cur*k; i < 4*cur*k; i++) if (--Q[i] < 0) Q[i] += mod;

        for (int j = 0; j < 2*k; j++) {
            for (int i = 0; i < 2*cur; i++) F[i] = P[2*cur*j+i], G[i] = Q[2*cur*j+i];
            for (int i = 0; i < cur; i++) {
                f[i] = W[i]*((F[2*i]*G[2*i+1]-F[2*i+1]*G[2*i])%mod+mod)%mod;
                g[i] = G[2*i]*G[2*i+1]%mod;
            }
            ntt(f, cur, -1), ntt(g, cur, -1);
            fill(f.begin()+cur/2, f.end(), 0), fill(g.begin()+cur/2, g.end(), 0);
            for (int i = 0; i < cur; i++) P[cur*j+i] = f[i], Q[cur*j+i] = g[i];
        }

        fill(P.begin()+2*cur*k, P.end(), 0), fill(Q.begin()+2*cur*k, Q.end(), 0);
        for (int j = 0; j < 4*k; j++) Q[cur*j] = 0;
    }

    int k = len;
    for (int i = 0; i < k; i++) P[i] = P[2*i];
    P.resize(k);

    int c = inv(k);
    for (auto &x : P) x = x*c%mod;
    ntt(P, k, -1);
    reverse(P.begin(), P.end());
    P.resize(m+1);
    return P;
};

auto pp = [&](auto &&self, vector<int> wt, vector<int> h, int m) -> vector<int> {
    if (wt.size() < h.size()) wt.resize(h.size());
    else if (h.size() < wt.size()) h.resize(wt.size());

    if (h.empty()) return vector<int>(m+1, 0);

    if (h[0]) {
        int c = h[0];
        h[0] = 0;
        auto A = self(self, wt, h, m);
        for (int i = 0; i <= m; i++) A[i] = A[i]*ifac(i)%mod;

        vector<int> B(m+1);
        for (int i = 0, pw = 1; i <= m; i++, pw = pw*c%mod) B[i] = pw*ifac(i)%mod;
    }
}

```

```

        A *= B, A.resize(m+1);
        for (int i = 0; i <= m; i++) A[i] = A[i]*fac(i)%mod;
        return A;
    }

    if (h.size() == 1) {
        vector<int> A(m+1);
        A[0] = wt[0];
        return A;
    }

    return pp0(wt, h, m);
};

if (f.size() < g.size()) f.resize(g.size());
else if (g.size() < f.size()) g.resize(f.size());

vector<int> wt(f.size());
for (int i = 0; i < (int)f.size(); i++) wt[i] = f[(int)f.size()-1-i];
return pp(pp, wt, g, n);
}

```

最短递推式

用途：对模 mod 的域上的有限序列 $a_0, a_1, \dots, a_{n-1}$ ，求与该序列相容的最短线性递推式；短序列用 Berlekamp–Massey，长序列可用 Half-GCD 快速版本。

返回值：返回最短递推多项式 C ，其中 $C[0]=1$ ，阶数 $L = C.\text{size}() - 1$ ，且对 $L \leq i < n$ 有 $a_i = -\sum_{j=1}^L C_j a_{i-j} \pmod{\text{mod}}$ 。

复杂度：普通版本时间 $O(nL)$ ，最坏为 $O(n^2)$ ，空间 $O(L)$ ；快速版本在 NTT 卷积下时间 $O(n \log^2 n)$ 。

使用条件： mod 下非零数均可逆（通常取质数）， $a[i]$ 已归一化到 $[0, \text{mod})$ ；依赖模逆 inv 。仓库骨架中 int 为 i64 ，当 mod 小于 2^{31} 时乘积仍在 i64 内；更大的模数需用 i128 完成乘法取模。

例如 Fibonacci 前缀返回 $C = (1, -1, -1)$ （各项对 mod 取模），表示 $a_i - a_{i-1} - a_{i-2} = 0$ 。

```
vector<int> berlekampMassey(vector<int> a) {
    int n = a.size();
    vector<int> C = {1}, B = {1};
    int L = 0, b = 1, m = 1;
    for (int i = 0; i < n; i++) {
        int d = 0;
        for (int j = 0; j <= L; j++) (d += C[j]*a[i-j]) %= mod;
        if (d == 0) {
            m++;
            continue;
        }
        auto tmp = C;
        int k = d*inv(b)%mod;
        if (B.size()+m > C.size()) C.resize(B.size()+m);
        for (int j = 0; j < (int)B.size(); j++) {
            (C[j+m] -= k*B[j]) %= mod;
            if (C[j+m] < 0) C[j+m] += mod;
        }
        if (2*L <= i) L = i+1-L, B = tmp, b = d, m = 1;
        else m++;
    }
    C.resize(L+1);
    return C;
}
```

Half-GCD 快速版本

将序列转成 Padé 逼近问题，再用 Half-GCD 加速多项式扩展欧几里得。返回值约定与普通版本相同。

依赖多项式加减、乘法、求逆与整除。

```
vector<int> berlekampMasseyFast(vector<int> a) {
    using Arr = array<vector<int>, 2>;
    using Mat = array<vector<int>, 4>;
    auto norm = [&](vector<int> a) {
        while (a.size() && a.back() == 0) a.pop_back();
        return a;
    };
}
```

```

};

auto mul = [&](Mat A, Mat B) {
    return Mat{norm(A[0]*B[0]+A[1]*B[2]), norm(A[0]*B[1]+A[1]*B[3]),
               norm(A[2]*B[0]+A[3]*B[2]), norm(A[2]*B[1]+A[3]*B[3])};
};

auto apply = [&](Mat A, Arr b) {
    return Arr{norm(A[0]*b[0]+A[1]*b[1]), norm(A[2]*b[0]+A[3]*b[1])};
};

auto step = [&](vector<int> q) {
    Mat Q;
    Q[1] = {1}, Q[2] = {1};
    Q[3] = norm(vector<int>()-q);
    return Q;
};

auto halfGcd = [&](auto &&self, Arr a) -> Mat {
    int m = a[0].size()/2;
    if ((int)a[1].size() <= m) {
        Mat I;
        I[0] = {1}, I[3] = {1};
        return I;
    }
    Arr h = {vector<int>(a[0].begin()+m, a[0].end()),
             vector<int>(a[1].begin()+m, a[1].end())};
    Mat R = self(self, h);
    a = apply(R, a);
    if ((int)a[1].size() <= m) return R;
    Mat Q = step(norm(a[0]/a[1]));
    R = mul(Q, R), a = apply(Q, a);
    if ((int)a[1].size() <= m) return R;
    int k = 2*m+1-a[0].size();
    h = {vector<int>(a[0].begin()+k, a[0].end()),
         vector<int>(a[1].begin()+k, a[1].end())};
    return mul(self(self, h), R);
};

int n = a.size();
reverse(a.begin(), a.end()), a = norm(a);
if (a.empty()) return {1};
vector<int> x(n+1);
x[n] = 1;
Mat M = halfGcd(halfGcd, Arr{x, a});
Arr r = apply(M, Arr{x, a});
if ((int)r[1].size() > n-(int)r[0].size()+1) {
    M = mul(step(norm(r[0]/r[1])), M);
}
auto C = M[3];
int iv = inv(C.back());
for (auto &x : C) x = x*iv%mod;
reverse(C.begin(), C.end());
return C;
}

```

常系数齐次线性递推

用途：已知 k 阶常系数齐次线性递推及初值，求下标很大的第 N 项；将递推转成生成函数后，调用“分式第 N 项”。

复杂度：使用快速卷积时为 $O(k \log k \log N)$ ，空间 $O(k)$ 。

使用条件：运算在域上进行； k 为实际阶数，即 $c_k \neq 0$ 。已知 $a_0, \dots, a_{k-1}$ ，并约定递推对所有 $n \geq k$ 成立。

设递推与初值多项式分别为

$$a_n = \sum_{j=1}^k c_j a_{n-j}, \quad A(x) = \sum_{i=0}^{k-1} a_i x^i.$$

构造递推多项式

$$C(x) = 1 - \sum_{j=1}^k c_j x^j.$$

则第 N 项就是

$$a_N = [x^N] \frac{A(x)C(x) \bmod x^k}{C(x)}.$$

即先计算分子 $P(x) = A(x)C(x) \bmod x^k$ ，再用“分式第 N 项”计算 $[x^N]P(x)/C(x)$ 。若 $C(x)$ 来自上一章 Berlekamp–Massey 的返回值，可直接使用。

顺序	模板	用途	复杂度提示
01	i128 重载	输入输出与常用整数函数	输入输出 $O(L)$, GCD $O(\log V)$
02	datetime	日期、时间、格式化与解析	属性读取 $O(1)$, 解析 $O(L)$
03	Fraction	精确有理数	取决于大整数位数与 GCD
04	math	常用数学函数	标量函数通常视为 $O(1)$

i128 输入输出与常用函数重载

用途： 为 i128 补充十进制流输入输出，以及 abs、gcd、lcm、to_string 等常用接口。

依赖： GCC 或 Clang 的 __int128。

兼容： 旧版 libstdc++ 在严格标准模式下可能没有 i128 的 abs 重载，而 GNU 模式下可能已有同签名重载；这里用函数模板按需补充，避免直接重载产生冲突。

范围： 输入须为 i128 范围内的合法十进制整数。abs、gcd、lcm 的参数不能为 -2^{127} ，结果须能由 i128 表示。

操作	含义	复杂度	备注
cin >> x、cout << x	读写十进制 i128	$O(L)$	输入支持正负号， L 为位数
to_string(x)	转为十进制字符串	$O(L)$	支持 i128 最小值
abs(x)	返回绝对值	$O(1)$	参数不能为 -2^{127}
gcd(a,b)	返回非负最大公约数	$O(\log \min(a , b))$	gcd(0,0)=0
lcm(a,b)	返回非负最小公倍数	$O(\log \min(a , b))$	任一参数为 0 时返回 0

```
using i128 = __int128;
using u128 = unsigned __int128;

istream &operator>>(istream &is, i128 &x) {
    string s;
    if (!(is >> s)) return is;
    bool neg = s[0] == '-';
    size_t p = s[0] == '-' || s[0] == '+';
    x = 0;
    for (; p < s.size(); p++)
        x = x*10 + (neg ? '0'-s[p] : s[p]-'0');
    return is;
}

string to_string(i128 x) {
    bool neg = x < 0;
    u128 y = x;
    if (neg) y = -y;
    string s;
    do s += y%10+'0', y /= 10; while (y);
    if (neg) s += '-';
    reverse(s.begin(), s.end());
    return s;
}

ostream &operator<<(ostream &os, i128 x) {
    return os << to_string(x);
}

i128 abs2(i128 x) {
    return x < 0 ? -x : x;
}
```

```
template<class T>
enable_if_t<is_same_v<T, i128>, i128> abs(T x) {
    return abs2(x);
}

i128 gcd(i128 a, i128 b) {
    a = abs(a), b = abs(b);
    while (b) a %= b, swap(a, b);
    return a;
}

i128 lcm(i128 a, i128 b) {
    if (!a || !b) return 0;
    return abs(a/gcd(a, b)*b);
}
```

Python `datetime` 库

用途： 构造、读取、格式化和解析日期、时间与日期时间对象。

复杂度： 固定字段的构造和属性读取可视为 $O(1)$ ；格式化、解析为 $O(L)$ ，其中 L 是字符串长度。

`date`：只表示年月日

```
from datetime import date

today = date.today()
print(today)          # 例如: 2026-03-12
print(today.year)     # 年
print(today.month)    # 月
print(today.day)      # 日

d = date(2026, 3, 12)
print(d)
print(d.strftime("%Y/%m/%d")) # 2026/03/12

d2 = date.fromisoformat("2026-03-12")
print(d2)
```

`time`：只表示时分秒

```
from datetime import time

t = time(14, 30, 45)
print(t)          # 14:30:45
print(t.hour)     # 时
print(t.minute)   # 分
print(t.second)   # 秒

t = time(14, 30, 45, 123456)
print(t)
print(t.strftime("%H:%M:%S")) # 14:30:45

t2 = time.fromisoformat("14:30:45")
print(t2)
```

`datetime`：同时表示日期和时间

```
from datetime import datetime

dt = datetime(2026, 3, 12, 14, 30, 45)
print(dt) # 2026-03-12 14:30:45

now = datetime.now()
print(now)
print(dt.strftime("%Y-%m-%d %H:%M:%S")) # 2026-03-12 14:30:45

dt2 = datetime.strptime("2026-03-12 14:30:45", "%Y-%m-%d %H:%M:%S")
```

```
print(dt2)

d = dt.date() # 提取 date
t = dt.time() # 提取 time
print(d, t)

dt3 = datetime.combine(d, t) # 合并回 datetime
print(dt3)
```

提示： `date` 只含日期，`time` 只含时间；需要同时处理两者时使用 `datetime`。

Python Fraction 库

用途：精确表示和运算有理数，适合比例计算、配方换算以及需要避免浮点误差的场景。

复杂度：取决于分子、分母的大整数位数；构造和四则运算通常还包含 GCD 约分，不能简单视为固定 $O(1)$ 。

创建、约分与属性

```
from fractions import Fraction

a = Fraction(3, 4)          # 3/4
b = Fraction("2/5")         # 2/5
c = Fraction("0.125")       # 1/8（推荐用字符串）
print(a, b, c)

f = Fraction(6, 8)
print(f) # 3/4
print(f.numerator) # 分子
print(f.denominator) # 分母
```

常见运算

```
x = Fraction(1, 3)
y = Fraction(1, 6)

print(x + y) # 1/2
print(x - y) # 1/6
print(x * y) # 1/18
print(x / y) # 2
```

limit_denominator()

`Fraction.limit_denominator(max_denominator=1000000)` 返回最接近当前值且分母不超过正整数上限 `max_denominator` 的新分数；上限越小，结果通常越简洁，但误差可能增加。

```
pi_like = Fraction("3.14159")
print(pi_like.limit_denominator(100)) # 311/99
print(pi_like.limit_denominator(1000)) # 355/113
```

与 float、Decimal 互转

```
from decimal import Decimal

f1 = Fraction.from_float(0.5) # 1/2
f2 = Fraction.from_decimal(Decimal("1.25")) # 5/4

print(float(Fraction(3, 8))) # 0.375
print(f1, f2)
```

提示： 需要精确分数时优先使用 `Fraction`，尽量用整数或字符串构造；展示时可用 `limit_denominator()` 得到更直观的近似分数。

Python `math` 库常用函数

用途： 提供取整、初等函数、三角函数、组合计数、距离和高精度浮点求和。

复杂度： 普通浮点标量函数通常可视为常数时间；`factorial`、`comb`、`perm` 的开销随输入值和大整数位数增长；`fsum` 对 n 个元素为 $O(n)$ 。

取整、幂与对数

```
import math

print(math.ceil(3.2))    # 向上取整: 4
print(math.floor(3.8))   # 向下取整: 3
print(math.trunc(-3.9))  # 截断取整: -3
print(math.fabs(-2.5))   # 绝对值: 2.5

print(math.sqrt(9))      # 平方根: 3.0
print(math.pow(2, 3))    # 2^3: 8.0
print(math.exp(1))       # e^1
print(math.log(8, 2))    # 以2为底的对数: 3.0
print(math.log10(1000))  # 常用对数: 3.0
```

三角函数与角度转换

```
rad = math.radians(180)  # 角度 -> 弧度
deg = math.degrees(math.pi) # 弧度 -> 角度
print(rad, deg)

print(math.sin(math.pi / 2)) # 1.0
print(math.cos(0))           # 1.0
print(math.tan(math.pi / 4)) # 1.0 (近似)

print(math.pi)              # 圆周率
print(math.e)               # 自然常数 e
```

阶乘、最大公约数、组合数

```
print(math.factorial(5)) # 120
print(math.gcd(12, 18))  # 6
print(math.comb(5, 2))   # 10 (组合数)
print(math.perm(5, 2))   # 20 (排列数)
```

距离与求和

```
print(math.hypot(3, 4))    # 5.0, 勾股计算
print(math.fsum([0.1, 0.1, 0.1])) # 高精度浮点求和
```

提示： 几何、对数和三角函数优先使用 `math`，高精度浮点累加使用 `math.fsum()`；金额计算应使用 `Decimal`，不要依赖二进制浮点。